

MATH70102

UNSUPERVISED LEARNING

Mikko Pakkanen

Department of Mathematics, Imperial College London

Lecture notes

Updated: 20 April 2026



Contents

1	Dimensionality reduction	5
1.1	Rationale for dimensionality reduction	5
1.2	Principal component analysis	7
1.3	Sparse principal component analysis	12
1.4	Kernel principal component analysis	13
1.5	Autoencoder	16
1.6	Canonical correlation analysis	17
1.7	Non-negative matrix factorisation	20
1.8	Independent component analysis	22
1.9	Multidimensional scaling and non-linear dimensionality reduction methods	26
2	Density estimation	35
2.1	Overview of density estimation	35
2.2	Parametric density estimation	36
2.3	Estimation of mixtures	39
2.4	Histogram	43
2.5	Kernel density estimation	47
3	Clustering	52
3.1	K -means clustering	52
3.2	K -medoids clustering	54
3.3	Hierarchical clustering	57
3.4	Evaluation measures	60
4	Anomaly and outlier detection	68
4.1	Anomalies and outliers	68
4.2	Local outlier factor	69
4.3	One-class support vector machine	71
	Bibliography	75

Preface

Welcome to MATH70102 UNSUPERVISED LEARNING! These lecture notes cover the methodological and theoretical contents of the module. We will generally not look into practical application of the methods in these notes since this aspect will be covered in the Jupyter notebooks that will be distributed alongside.

The overall structure and organisation of the contents mirror an earlier set of notes prepared by my colleague Dr Marina Evangelou. However, the current exposition is my own — hence I am solely responsible for any inaccuracies or omissions. As references I have used [Hastie et al. \(2009\)](#) and, to a lesser extent, [Bishop \(2006\)](#), plus a number of original articles that are cited along the way.

I may make minor updates to these notes while the module is in progress. (A changelog is kept on the next page.) As a result, the page/equation/figure numbers may evolve slightly. But I expect that the section numbers remain stable.

Any comments or reports of misprints (even if you are unsure) are welcome. To this end, please contact me by email (m.pakkanen@imperial.ac.uk) or via the Ed discussion forum.

I hope you will find the module interesting and rewarding!

Mikko

Changelog

- ★ 18 April 2024 — First release
- ★ 25 April 2025 — Added graphics, corrected typos, rewrote the section on non-linear dimensionality reduction
- ★ 9 May 2025 — Added a reference to the derivation of the modified Gram matrix for kernel PCA under centring

1 Dimensionality reduction

1.1 Rationale for dimensionality reduction

1.1.1 Curse of dimensionality

To motivate *dimensionality reduction* methods, we shall briefly review the phenomenon known as the *curse of dimensionality*, first identified by Bellman (1957) in the context of dynamic programming.

Suppose that our task is to analyse a data set consisting of d -dimensional samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$. The immediate difficulties that arise from large dimensionality d are:

- ★ Difficulty of *visualising* the data (in terms of dependence and distribution).
- ★ Costly *storage* of data, especially if both d and n are large.
- ★ Meaningful measurement of *distances*.

But the issues that arise from large dimensionality may also be more subtle:

- ★ *Computational cost* — many numerical algorithms are at least *quadratic* in *computational complexity* in terms of dimensionality, that is, the number of operations they require grows at least as fast as d^2 . (*Multiplication of square matrices* and *solving systems of linear equations* are examples.) Then the impact of having, say, $d = 100$ versus $d = 10$ can be significant, if not enormous.

Sometimes large dimensionality may prevent us from using a particular method altogether:

Example 1.1. If the dimensionality d exceeds the number of samples, n , then it is possible to show that the *sample covariance matrix* of $\mathbf{x}_1, \dots, \mathbf{x}_n$ is *not invertible*. Any method that requires the inverse of the sample covariance matrix, which is known as the *sample precision matrix*, becomes inapplicable. (For example, *weighted least-squares estimation*.)

1.1.2 Intrinsic dimensionality and latent variables

When working with multivariate data sets, we often observe that some of the variables in the data set are:

- ★ *irrelevant* — they do not contain useful information concerning the problem we are studying (they may be just “noise”),

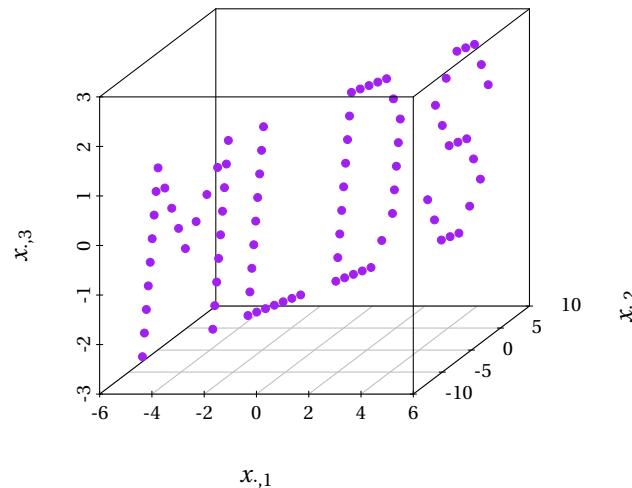


Figure 1.1: A data set of points that live in a two-dimensional linear subspace of $\mathbb{R}^3$. If the orientation and location of the points are immaterial, the intrinsic dimensionality of the data set is two.

★ *redundant* — they contain information that is already conveyed by other variables in the data set.

It is also possible that the observed variables are in fact governed by a smaller set of unobservable *latent variables*.

In dimensionality reduction, we are thus often interested in determining the *intrinsic dimensionality* of the data set, which is the dimensionality of a minimal set of variables that describe the essential information and variation present in the data. While determining the intrinsic dimensionality, we seek to *remove* the irrelevant and redundant variables and/or to *recover* the underlying latent variables.

Remark 1.2. Note, however, that the concept of intrinsic dimensionality is *not entirely objective*. What is “essential” about the data is context-dependent and subjective (cf. Figure 1.1). As a rule, dimensionality reduction is “lossy”. It is then important to determine whether the lost information is essential or not.

If the data are low dimensional to begin with or if the nature of the data is well understood, it may be possible to perform dimensionality reduction manually/subjectively by simply *removing* the redundant or irrelevant component in the data. But generally this is not feasible, and we need proper, *data-driven* algorithms for dimensionality reduction.

1.2 Principal component analysis

1.2.1 Formulation

Principal component analysis (PCA) is a method to linearly transform multivariate data to new *orthogonal* variables, called *principal components* (PCs). While fundamentally it is a data transformation/rotation method, it lends itself to dimensionality reduction when we discard some of the less important principal components. We will discuss this in Section 1.2.2. PCA was invented by *Pearson* (1901) and later independently re-discovered by *Hotelling* (1933).

There are many formulations of PCA, the most common of which is perhaps the *maximum variance* formulation, which we look into first. To this end, consider d -dimensional data consisting of samples $\mathbf{x}_i \in \mathbb{R}^d$, $i = 1, \dots, n$.¹ Their sample mean and covariance matrix are defined by

$$\bar{\mathbf{x}} := \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \in \mathbb{R}^d, \quad \Sigma := \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \in \mathbb{R}^{d \times d},$$

respectively.² A generic vector $\mathbf{c} \in \mathbb{R}^d$ defines a projection of the data via $\mathbf{c}^T \mathbf{x}_i \in \mathbb{R}$, $i = 1, \dots, n$. The sample variance of the projected data $\mathbf{c}^T \mathbf{x}_i \in \mathbb{R}$, $i = 1, \dots, n$, is then

$$V(\mathbf{c}) := \frac{1}{n} \sum_{i=1}^n (\mathbf{c}^T \mathbf{x}_i - \mathbf{c}^T \bar{\mathbf{x}})^2 = \mathbf{c}^T \Sigma \mathbf{c} \in \mathbb{R}_{\geq 0}.$$

It does not make sense to maximise $V(\mathbf{c})$ over all $\mathbf{c} \in \mathbb{R}^d$ — we can typically make $V(\mathbf{c})$ arbitrarily large by taking $\|\mathbf{c}\| \rightarrow \infty$. Hence, we introduce a normalising constraint $\|\mathbf{c}\| = 1$, which is equivalent to $\|\mathbf{c}\|^2 = \mathbf{c}^T \mathbf{c} = 1$. The constrained maximisation problem

$$\operatorname{argmax}_{\mathbf{c} \in \mathbb{R}^d} V(\mathbf{c}) \quad \text{such that} \quad \mathbf{c}^T \mathbf{c} = 1 \tag{1.3}$$

can be solved using the *Lagrangian function*

$$\mathcal{L}(\mathbf{c}, \lambda) := V(\mathbf{c}) + \lambda(1 - \mathbf{c}^T \mathbf{c}) = \mathbf{c}^T \Sigma \mathbf{c} + \lambda(1 - \mathbf{c}^T \mathbf{c})$$

with Lagrange multiplier $\lambda \in \mathbb{R}$.

Differentiating $\mathcal{L}(\mathbf{c}, \lambda)$ with respect to $\mathbf{c}$, i.e., working out the gradient

$$\nabla_{\mathbf{c}} \mathcal{L}(\mathbf{c}, \lambda) = 2\Sigma \mathbf{c} - 2\lambda \mathbf{c},$$

¹Application of PCA may become problematic if the dimensionality d is approximately equal to or exceeds the number of samples n . For more information, see, e.g., *Johnstone and Paul* (2018).

²Alternatively, normalisation by $\frac{1}{n-1}$ instead of $\frac{1}{n}$ could be used for the sample covariance matrix but, apart from cases where n is small, the difference is insignificant in practice.

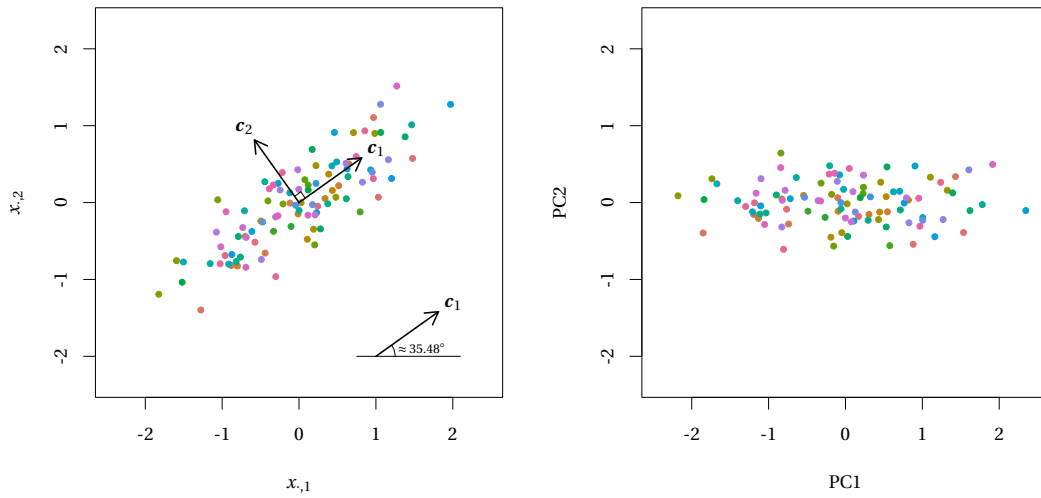


Figure 1.2: PCA applied to a data set in $\mathbb{R}^2$. The PCs on the right are obtained by rotating the samples on the left, which are centred, clockwise by the angle of the first loadings vector $\mathbf{c}_1$, which is approximately 35.48° .

we find that a maximiser $(\mathbf{c}_1, \lambda_1)$ of the Lagrangian must necessarily solve

$$\nabla_{\mathbf{c}} \mathcal{L}(\mathbf{c}_1, \lambda_1) = 0 \iff \Sigma \mathbf{c}_1 = \lambda_1 \mathbf{c}_1.$$

Hence, $\mathbf{c}_1$ is an *eigenvector* of Σ , and λ_1 is the corresponding *eigenvalue*. Subsequently, such a maximiser $\mathbf{c}_1$ defines the first PC of the data (with centring) via $\mathbf{c}_1^T (\mathbf{x}_i - \bar{\mathbf{x}})$, $i = 1, \dots, n$. Note that the sample variance of the first PC is

$$V(\mathbf{c}_1) = \mathbf{c}_1^T \Sigma \mathbf{c}_1 = \lambda_1 \mathbf{c}_1^T \mathbf{c}_1 = \lambda_1.$$

The vectors $\mathbf{c}_2, \dots, \mathbf{c}_d$ determining the remaining PCs are defined similarly in an inductive fashion. Suppose that we have already worked out the vectors $\mathbf{c}_1, \dots, \mathbf{c}_{j-1}$ for some $j = 2, \dots, n$. Then $\mathbf{c}_j$ is obtained as a maximiser of the sample variance $V(\mathbf{c})$ under the constraint $\mathbf{c}^T \mathbf{c} = 1$ but with an additional constraint that $\mathbf{c}$ is *orthogonal* to the vectors $\mathbf{c}_1, \dots, \mathbf{c}_{j-1}$. That is, $\mathbf{c}_j$ solves

$$\operatorname{argmax}_{\mathbf{c} \in \mathbb{R}^d} V(\mathbf{c}) \quad \text{such that} \quad \underbrace{\mathbf{c}^T \mathbf{c} = 1}_{\text{normalisation}} \quad \text{and} \quad \underbrace{\mathbf{c}^T \mathbf{c}_k = 0 \text{ for any } k < j}_{\text{orthogonality}}. \quad (1.4)$$

This optimisation problem can again be solved using a Lagrangian function. It then transpires that $\mathbf{c}_j$ is also an eigenvector of Σ and that the Lagrange multiplier λ_j for the normalisation constraint $\mathbf{c}^T \mathbf{c} = 1$ is the matching eigenvalue. The j -th PC is then defined as $\mathbf{c}_j^T (\mathbf{x}_i - \bar{\mathbf{x}})$, $i = 1, \dots, n$. Moreover, $V(\mathbf{c}_j) = \lambda_j$. Note that the

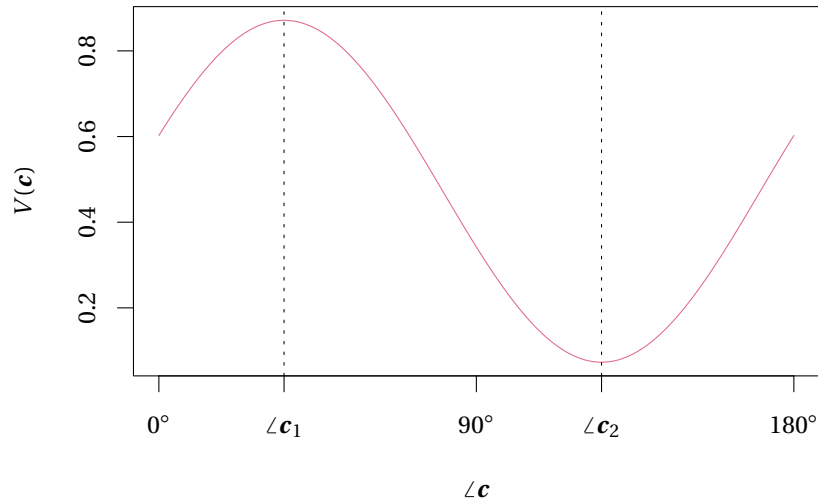


Figure 1.3: The sample variance $V(\mathbf{c})$ of the data set of Figure 1.2 projected onto vector $\mathbf{c} \in \mathbb{R}^2$ such that $\|\mathbf{c}\| = 1$. The vector $\mathbf{c}$ is parameterised by its angle $\angle \mathbf{c}$ in $\mathbb{R}^2$. Note that the highest variance is indeed obtained at the angle of the first loadings vector $\mathbf{c}_1$, while $\mathbf{c}_2$ is constrained to have a 90° -angle to $\mathbf{c}_1$.

orthogonality constraint $\mathbf{c}^T \mathbf{c}_k = 0$ for any $k < j$ is the more restrictive the larger j is. Thus, we are seeking the maximiser $\mathbf{c}_j$ of $V(\mathbf{c})$ from a progressively *smaller subspace* of $\mathbb{R}^d$ as j increases. Accordingly,

$$\underbrace{V(\mathbf{c}_1)}_{=\lambda_1} \geq \underbrace{V(\mathbf{c}_2)}_{=\lambda_2} \geq \cdots \geq \underbrace{V(\mathbf{c}_d)}_{=\lambda_d}. \quad (1.5)$$

This indicates that the first PC has the *largest sample variance*, the second PC has the *second largest sample variance* and so on. For an illustration of PCA in $\mathbb{R}^2$, see Figures 1.2 and 1.3.

Remark 1.6. Since we know that $\mathbf{c}_1, \dots, \mathbf{c}_d$ are eigenvectors of Σ and $\lambda_1, \dots, \lambda_d$ are the matching eigenvalues, to perform PCA in practice, we do not have to solve (1.3) or (1.4) numerically. Instead, we can use numerical eigendecomposition algorithms, or the *singular value decomposition* (SVD), to find $\mathbf{c}_1, \dots, \mathbf{c}_d$ and $\lambda_1, \dots, \lambda_d$.

Remark 1.7. The entries of the vectors $\mathbf{c}_1, \dots, \mathbf{c}_d$ are sometimes called *loadings*. However, this term is not used consistently (the term has been overloaded if you pardon the pun), as it may also be used to refer to the entries of $\sqrt{\lambda_1} \mathbf{c}_1, \dots, \sqrt{\lambda_d} \mathbf{c}_d$. In these lectures, we adopt the former interpretation when we use the word “loading”.

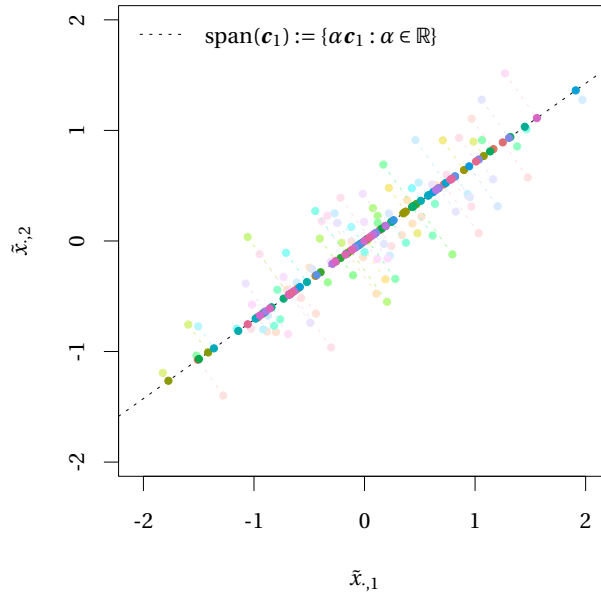


Figure 1.4: The data set of Figure 1.2 reconstructed from its first PC. In this case reconstruction entails projecting the samples orthogonally onto the linear space spanned by the first loadings vector $\mathbf{c}_1$.

1.2.2 Dimensionality reduction

As mentioned in passing above, we can use PCA for dimensionality reduction by discarding PCs deemed negligible. In the context of PCA, PCs with *low enough sample variance* may be regarded as insignificant. In view of (1.5), we select $q = 1, \dots, d-1$ such that the sample variance $V(\mathbf{c}_{q+1}) = \lambda_{q+1}$ of the $q+1$ -st PC is small enough. We then retain the PCs up to the q -th PC (inclusive) and discard the rest. While intuitively plausible, this procedure can also be formalised via *reconstruction error minimisation* as follows.

Suppose that we seek to approximate the original samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ by new vectors $\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n$ in $\mathbb{R}^d$ which take specific form,

$$\hat{\mathbf{x}}_i := \boldsymbol{\mu} + U_q \mathbf{z}_i,$$

where $\boldsymbol{\mu} \in \mathbb{R}^d$ is a location vector, $U_q \in \mathbb{R}^{d \times q}$ is a matrix whose columns are orthogonal vectors with unit norm and $\mathbf{z}_i \in \mathbb{R}^q$ for any $i = 1, \dots, n$. Then we seek to minimise the (mean-square) *reconstruction error*

$$\frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu} - U_q \mathbf{z}_i\|^2 \quad (1.8)$$

over $\boldsymbol{\mu}$, U_q and $\mathbf{z}_1, \dots, \mathbf{z}_n$ satisfying the above requirements. So, effectively we are trying to recover $\mathbf{x}_1, \dots, \mathbf{x}_n$ optimally from the lower-dimensional data $\mathbf{z}_1, \dots, \mathbf{z}_n$.

It can be shown (Bishop, 2006, Sect. 12.1.2) that (1.8) is minimised by

$$\begin{aligned}\boldsymbol{\mu} &= \bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i, \\ U_q &= [\mathbf{c}_1 \cdots \mathbf{c}_q] \\ \mathbf{z}_i &= U_q^T (\mathbf{x}_i - \bar{\mathbf{x}}), \quad i = 1, \dots, n,\end{aligned}$$

where we interpret all vectors as column vectors. Note that the j -th component of $\mathbf{z}_i$ (for $j \in 1, \dots, q$) is the j -th PC of the i -th sample, i.e., $\mathbf{c}_j^T (\mathbf{x}_i - \bar{\mathbf{x}})$. Thus, the first q PCs provide an optimal q -dimensional description of the original data in the above sense. For an illustration of reconstruction in $\mathbb{R}^2$, see Figure 1.4.

1.2.3 Selecting dimensionality

The number q of PCs for the lower-dimensional description of the data is usually chosen based on an analysis of the sample variances of the PCs, i.e., $V(\mathbf{c}_j) = \lambda_j$, $j = 1, \dots, d$. Specifically, we can compare them to the *total variance* given by the sum of the variances of all PCs,

$$\sum_{j=1}^d V(\mathbf{c}_j) = \sum_{j=1}^d \lambda_j.$$

Plotting the ratio

$$\frac{\lambda_j}{\sum_{j'=1}^d \lambda_{j'}},$$

interpreted as the proportion of *variance explained* by the j -th PC, across all $j = 1, \dots, d$, yields the *scree plot*. Alternatively, we can also look at the cumulative variance of the first q PCs via the ratio

$$\frac{\sum_{j=1}^q \lambda_j}{\sum_{j'=1}^d \lambda_{j'}}, \quad q = 1, \dots, d,$$

and choose q such that the ratio exceeds a level deemed sufficiently high.

Remark 1.9. There are no hard and fast rules about how much variance the chosen q PCs ought to explain. The so-called *elbow method* suggests that we look for q after which the scree plot levels off — an “elbow”. This heuristic has two caveats. Firstly, we may not always find a clear elbow, or there may be multiple elbows. Secondly, if the data are constrained to a proper linear subspace of $\mathbb{R}^d$, then it may be preferable to look for $q < d$ with nearly 100% explained variance instead of settling for an elbow at a lower level.

1.3 Sparse principal component analysis

A well-known limitation of PCA is that the principal components may be *hard to interpret* since they are linear combinations of *all* input variables (apart from some exceptional cases). A pragmatic and oft-used solution is to simply adjust small loadings in the PCA eigenvectors to zero. But such an approach is open to criticism for being ad hoc and arbitrary — after all, how do we decide what is small?

Sparse principal component analysis (SPCA), introduced by Zou et al. (2006), offers a more principled approach to finding principal components with *sparsity*, i.e., with many loadings typically shrunk to zero. SPCA could be based on PCA with L^1 penalty similar to the *lasso* regression seen in supervised learning. Adopting the notation previously used to define PCA, we could seek to solve the j -th loadings vector $\mathbf{c}_j = (c_{j,1}, \dots, c_{j,d}) \in \mathbb{R}^d$ by maximising

$$\begin{aligned} V(\mathbf{c}_j) = \mathbf{c}_j^T \Sigma \mathbf{c}_j \quad \text{such that} \quad & \mathbf{c}_j^T \mathbf{c}_j = 1, \\ & \mathbf{c}_j^T \mathbf{c}_k = 0 \text{ for any } k < j \text{ (if } j > 1), \\ & \underbrace{\sum_{i=1}^d |c_{j,i}|}_{L^1 \text{ penalty}} \leq t, \end{aligned}$$

where Σ is the sample covariance matrix of the data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and $t \geq 0$ is a tuning parameter. (This is the SCoTLASS approach that precedes SPCA, developed by Jolliffe et al. (2003).) Instead, Zou et al. (2006) have built SPCA on the more general *elastic net* penalty which is a linear combination L^1 (lasso) and L^2 (ridge) penalties.

Assume for simplicity that the data are already centred, i.e., $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{0}$. Following the reconstruction error minimisation approach to PCA, for dimensionality $q \leq d$, SPCA is performed by minimising

$$F(\Theta, U) := \sum_{i=1}^n \|\mathbf{x}_i - \Theta U^T \mathbf{x}_i\|^2 + \underbrace{\lambda \sum_{j=1}^q \|\mathbf{u}_j\|_2^2}_{L^2 \text{ penalty}} + \underbrace{\sum_{j=1}^q \lambda_{1,j} \|\mathbf{u}_j\|_1}_{L^1 \text{ penalty}}$$

over all matrices $\Theta \in \mathbb{R}^{d \times q}$ such that $\Theta^T \Theta = I_q$ and all matrices $U \in \mathbb{R}^{d \times q}$ with columns $\mathbf{u}_1, \dots, \mathbf{u}_q \in \mathbb{R}^d$.³ The quantities $\lambda \geq 0$ and $\lambda_{1,1}, \dots, \lambda_{1,q} \geq 0$ are tuning

³We recall that for $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$, the L^1 and L^2 norms are defined by $\|\mathbf{x}\|_1 := |x_1| + \dots + |x_d|$ and $\|\mathbf{x}\|_2 := \|\mathbf{x}\|_2 := (x_1^2 + \dots + x_d^2)^{1/2}$, respectively.

parameters. According to [Hastie et al. \(2009, p. 551\)](#), the function $F(\Theta, U)$ is *not* jointly *convex* in both Θ and U , which would impede the practical minimisation of $F(\Theta, U)$. However, both $\Theta \mapsto F(\Theta, U^*)$ and $U \mapsto F(\Theta^*, U)$ are convex for fixed U^* and Θ^* , respectively. Hence, the minimisation of $F(\Theta, U)$ is done iteratively by alternatingly minimising $\Theta \mapsto F(\Theta, U^*)$ and $U \mapsto F(\Theta^*, U)$, where U^* and Θ^* are the respective previous minimisers, until convergence is reached. (In fact, minimisation with respect to Θ can be obtained from singular value decomposition.)

Remark 1.10. As discussed in [Zou et al. \(2006, Sect. 3.4\)](#) the principal components obtained by SPCA *may not be uncorrelated*, unlike in the classical PCA.

1.4 Kernel principal component analysis

Ordinary PCA is inherently a *linear* method. However, many multivariate data sets encountered in practice exhibit *non-linear* relationships between the variables, which motivates the quest for non-linear extensions of PCA.

The first non-linear extension of PCA we study here is kernel principal component analysis, developed by [Schölkopf et al. \(1997\)](#). Kernel PCA builds on the idea that we can map the data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ to a typically higher dimensional⁴ *feature space* $\mathbb{R}^p$ using a *feature map* $\mathbf{f}: \mathbb{R}^d \rightarrow \mathbb{R}^p$ and then perform classical PCA on the transformed data $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$ in $\mathbb{R}^p$. However, since the feature map $\mathbf{f}$ may require high dimensionality p to effectively linearise the non-linearities of the data, it is often desirable to avoid working directly with $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$. Fortunately, this is achievable using the so-called *kernel trick*, which is the basis of kernel learning methods.

To formulate kernel PCA, following [Schölkopf et al. \(1997\)](#), consider first data $\mathbf{x}_1, \dots, \mathbf{x}_n$ and feature map $\mathbf{f}$ such that $\frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i) = \mathbf{0}$. (Removing this centredness assumption is not entirely trivial. We will discuss this in Remark 1.20 below.) The sample covariance matrix of the transformed data $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$ is then

$$\Sigma_{\mathbf{f}} := \frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i) \mathbf{f}(\mathbf{x}_i)^T \in \mathbb{R}^{p \times p}. \quad (1.11)$$

If we were to perform classical PCA on the transformed data, then any loadings vector $\mathbf{c} \in \mathbb{R}^p$ would be an eigenvector of $\Sigma_{\mathbf{f}}$, that is,

$$\Sigma_{\mathbf{f}} \mathbf{c} = \lambda \mathbf{c} \quad (1.12)$$

⁴In general, the feature space need not be finite-dimensional, but we stick to finite-dimensional, Euclidean spaces here for simplicity.

with the associated eigenvalue $\lambda > 0$.⁵ Substituting (1.11) into (1.12) we get

$$\frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i) \underbrace{(\mathbf{f}(\mathbf{x}_i)^T \mathbf{c})}_{\in \mathbb{R}} = \lambda \mathbf{c},$$

whereby $\mathbf{c}$ is in the linear subspace of $\mathbb{R}^p$ spanned by $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$, namely,

$$\mathbf{c} = \sum_{i=1}^n \alpha_i \mathbf{f}(\mathbf{x}_i) \quad (1.13)$$

with $\alpha_i := \frac{\mathbf{f}(\mathbf{x}_i)^T \mathbf{c}}{\lambda n}$, $i = 1, \dots, n$. Thus, equation (1.12) is equivalent to the equations

$$\mathbf{f}(\mathbf{x}_i)^T \Sigma_f \mathbf{c} = \lambda \mathbf{f}(\mathbf{x}_i)^T \mathbf{c}, \quad i = 1, \dots, n. \quad (1.14)$$

Write now $\boldsymbol{\alpha} := (\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n$ and introduce the *Gram matrix*

$$K := \begin{bmatrix} \mathbf{f}(\mathbf{x}_1)^T \mathbf{f}(\mathbf{x}_1) & \cdots & \mathbf{f}(\mathbf{x}_1)^T \mathbf{f}(\mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ \mathbf{f}(\mathbf{x}_n)^T \mathbf{f}(\mathbf{x}_1) & \cdots & \mathbf{f}(\mathbf{x}_n)^T \mathbf{f}(\mathbf{x}_n) \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Substituting (1.11) and (1.13) into (1.14), after some algebra, we find that (1.14) is equivalent to the equation

$$K^2 \boldsymbol{\alpha} = n \lambda K \boldsymbol{\alpha}, \quad (1.15)$$

which we seek to solve for $\boldsymbol{\alpha}$ and positive $\lambda > 0$. We note that if $\boldsymbol{\alpha}$ solves the *eigenvalue problem*

$$K \boldsymbol{\alpha} = n \lambda \boldsymbol{\alpha}, \quad (1.16)$$

then it also solves (1.15). (It can further be shown that solutions to (1.15) that are not solutions to (1.16) will not give rise to any additional vectors in (1.13).) As in classical PCA, we impose *normalisation* on the eigenvectors, and by implication on $\boldsymbol{\alpha}$, namely,

$$1 = \mathbf{c}^T \mathbf{c} = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \mathbf{f}(\mathbf{x}_i)^T \mathbf{f}(\mathbf{x}_j) \alpha_j = \boldsymbol{\alpha}^T K \boldsymbol{\alpha} = n \lambda \boldsymbol{\alpha}^T \boldsymbol{\alpha}. \quad (1.17)$$

Suppose now that $\mathbf{c}_j \in \mathbb{R}^p$ corresponds to the j -th kernel principal component (PC), representable via (1.13) using vector $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_n)$. Then the projection of input $\mathbf{x} \in \mathbb{R}^d$ onto this kernel PC is given by

$$\mathbf{c}_j^T \mathbf{f}(\mathbf{x}) = \sum_{i=1}^n \alpha_i \mathbf{f}(\mathbf{x}_i)^T \mathbf{f}(\mathbf{x}). \quad (1.18)$$

⁵Since the sample covariance matrix Σ_f is positive semi-definite, all of its eigenvalues are non-negative. Moreover, zero eigenvalues are not of interest in PCA.

The *kernel trick* leverages the insight that all the equations we need in order to perform kernel PCA, that is, (1.16), (1.17) and (1.18), only depend on the feature map $\mathbf{f}$ through *scalar products* of the form $\mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{y})$. This makes it possible to avoid working directly with the feature map by specifying $\mathbf{f}$ implicitly⁶ using a *kernel function* $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ via

$$\mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{y}) := k(\mathbf{x}, \mathbf{y}), \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^d. \quad (1.19)$$

The advantage of the kernel trick is that we do not have to work in the potentially *high-dimensional* feature space $\mathbb{R}^p$, involving a $p \times p$ matrix. However, this advantage is diminished if the number of samples, n , is very large since we need to deal with the $n \times n$ Gram matrix K under the kernel trick. In practice, we use standard kernels, such as:

★ *polynomial*: $k(\mathbf{x}, \mathbf{y}) := (\mathbf{x}^T \mathbf{y} + c_0)^r$, $c \in \mathbb{R}$, $r \in \mathbb{N}$,

★ *radial basis function*: $k(\mathbf{x}, \mathbf{y}) := \exp(\gamma \|\mathbf{x} - \mathbf{y}\|^2)$, $\gamma \in \mathbb{R}_{>0}$,

★ *sigmoid*: $\tanh(\gamma \mathbf{x}^T \mathbf{y} + c)$, $\gamma \in \mathbb{R}_{>0}$, $c \in \mathbb{R}$.

Figure 1.5 provides an illustration of kernel PCA using the radial basis function kernel and a comparison to ordinary PCA.

Remark 1.20. After performing the kernel trick, the assumption $\frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i) = \mathbf{0}$ becomes somewhat less straightforward to remove. Since we do not have access to the feature map $\mathbf{f}$, we *cannot* explicitly *centre* the features $\mathbf{f}(\mathbf{x}_1), \dots, \mathbf{f}(\mathbf{x}_n)$. However, it is possible to introduce the centring in the feature map by defining $\tilde{\mathbf{f}}(\mathbf{x}) := \mathbf{f}(\mathbf{x}) - \frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i)$. The above derivation of kernel PCA carries over, and the kernel trick remains applicable, but the Gram matrix K needs to be replaced by

$$\tilde{K} := K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n,$$

where $\mathbf{1}_n \in \mathbb{R}^{n \times n}$ is such that each of its elements is equal to $\frac{1}{n}$. A derivation of $\tilde{K}$ is given in Schölkopf et al. (1998, Appendix B).

Remark 1.21. Kernel PCA is useful for low-dimensional *feature construction* (e.g., for classification), but one *cannot reconstruct* the original data from the kernel

⁶As long as k is *symmetric*, $k(\mathbf{x}, \mathbf{y}) = k(\mathbf{y}, \mathbf{x})$ for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, and *positive definite*, $\sum_{i=1}^m \sum_{j=1}^m k(\mathbf{x}_i, \mathbf{x}_j) a_i a_j \geq 0$ for any $\mathbf{x}_1, \dots, \mathbf{x}_m \in \mathbb{R}^d$, $a_1, \dots, a_m \in \mathbb{R}$ and $m \in \mathbb{N}$, the existence of a feature map $\mathbf{f}$ satisfying (1.19) is ensured by *Mercer's theorem*, although the feature space is typically infinite-dimensional, see: https://en.wikipedia.org/wiki/Mercer's_theorem

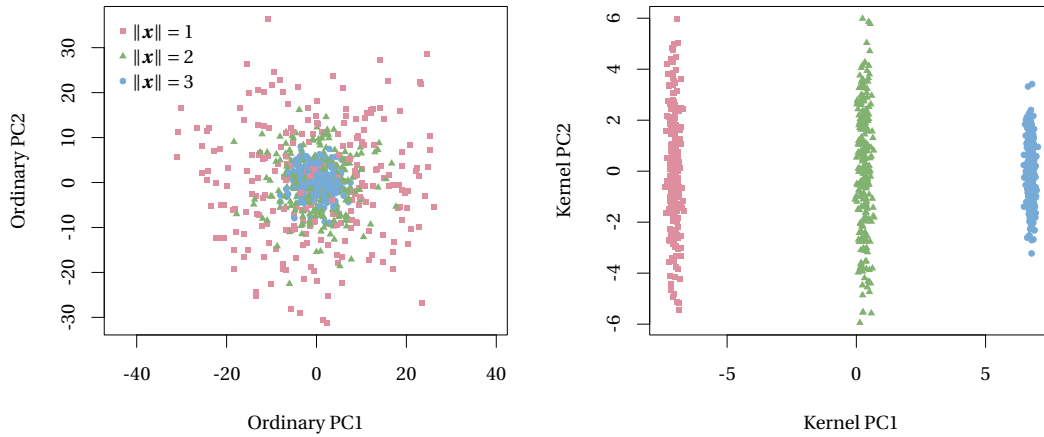


Figure 1.5: A data set that consists of samples on three concentric spheres in $\mathbb{R}^{100}$ with radii 1, 2 and 3, respectively, reduced to its first two ordinary PCs (left) and to the first two kernel PCs (right) using the radial basis function kernel with $\gamma = 0.1$.

PCs. The first obstacle is that after performing the kernel trick we do not have access to the feature map $\mathbf{f}$, which we would need to *invert* in order to map the kernel PCs in the feature space $\mathbb{R}^p$ back to the input space $\mathbb{R}^d$. The second obstacle is that even if $\mathbf{f}$ were explicitly known, the kernel PCs would possibly lie *outside* the *image* of $\mathbf{f}$, i.e., $\mathbf{f}(\mathbb{R}^d) := \{\mathbf{f}(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^d\} \subset \mathbb{R}^p$, making inversion impossible.

1.5 Autoencoder

An *autoencoder* (Bourlard and Kamp, 1988; Kramer, 1991) is another non-linear extension of PCA. Unlike kernel PCA, it approaches dimensionality reduction from the point of view of *reconstruction error minimisation*. Autoencoders, which are sometimes also called *autoassociative neural networks* (Bishop, 2006, Sect. 12.4.2), are in the purview of *deep learning*, so we will only give here a high-level overview.

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, an autoencoder first maps the data into a lower-dimensional space $\mathbb{R}^q$ using a function $\mathbf{E} : \mathbb{R}^d \rightarrow \mathbb{R}^q$ called the *encoder*. (That is, the dimensionality q is supposed to be much lower than d .) Then the transformed data are mapped from $\mathbb{R}^q$ back into $\mathbb{R}^d$ using another function $\mathbf{D} : \mathbb{R}^q \rightarrow \mathbb{R}^d$ called the *decoder* — see Figure 1.6 for an illustration. The objective is that the output $\mathbf{D}(\mathbf{E}(\mathbf{x}_i))$ is as close as possible to the input $\mathbf{x}_i$ for any $i = 1, \dots, n$.

The encoder $\mathbf{E} = \mathbf{E}_\phi$ and decoder $\mathbf{D} = \mathbf{D}_\theta$ are set up as *feedforward neural*

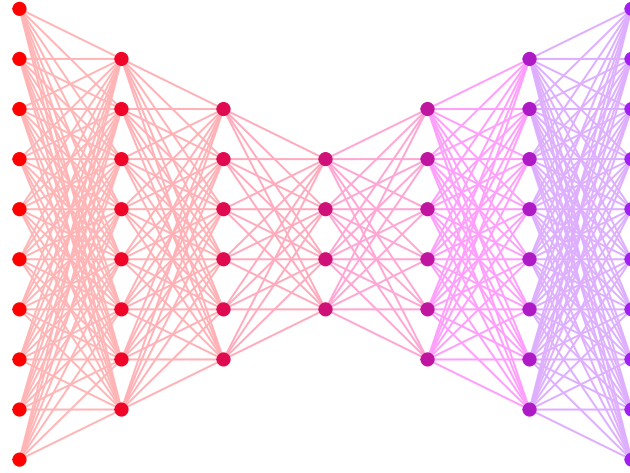


Figure 1.6: Schematic of an autoencoder architecture as a neural network with $d = 10$ and $q = 4$. The first half of the network forms the encoder and the second half the decoder.

networks with their respective parameter vectors ϕ and θ . The parameters in ϕ and θ (neural network *weights* and *biases*) are trained by minimising the reconstruction error

$$\frac{1}{n} \sum_{i=1}^n \|x_i - D(E(x_i))\|^2$$

using *gradient descent* (or minimising its mini-batch version using *stochastic gradient descent*).

Remark 1.22. If the decoder D is constrained to be *linear* and if the architecture of the encoder E covers all linear maps from $\mathbb{R}^d$ to $\mathbb{R}^q$, then the autoencoder becomes equivalent to classical PCA.

1.6 Canonical correlation analysis

1.6.1 Formulation

Canonical correlation analysis (CCA), developed by Hotelling (1936), is an extension of principal component analysis which applies to *two multivariate data sets* with concurrent (but not necessarily equidimensional) samples. It aims to find projections of the two data sets that have *maximal correlation*.

Consider two multivariate data sets $x_1, \dots, x_n \in \mathbb{R}^d$ and $y_1, \dots, y_n \in \mathbb{R}^e$. Their *sample cross-covariance matrix* is defined as

$$\Sigma_{xy} := \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})^T \in \mathbb{R}^{d \times e},$$

using sample means $\bar{\mathbf{x}} := \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and $\bar{\mathbf{y}} := \frac{1}{n} \sum_{i=1}^n \mathbf{y}_i$. Given vectors $\mathbf{a} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^e$, the *sample correlation* between the projected (univariate) data sets $\mathbf{a}^T \mathbf{x}_i$, $i = 1, \dots, n$, and $\mathbf{b}^T \mathbf{y}_i$, $i = 1, \dots, n$ is

$$\begin{aligned} \rho_{xy} &:= \rho_{xy}(\mathbf{a}, \mathbf{b}) := \frac{\frac{1}{n} \sum_{i=1}^n (\mathbf{a}^T \mathbf{x}_i - \mathbf{a}^T \bar{\mathbf{x}})(\mathbf{b}^T \mathbf{y}_i - \mathbf{b}^T \bar{\mathbf{y}})}{\sqrt{\frac{1}{n} \sum_{i=1}^n (\mathbf{a}^T \mathbf{x}_i - \mathbf{a}^T \bar{\mathbf{x}})^2} \sqrt{\frac{1}{n} \sum_{i=1}^n (\mathbf{b}^T \mathbf{y}_i - \mathbf{b}^T \bar{\mathbf{y}})^2}} \\ &= \frac{\mathbf{a}^T \Sigma_{xy} \mathbf{b}}{\sqrt{\mathbf{a}^T \Sigma_x \mathbf{a}} \sqrt{\mathbf{b}^T \Sigma_y \mathbf{b}}} \in [-1, 1], \end{aligned}$$

where $\Sigma_x \in \mathbb{R}^{d \times d}$ and $\Sigma_y \in \mathbb{R}^{e \times e}$ are the sample covariance matrices of $\mathbf{x}_1, \dots, \mathbf{x}_n$ and $\mathbf{y}_1, \dots, \mathbf{y}_n$, respectively. The matrices Σ_x and Σ_y are assumed to be *invertible* in what follows.

The first pair of *canonical variables* is defined by

$$u_{1,i} := \mathbf{a}_1^T \mathbf{x}_i, \quad i = 1, \dots, n, \quad v_{1,i} := \mathbf{b}_1^T \mathbf{y}_i, \quad i = 1, \dots, n,$$

where $\mathbf{a}_1$ and $\mathbf{b}_1$ maximise $\rho_{xy}(\mathbf{a}_1, \mathbf{b}_1)$. Note that they are *not unique* — they are unique only up to scaling by a positive scalar since $\rho_{xy}(\alpha \mathbf{a}_1, \beta \mathbf{b}_1) = \rho_{xy}(\mathbf{a}_1, \mathbf{b}_1)$ for $\alpha, \beta > 0$. Subsequently, we define the k -th pair of canonical variables for $k = 2, \dots, \min\{d, e\}$ similarly by

$$u_{k,i} := \mathbf{a}_k^T \mathbf{x}_i, \quad i = 1, \dots, n, \quad v_{k,i} := \mathbf{b}_k^T \mathbf{y}_i, \quad i = 1, \dots, n,$$

where $\mathbf{a}_k \in \mathbb{R}^d$ and $\mathbf{b}_k \in \mathbb{R}^e$ maximise $\rho_{xy}(\mathbf{a}_k, \mathbf{b}_k)$ such that⁷

$$\mathbf{a}_k^T \Sigma_x \mathbf{a}_j = 0 \quad \text{for any } j < k, \quad (1.23)$$

$$\mathbf{b}_k^T \Sigma_y \mathbf{b}_j = 0 \quad \text{for any } j < k. \quad (1.24)$$

The orthogonality condition (1.23) will ensure that the *sample correlation* between the canonical variables $u_{k,i}$, $i = 1, \dots, n$, and $u_{j,i}$, $i = 1, \dots, n$, *is zero* for any $j < k$, and (1.24) is an analogous requirement for $v_{k,i}$, $i = 1, \dots, n$, and $v_{j,i}$, $i = 1, \dots, n$. Since the constraints (1.23) and (1.24) become stricter as k increases, it follows that

$$\rho_{xy}(\mathbf{a}_1, \mathbf{b}_1) \geq \rho_{xy}(\mathbf{a}_2, \mathbf{b}_2) \geq \dots \geq \rho_{xy}(\mathbf{a}_{\min\{d,e\}}, \mathbf{b}_{\min\{d,e\}}).$$

Remark 1.25. The constraints (1.23) and (1.24) are the reason why we cannot find more than $\min\{d, e\}$ pairs of canonical variables. We can find at most d orthogonal components from $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and at most e orthogonal components from $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^e$.

⁷In Lecture Video 5, the matrices Σ_x and Σ_y have been inadvertently omitted from the orthogonality conditions.

1.6.2 Solution

We outline here the derivation of canonical variables. Like principal components, canonical variables could be solved using the Lagrange multiplier method, but we follow here a somewhat more direct approach, cf. [Wikipedia \(2023\)](#). To this end, we first introduce the symmetric square roots $\Sigma_x^{1/2}$ and $\Sigma_y^{1/2}$ of the covariance matrices Σ_x and Σ_y and their (symmetric) inverses $\Sigma_x^{-1/2}$ and $\Sigma_y^{-1/2}$.⁸ We can then re-parameterise the problem using

$$\tilde{\mathbf{a}} := \Sigma_x^{1/2} \mathbf{a}, \quad \tilde{\mathbf{b}} := \Sigma_y^{1/2} \mathbf{b},$$

whereby

$$\rho = \rho(\Sigma_x^{-1/2} \tilde{\mathbf{a}}, \Sigma_y^{-1/2} \tilde{\mathbf{b}}) = \frac{\tilde{\mathbf{a}}^T \Sigma_x^{-1/2} \Sigma_{xy} \Sigma_y^{-1/2} \tilde{\mathbf{b}}}{\|\tilde{\mathbf{a}}\| \|\tilde{\mathbf{b}}\|}.$$

Note that, thanks to the normalising denominator, the *norms* of $\tilde{\mathbf{a}}$ and $\tilde{\mathbf{b}}$ become *immaterial*, only the *angles* of the two vectors *matter*. Write then the numerator as a scalar product,

$$\tilde{\mathbf{a}}^T \Sigma_x^{-1/2} \Sigma_{xy} \Sigma_y^{-1/2} \tilde{\mathbf{b}} = (\Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}})^T \tilde{\mathbf{b}}.$$

By the Cauchy–Schwarz inequality,

$$(\Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}})^T \tilde{\mathbf{b}} \leq \|\Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}\| \|\tilde{\mathbf{b}}\|,$$

where *equality* holds when the vectors $\Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}$ and $\tilde{\mathbf{b}}$ are *linearly dependent* (i.e., one is a scalar multiple of the other). Therefore, to maximise the left-

⁸There are many ways to define a square root of a square matrix (which need not be unique in any case), but for our purposes, the eigendecomposition is the most straightforward. For simplicity, consider Σ_x only. Recall that we can write

$$\Sigma_x = O \Lambda O^T,$$

where $O \in \mathbb{R}^{d \times d}$ is an orthogonal matrix, i.e., $O^{-1} = O^T$, and $\Lambda \in \mathbb{R}^{d \times d}$ is a diagonal matrix with the eigenvalues of Σ_x on the diagonal. Denoting by $\Lambda^{1/2}$ the matrix where we replace the eigenvalues in Λ (which are all non-negative) by their square roots, we can then define

$$\Sigma_x^{1/2} := O \Lambda^{1/2} O^T,$$

which is symmetric by construction. Moreover, when the eigenvalues are (strictly) positive, $\Lambda^{1/2}$ can be inverted element by element, and we can further define

$$\Sigma_x^{-1/2} := O \Lambda^{-1/2} O^T.$$

hand side and ensure equality, we take

$$\tilde{\mathbf{b}}_1 := \Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}_1.$$

To find optimal $\tilde{\mathbf{a}}_1$, we seek to maximise

$$\begin{aligned} \left\| \Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}_1 \right\|^2 &= \left(\Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}_1 \right)^T \Sigma_y^{-1/2} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}_1 \\ &= \tilde{\mathbf{a}}_1^T \Sigma_x^{-1/2} \Sigma_{xy} \Sigma_y^{-1} \Sigma_{xy}^T \Sigma_x^{-1/2} \tilde{\mathbf{a}}_1, \end{aligned} \quad (1.26)$$

which is a *quadratic form*. Since the norm of the vector $\tilde{\mathbf{a}}_1$ does not affect ρ , we can (and, in fact, we must) normalise it, i.e., $\|\tilde{\mathbf{a}}_1\| = 1$. Like in principal component analysis, under this constraint the optimal $\tilde{\mathbf{a}}_1$ is then the *eigenvector* of $\Sigma_x^{-1/2} \Sigma_{xy} \Sigma_y^{-1} \Sigma_{xy}^T \Sigma_x^{-1/2}$ corresponding to its *largest eigenvalue*. Mapping the solution back to the original variables $\mathbf{a}$ and $\mathbf{b}$, we have:

- ★ $\mathbf{a}_1$ is the eigenvector of $\Sigma_x^{-1} \Sigma_{xy} \Sigma_y^{-1} \Sigma_{xy}^T$ corresponding to its largest eigenvalue,
- ★ $\mathbf{b}_1 := \Sigma_y^{-1} \Sigma_{xy}^T \mathbf{a}_1$.

The remaining pairs of canonical variables can be found similarly. More precisely, for $k = 2, \dots, \min\{d, e\}$,

- ★ $\mathbf{a}_k$ is the eigenvector of $\Sigma_x^{-1} \Sigma_{xy} \Sigma_y^{-1} \Sigma_{xy}^T$ corresponding to its k -th largest eigenvalue,
- ★ $\mathbf{b}_k := \Sigma_y^{-1} \Sigma_{xy}^T \mathbf{a}_k$.

The conditions (1.23) and (1.24) then follow from the orthogonality property of eigenvectors.

1.7 Non-negative matrix factorisation

1.7.1 Setting and motivation

Non-negative matrix factorisation (NMF) is an unsupervised learning technique that seeks to find an *approximate* factorisation

$$X \approx WH \quad (1.27)$$

of a data matrix $X \in \mathbb{R}_{\geq 0}^{d \times n}$ in terms of matrices $W \in \mathbb{R}_{\geq 0}^{d \times q}$ and $H \in \mathbb{R}_{\geq 0}^{q \times n}$ where $q \leq \max\{d, n\}$, but ideally q much smaller than d and n . The data matrix could be built from samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, that is,

$$X = \begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_n \end{bmatrix}, \quad (1.28)$$

but it need not have such a structure. For example, in a common application of NMF, X would be a (monochrome) image.

To compare NMF to PCA in the case (1.28), it is helpful to write H in terms of its columns as

$$H = \begin{bmatrix} \mathbf{h}_1 & \cdots & \mathbf{h}_n \end{bmatrix}$$

with vectors $\mathbf{h}_1, \dots, \mathbf{h}_n \in \mathbb{R}^q$. Then we can think of $\mathbf{h}_i$ as a *lower-dimensional description* of the sample $\mathbf{x}_i$ for any $i = 1, \dots, n$, while W describes the linear map that aims to reconstruct $\mathbf{x}_i$ from $\mathbf{h}_i$. Thus, $\mathbf{h}_i$ is analogous to the vector $\mathbf{z}_i$ in the reconstruction error minimisation formulation of PCA, which contains the data on the principal components for the i -th sample.

As the name suggests, the non-negativity of the entries of W and H is key. Preserving non-negativity is desirable in certain applications where the data are inherently non-negative, such as *imaging*. The non-negativity of W and H can also be helpful as far as *interpretability* is concerned. NMF provides an approximate decomposition of the data matrix X into *additive* components that *cannot cancel each other*, unlike principal components, which assume both positive and negative values.

1.7.2 Formulation and algorithm

To formalise NMF, we need to determine how well the approximation (1.27) holds. As discussed by Lee and Seung (2000), one could use either the squared *Euclidean distance* between matrices X and WH , given by

$$\sum_{i=1}^d \sum_{j=1}^n (X_{i,j} - (WH)_{i,j})^2 \quad (1.29)$$

or the *quasi-likelihood function*

$$L(W, H) := \sum_{i=1}^d \sum_{j=1}^n \left(X_{i,j} \log((WH)_{i,j}) - (WH)_{i,j} \right),$$

where the summand is the log-likelihood contribution of the observation $X_{i,j}$ if it were Poisson-distributed with mean $(WH)_{i,j}$. Following Lee and Seung (1999), we adopt here the latter, $L(W, H)$.

Neither $L(W, H)$ nor the Euclidean distance (1.29) can be optimised analytically with respect to W and H . To numerically maximise $L(W, H)$, Lee and Seung (2000) develop an *iterative algorithm* where the following updates are performed

alternatingly until convergence is achieved:

$$W_{i,k} \leftarrow W_{i,k} \frac{\sum_{j=1}^n H_{k,j} X_{i,j} / (WH)_{i,j}}{\sum_{j=1}^n H_{k,j}}, \quad i = 1, \dots, d, \quad k = 1, \dots, q,$$

$$H_{k,j} \leftarrow H_{k,j} \frac{\sum_{i=1}^d W_{i,k} X_{i,j} / (WH)_{i,j}}{\sum_{i=1}^d W_{i,k}}, \quad k = 1, \dots, q, \quad j = 1, \dots, n.$$

Moreover, *initial values* for W and H must be specified. Packages that implement NMF generally offer various choices, covering both data-driven and randomised initialisers.

Remark 1.30. Neither $L(W, H)$ nor the Euclidean distance (1.29) are convex in W and H , which is why iterative algorithms may only converge to a *local* optimum. Hence, the choice of (and experimentation with) initial values becomes important. Moreover, the factorisation *need not be unique* even in the ideal case $X = WH$. For details, see [Hastie et al. \(2009, Sect. 14.6\)](#) and [Donoho and Stodden \(2003\)](#).

1.8 Independent component analysis

1.8.1 Motivation and assumptions

Independent component analysis (ICA) is a feature extraction technique motivated by the so-called *cocktail-party problem*,⁹ which can be described as follows. Suppose that two people are speaking simultaneously in an otherwise silent room. Sound is then recorded through two microphones placed in different locations in the room. Let us denote the two speakers' speech signals by $S_1(t)$ and $S_2(t)$, respectively. Ignoring acoustics (delay, reverberation) and possible imperfections in audio recording (distortion, noise etc), the signals $X_1(t)$ and $X_2(t)$ recorded through the two microphones can be expressed as *linear mixtures*

$$\begin{aligned} X_1(t) &= a_{1,1} S_1(t) + a_{1,2} S_2(t), \\ X_2(t) &= a_{2,1} S_1(t) + a_{2,2} S_2(t), \end{aligned} \tag{1.31}$$

with some weights $a_{1,1}, a_{1,2}, a_{2,1}, a_{2,2} \in \mathbb{R}$ that depend on the locations of the speakers and microphones but are unknown. The problem is then about extracting the individual speech signals $S_1(t)$ and $S_2(t)$ from the recordings $X_1(t)$ and $X_2(t)$.

The cocktail-party problem is generally *not solvable* unless further assumptions are made. This is due to lack of *identifiability* — there may be infinitely many ways to express $X_1(t)$ and $X_2(t)$ in the form (1.31). To make the problem

⁹Such problems are more broadly called *blind source separation* problems.

tractable, we introduce a *probabilistic* framework where $S_1(t)$ and $S_2(t)$ are *random variables*. For simplicity, we drop the time index t and simply write S_1 and S_2 . Then, the observed X_1 and X_2 are random variables satisfying

$$\begin{aligned} X_1 &= a_{1,1}S_1 + a_{1,2}S_2, \\ X_2 &= a_{2,1}S_1 + a_{2,2}S_2. \end{aligned}$$

or

$$\underbrace{\begin{bmatrix} X_1 \\ X_2 \end{bmatrix}}_{=:X} = \underbrace{\begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}}_{=:A} \underbrace{\begin{bmatrix} S_1 \\ S_2 \end{bmatrix}}_{=:S}$$

in matrix form. The key assumption that makes the problem solvable and which ICA builds on is that the components S_1 and S_2 are *mutually independent*. Such an assumption is reasonable in the context of the cocktail-party problem, but also is not too restrictive in feature extraction in general.

Another key assumption behind ICA is that the independent components S_1 and S_2 are *non-Gaussian*, i.e., not normally distributed. To see why this is necessary,¹⁰ suppose that S_1 and S_2 are standard normal and mutually independent. Then, both S and X are *multivariate normal*, $S \sim N_2(\mathbf{0}, I_2)$ and $X \sim N_2(\mathbf{0}, AA^T)$. But for any *orthogonal* matrix $O \in \mathbb{R}^{2 \times 2}$, i.e., satisfying $O^T O = I_2 = OO^T$, we can write

$$X = AI_2S = \underbrace{AO^T}_{=: \tilde{A}} \underbrace{OS}_{=: \tilde{S}}.$$

Now, $\tilde{S} \sim N_2(\mathbf{0}, OO^T) = N_2(\mathbf{0}, I_2)$, so it is *equal in distribution* to S , while $\tilde{A}$ is typically *not equal* to A . Hence, the representation of X as a linear transformation of standard normal random variables is *not unique*. This would again amount to lack of identifiability and will be ruled out by assuming non-Gaussianity.

1.8.2 Model and derivation

It is not straightforward to pinpoint who invented ICA, as it has appeared in various guises (and under different names) in the literature throughout the 1980s and 1990s (Comon, 1994). Our formulation of ICA here follows Hastie et al. (2009, Sect. 14.7.2) and Hyvärinen and Oja (2000).

ICA builds on a probabilistic model where a d -dimensional random vector X satisfies

$$X = AS$$

¹⁰Strictly speaking, it is necessary that just one of the independent components is non-Gaussian, but it is nevertheless standard to assume that they all are.

for some matrix $A \in \mathbb{R}^{d \times d}$ and a d -dimensional random vector $\mathbf{S}$ consisting of mutually independent components with zero mean and unit variance. It is standard to assume that $\mathbf{X}$ has been *whitened*¹¹ so that $\mathbf{E}[\mathbf{X}] = \mathbf{0}$ and $\mathbf{E}[\mathbf{X}\mathbf{X}^T] = I_d$. In practice, this can be achieved by transforming the raw data to centred principal components and normalising them by their standard deviations. The data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ on which ICA is performed are thought of as *realisations* of the random vector $\mathbf{X}$.

Note that, since $\mathbf{E}[\mathbf{X}] = \mathbf{0} = \mathbf{E}[\mathbf{S}]$ and $\mathbf{E}[\mathbf{X}\mathbf{X}^T] = I_d = \mathbf{E}[\mathbf{S}\mathbf{S}^T]$, the matrix A is necessarily *orthogonal*. Hence, we look for an orthogonal matrix A such that

$$\mathbf{S} = A^T \mathbf{X}$$

has independent and non-Gaussian components. To measure independence, *entropy* can be used. We recall that the *differential entropy* of a random variable Y with probability density function (pdf) g_Y is

$$H(Y) := \mathbf{E}[-\log g_Y(Y)] = - \int_{\mathbb{R}} g_Y(y) \log g_Y(y) dy.$$

Similarly, the differential entropy of a d -dimensional random vector $\mathbf{Y}$ with pdf g_Y is

$$H(\mathbf{Y}) := \mathbf{E}[-\log g_Y(\mathbf{Y})] = - \int_{\mathbb{R}^d} g_Y(\mathbf{y}) \log g_Y(\mathbf{y}) d\mathbf{y}.$$

It is known that among all continuous distributions with fixed mean and variance, the corresponding normal distribution has the *maximal entropy*.¹² Using differential entropy, we can define the *mutual information* between the components $Y_1, \dots, Y_d$ of a random vector $\mathbf{Y}$ via

$$I(\mathbf{Y}) := \sum_{i=1}^d H(Y_i) - H(\mathbf{Y}).$$

It is possible to show using *Jensen's inequality* that $I(\mathbf{Y}) \geq 0$. Moreover, value *zero is attained* when the components of $\mathbf{Y}$ are mutually *independent*.

Returning to the ICA problem, we define $\mathbf{Y} := A^T \mathbf{X}$ and seek to minimise

$$I(\mathbf{Y}) = \sum_{i=1}^d H(Y_i) + \int_{\mathbb{R}^d} g_Y(\mathbf{y}) \log g_Y(\mathbf{y}) d\mathbf{y}.$$

¹¹Whitening refers to *white noise*, which is a term used in time series analysis and signal processing for stochastic processes consisting of uncorrelated (but not necessarily independent) random variables with constant variance.

¹²See, e.g., https://en.wikipedia.org/wiki/Differential_entropy#Maximization_in_the_normal_distribution

over all orthogonal matrices $A \in \mathbb{R}^{d \times d}$. Using $g_Y(\mathbf{y}) = g_X(A\mathbf{y})|\det(A)|$, where $|\det(A)| = 1$ since A is orthogonal, we get

$$\begin{aligned} \int_{\mathbb{R}} g_Y(\mathbf{y}) \log g_Y(\mathbf{y}) d\mathbf{y} &= \int_{\mathbb{R}} g_X(A\mathbf{y}) \log g_X(A\mathbf{y}) d\mathbf{y} \\ &= \int_{\mathbb{R}} \underbrace{|\det(A^T)|}_{=|\det(A)|=1} g_X(\mathbf{x}) \log g_X(\mathbf{x}) d\mathbf{x} = H(\mathbf{X}), \end{aligned}$$

substituting $\mathbf{y} = A^T \mathbf{x}$. Hence,

$$I(\mathbf{Y}) = \sum_{i=1}^d H(Y_i) - H(\mathbf{X}),$$

and minimising $I(\mathbf{Y})$ with respect to A amounts to just minimising

$$\sum_{i=1}^d H(Y_i). \quad (1.32)$$

Thereby, we try to make $Y_1, \dots, Y_d$ not only as *mutually independent as possible* but also *as non-Gaussian as possible*.

Working with the differential entropies $H(Y_1), \dots, H(Y_d)$ in (1.32) using *sample data* is awkward, however, since they require knowledge of the pdfs of $Y_1, \dots, Y_d$, which in turn depend on the matrix A we are optimising over. To make ICA feasible, it is standard to first replace each $H(Y_i)$ with *negentropy* (a contraction of “negative entropy”)

$$J(Y_i) := H(Z_i) - H(Y_i),$$

where Z_i is normally distributed with mean zero and variance equal to that of Y_i , and while doing so, we need to switch from minimisation to *maximisation*. The negentropy $J(Y_i)$ is another measure of how much Y_i departs from Gaussianity and since a normal distribution maximises entropy, we have $J(Y_i) \geq 0$. The negentropy $J(Y_i)$ per se is not any more tractable than the original differential entropy $H(Y_i)$, but it is amenable to *approximation*. More specifically, as discussed by Hyvärinen and Oja (2000, Sect. 4.2.3), we can use

$$J(Y_i) \approx (\mathbf{E}[G(Y_i)] - \mathbf{E}[G(Z_i)])^2 \quad (1.33)$$

with a suitably chosen function $G : \mathbb{R} \rightarrow \mathbb{R}$. There are several different forms of function G that could be used, but Hyvärinen and Oja (2000) recommend $G(x) := \frac{1}{a} \log \cosh(ax)$ with $a \in [1, 2]$. Finally, the expectation $\mathbf{E}[G(Y_i)]$ in (1.33) is estimated using the corresponding sample mean and optimisation is then carried out using Newton’s method (Hyvärinen and Oja, 2000, Sect. 6).

Remark 1.34. It is worthwhile to be aware of a few caveats of ICA which are *not* shared by PCA. Firstly, ICA is unable to recover the *variances* of the independent components. Hence, it was of no loss of generality to assume at the outset that the variances are equal to unity. Moreover, ICA is unable to recover the *signs* of the components, but this is also a limitation of PCA. Secondly, the *ordering* of the independent components is *arbitrary* and conveys no information about their *importance* (unlike the ordering of principal components).

1.9 Multidimensional scaling and non-linear dimensionality reduction methods

1.9.1 Multidimensional scaling

Multidimensional scaling (MDS) in its simplest form is a dimensionality reduction method where we start with samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ in a high-dimensional space, which we seek to map to samples $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^q$ in a lower dimensional space, where $q \leq n$, while preserving the *geometry* of the samples as much as possible — see Figure 1.7. The preservation of geometry can be measured through *distances*, or *dissimilarities*, i.e.,

$$\underbrace{\|\mathbf{x}_i - \mathbf{x}_j\|}_{=:d_{i,j}} \approx \|\mathbf{z}_i - \mathbf{z}_j\|, \quad i, j = 1, \dots, n, \quad (1.35)$$

or through centred *scalar products*, or *similarities*, i.e.,

$$\underbrace{(\mathbf{x}_i - \bar{\mathbf{x}})^T (\mathbf{x}_j - \bar{\mathbf{x}})}_{=:s_{i,j}} \approx (\mathbf{z}_i - \bar{\mathbf{z}})^T (\mathbf{z}_j - \bar{\mathbf{z}}), \quad i, j = 1, \dots, n, \quad (1.36)$$

with $\bar{\mathbf{x}} := \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and $\bar{\mathbf{z}} := \frac{1}{n} \sum_{i=1}^n \mathbf{z}_i$. More broadly, MDS can also be applied when *we only have access to the dissimilarities* $d_{i,j}$, $i, j = 1, \dots, n$, *or to the similarities* $s_{i,j}$, $i, j = 1, \dots, n$, rather than the actual samples $\mathbf{x}_1, \dots, \mathbf{x}_n$. In this way MDS is fundamentally different from the dimensionality reduction methods we have studied previously, which all require the full coordinate data $\mathbf{x}_1, \dots, \mathbf{x}_n$. Moreover, the distances $d_{i,j}$, $i, j = 1, \dots, n$, need *not* be Euclidean distances — they could as well be *geodesic distances* on a *manifold*, say (which is a point we will touch upon in Section 1.9.2, below).

Remark 1.37. In a Euclidean space, dissimilarities and similarities are related via the *polarisation identity*

$$s_{i,j} = \frac{1}{2} \left(\underbrace{\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2}_{=:s_{i,i}} + \underbrace{\|\mathbf{x}_j - \bar{\mathbf{x}}\|^2}_{=:s_{j,j}} - d_{i,j}^2 \right).$$

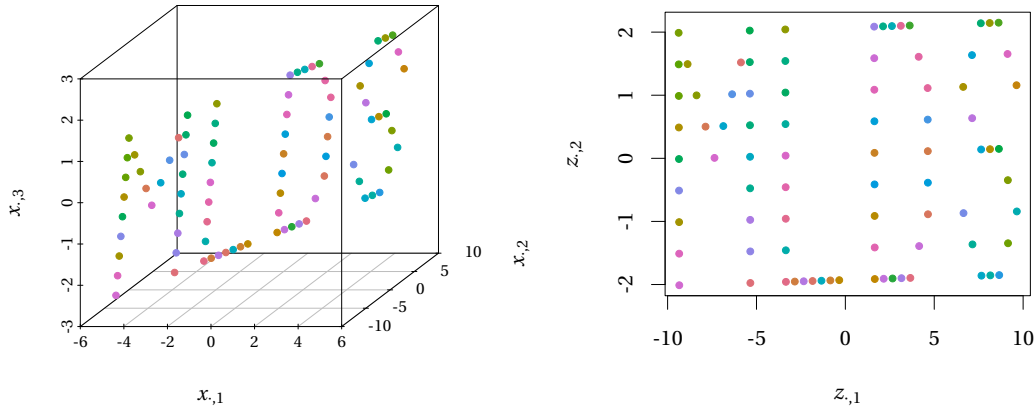


Figure 1.7: MDS applied to the data set from Figure 1.1, mapping from $\mathbb{R}^3$ to $\mathbb{R}^2$. In this case, the geometry of the original samples is perfectly preserved (confirming that the intrinsic dimensionality of the data set is at most two).

Different formulations of MDS mainly differ by whether they use dissimilarities or similarities and how they quantify (1.35) or (1.36). *Classical scaling*, also known as *Torgerson–Gower scaling* (Torgerson, 1958), is based on similarities and seeks to minimise the objective function, known as *strain*,

$$S_C(\mathbf{z}_1, \dots, \mathbf{z}_n) := \sum_{i,j} (s_{i,j} - (\mathbf{z}_i - \bar{\mathbf{z}})^T (\mathbf{z}_j - \bar{\mathbf{z}}))^2.$$

The strength of this approach is that the minimisation problem can be solved analytically, resembling ordinary PCA. Let $S := [s_{i,j}]_{i,j=1}^n \in \mathbb{R}^{n \times n}$, and let $\lambda_1, \dots, \lambda_n \geq 0$ and $\mathbf{u}_1, \dots, \mathbf{u}_n \in \mathbb{R}^n$ be the eigenvalues and eigenvectors, respectively, of S . Defining

$$U_q := [\mathbf{u}_1 \quad \dots \quad \mathbf{u}_q] \in \mathbb{R}^{n \times q}, \quad \Lambda_q^{1/2} := \begin{bmatrix} \sqrt{\lambda_1} & & 0 \\ & \ddots & \\ 0 & & \sqrt{\lambda_q} \end{bmatrix} \in \mathbb{R}^{q \times q},$$

the optimal $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^q$ can be shown to be the rows of the product matrix $U_q \Lambda_q^{1/2} \in \mathbb{R}^{n \times q}$ (Hastie et al., 2009, Exercise 14.11).

Least squares scaling, or *Kruskal–Shepard scaling* (Kruskal, 1964), uses dissimilarities instead and is found by minimising the objective function, known as *stress*,

$$S_M(\mathbf{z}_1, \dots, \mathbf{z}_n) := \sum_{i \neq j} (d_{i,j} - \|\mathbf{z}_i - \mathbf{z}_j\|)^2.$$

Despite their superficial similarity, classical scaling and least squares scaling are not equivalent. Least squares scaling is not analytically solvable, but a *gradient descent algorithm* can be used.

Sammon mapping (Sammon, 1969) is a modification of least squares scaling where weighting by the reciprocals of the dissimilarities is implemented, i.e.,

$$S_{Sm}(\mathbf{z}_1, \dots, \mathbf{z}_n) := \sum_{i \neq j} \frac{(d_{i,j} - \|\mathbf{z}_i - \mathbf{z}_j\|)^2}{d_{i,j}}. \quad (1.38)$$

This gives more emphasis to *small distances*, which are then preserved better.

1.9.2 Non-linear dimensionality reduction and manifold hypothesis

Dimensionality reduction is most effective in situations where we work with multivariate data in $\mathbb{R}^d$ that are confined (or approximately confined) to a *lower dimensional subset* of $\mathbb{R}^d$. If this subset is a linear space, then the linear dimensionality reduction methods we have studied so far are typically sufficient. However, more generally the subset could be a (non-linear) *manifold*, a concept that will be elucidated shortly. This is the so-called *manifold hypothesis* that is often invoked in machine learning. In kernel PCA, we try to find (implicitly) a feature map that (approximately) linearises the manifold, so that the effectiveness of ordinary PCA can be restored. However, this hinges on our ability to choose a kernel and hyperparameters that match the geometry of the manifold, which can be difficult if the manifold is *unknown*, as it usually is.¹³

An alternative approach to non-linear dimensionality reduction is to try to emphasise the preservation of the *local geometry* of the data in the spirit of MDS. The motivation for such an approach comes from the very definition of a manifold.¹⁴ Informally, a q -manifold M in $\mathbb{R}^d$, with $q \leq d$, is defined as a subset of $\mathbb{R}^d$ such that any point $\mathbf{x} \in M$ has a neighbourhood $U \subset M$ that *resembles* a neighbourhood in $\mathbb{R}^q$. A bit more formal description of the resemblance is that there exists a differentiable, one-to-one map $\boldsymbol{\varphi}$ from U to $\mathbb{R}^q$, which is called a *chart*.

Example 1.39. (i) The *unit circle* $\mathbb{S}^1 := \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\| = 1\}$ is a 1-manifold. Every point $\mathbf{x} \in \mathbb{S}^1$ has a neighbourhood U that can be mapped to an open interval in $\mathbb{R}$, as illustrated in Figure 1.8 (top). The chart $\boldsymbol{\varphi} : U \rightarrow \mathbb{R}$ can be derived, e.g., from the angular coordinate of each point in $\mathbb{S}^1$.

¹³To this end, context knowledge or exploratory data analysis (with low-dimensional data) may enable us to work out the rough geometry of the manifold, but this becomes effectively impossible in higher dimensions.

¹⁴The rigorous definition of a manifold is in the purview of *differential geometry*, an area of mathematical analysis, which is well beyond the scope of this module. An interested reader is, however, referred to, e.g., Lang (1995) for further reading.

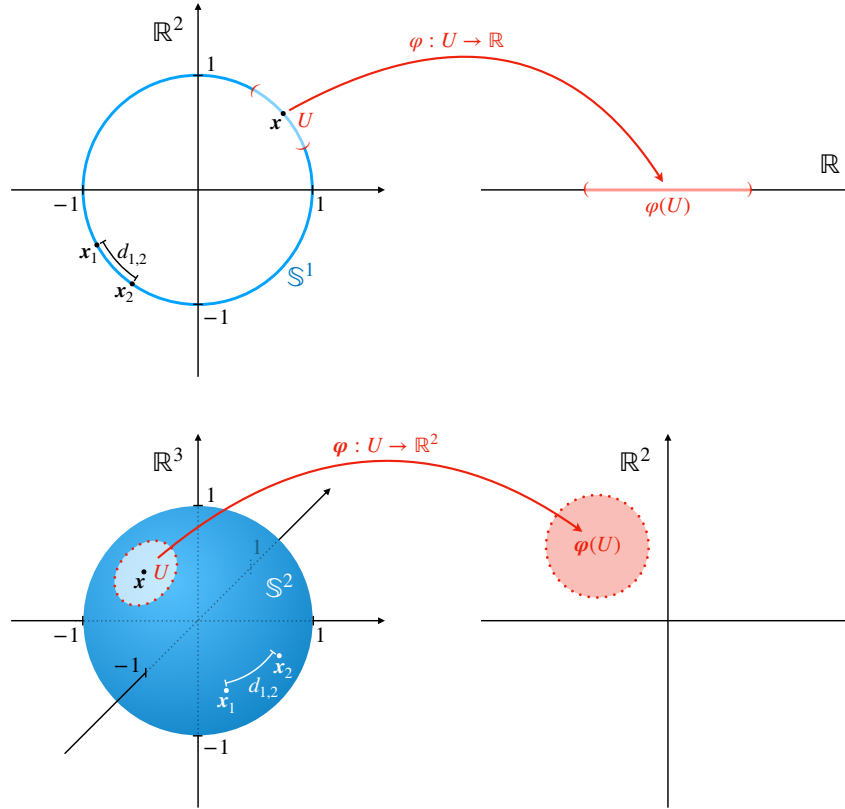


Figure 1.8: Schematics that illustrate the defining property of manifolds and geodesic distances for two manifolds: the unit circle $\mathbb{S}^1$ (top) and the unit sphere $\mathbb{S}^2$ (bottom).

- (ii) The *unit sphere* $\mathbb{S}^2 := \{\mathbf{x} \in \mathbb{R}^3 : \|\mathbf{x}\| = 1\}$ is a 2-manifold. For each point $\mathbf{x} \in \mathbb{S}^2$ there exists a neighbourhood U that can be mapped to an open set in $\mathbb{R}^2$, as shown in Figure 1.8 (bottom), while the chart $\varphi : U \rightarrow \mathbb{R}^2$ can be derived, e.g., from the polar coordinates of each point in $\mathbb{S}^2$.

When a manifold M enjoys some additional structure (M is a so-called *Riemannian manifold*), we can define the *geodesic distance* $d_{1,2}$ of two points $\mathbf{x}_1, \mathbf{x}_2 \in M$ as the length of the shortest path in M that connects $\mathbf{x}_1$ and $\mathbf{x}_2$.

Example 1.40. (i) For $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{S}^1$, the geodesic distance $d_{1,2}$ is the length of the (shorter) arc that connects $\mathbf{x}_1$ and $\mathbf{x}_2$; see Figure 1.8 (top).

- (ii) For $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{S}^2$, $d_{1,2}$ equals the *great-circle distance* of $\mathbf{x}_1$ and $\mathbf{x}_2$; see Figure 1.8 (bottom).

Subsequently, we seek a mapping of the data on a manifold M in $\mathbb{R}^d$ to a lower dimensional space $\mathbb{R}^q$ so that we *preserve the geodesic distances* between the sam-

ples on the manifold, as opposed to their Euclidean distances in the ambient space $\mathbb{R}^d$, as much as possible.

If the geodesic distances of the samples were known, then we could simply apply MDS to them. But typically we do *not* know them, or even what the manifold is, so *learning* them becomes part of the problem, which is why non-linear dimensionality reduction is sometimes called *manifold learning*. The key to solving the problem is the heuristic that the *local Euclidean geometry* of the samples (e.g., Euclidean distances between neighbouring or nearby samples) *is worth preserving*,¹⁵ since it is likely to be a reasonable “proxy” of the manifold geometry, whereas the *global Euclidean geometry* (e.g., Euclidean distances between samples that are far apart) *is not*.

Sammon mapping, as defined in (1.38) above, is a rudimentary embodiment of the local geometry preservation idea since it prioritises preservation small distances.

Isometric feature mapping (ISOMAP) (Tenenbaum et al., 2000) approximates the distance between each pair samples along the manifold by the shortest path between them on a *graph* constructed from nearest neighbour relationships. Subsequently, MDS is applied to these approximate distances to form the mapping to a lower dimensional space.

Local linear embedding (Roweis and Saul, 2000) develops first an approximation of each sample as a linear combination of its neighbouring samples. In other words, this establishes an approximate *linear relationship* between the sample and its nearest neighbours. Points in a lower dimensional space are then sought so that they preserve these linear relationships as closely as possible.

Local multidimensional scaling (local MDS) (Chen and Buja, 2009) is an extension of ordinary MDS, which we shall study in a bit more detail here. Local MDS first determines a set $\mathcal{N} \subset \{1, \dots, n\}^2$ neighbourhood relations such that $(i, j) \in \mathcal{N}$ if sample $\mathbf{x}_i$ is among the K -nearest neighbours of sample $\mathbf{x}_j$ or vice versa. The modified stress objective function is then defined as

$$S_L(\underbrace{\mathbf{z}_1, \dots, \mathbf{z}_n}_{\in \mathbb{R}^q}) := \sum_{(i,j) \in \mathcal{N}} (d_{i,j} - \|\mathbf{z}_i - \mathbf{z}_j\|)^2 + w \sum_{(i,j) \notin \mathcal{N}} (D - \|\mathbf{z}_i - \mathbf{z}_j\|)^2,$$

where $w > 0$ and $D > 0$ are tuning parameters. The first term encourages mapping that preserves distances between neighbouring samples, while in the second term D is chosen *large* so that non-neighbouring samples are mapped far

¹⁵This could be motivated by noting that the charts that characterise the manifold are locally *approximately linear* since they are differentiable.

apart by default. However, simultaneously w is chosen *small* so that the second term gets de-prioritised. If we let $D \rightarrow \infty$ and $w \sim \frac{\tau}{2D}$ for some $\tau > 0$, we get in the limit a simpler objective function

$$S_L(\mathbf{z}_1, \dots, \mathbf{z}_n) := \sum_{(i,j) \in \mathcal{N}} (d_{i,j} - \|\mathbf{z}_i - \mathbf{z}_j\|)^2 - \tau \sum_{(i,j) \notin \mathcal{N}} \|\mathbf{z}_i - \mathbf{z}_j\| + \text{constant}, \quad (1.41)$$

with only one tuning parameter τ , which is used in practical implementations of local MDS. (The constant in (1.41) depends on D but not on $\mathbf{z}_1, \dots, \mathbf{z}_n$, whereby it is immaterial as far as minimisation of $S_L(\mathbf{z}_1, \dots, \mathbf{z}_n)$ is concerned.)

1.9.3 t -distributed stochastic neighbour embedding

The *t -distributed stochastic neighbour embedding* (t -SNE) (van der Maaten and Hinton, 2008) is arguably the most popular non-linear dimensionality reduction method currently, in particular for visualisation of high-dimensional data. (However, it has an important caveat that limits its applicability to feature construction, which we discuss in Remark 1.42, below.)

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, we define their *probabilistic* (Gaussian) *similarities*

$$p_{j|i} := \frac{\exp\left(-\frac{1}{2\sigma_i^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\frac{1}{2\sigma_i^2} \|\mathbf{x}_i - \mathbf{x}_k\|^2\right)}, \quad j \neq i,$$

$$p_{i|i} := 0,$$

where $\sigma_i > 0$ for any $i = 1, \dots, n$ is a hyperparameter, the choice of which will be addressed shortly. Note that, by construction, the similarities emphasise *small distances*. For any $i = 1, \dots, n$, the values $p_{1|i}, \dots, p_{n|i} \geq 0$ form a discrete probability distribution, $\sum_{j=1}^n p_{j|i} = 1$, where each $j \neq i$ is furnished with a probability that is proportional to the probability density at the location $\mathbf{x}_j$ under a multivariate normal distribution $N(\mathbf{x}_i, \sigma_i^2 I_d)$ on $\mathbb{R}^d$. If we define the *symmetrisation*

$$p_{ij} := \frac{p_{j|i} + p_{i|j}}{2n}, \quad \text{for any } i, j = 1, \dots, n,$$

then

$$\sum_{i=1}^n \sum_{j=1}^n p_{ij} = \frac{1}{2n} \left(\sum_{j=1}^n \underbrace{\sum_{i=1}^n p_{i|j}}_{=1} + \sum_{i=1}^n \underbrace{\sum_{j=1}^n p_{j|i}}_{=1} \right) = \frac{n+n}{2n} = 1,$$

whereby p_{ij} , $(i, j) \in \{1, \dots, n\}^2$, form a discrete (joint) probability distribution on $\{1, \dots, n\}^2$, conveying symmetric similarities between the samples $\mathbf{x}_1, \dots, \mathbf{x}_n$.

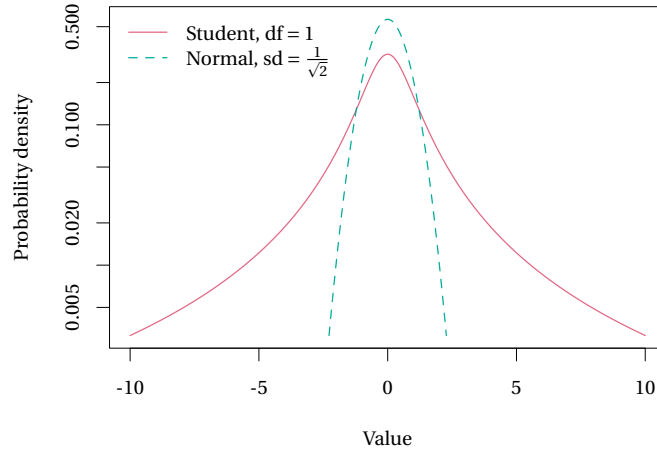


Figure 1.9: Comparison between the Student- t density with one degree of freedom (used in t -SNE) and the normal density with standard deviation $\frac{1}{\sqrt{2}}$ (used in SNE). Note that the y -axis is logarithmic.

The objective of t -SNE is now to map $\mathbf{x}_1, \dots, \mathbf{x}_n$ to an embedding $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^q$ while endeavouring to preserve their similarities. To define probabilistic similarities between $\mathbf{y}_1, \dots, \mathbf{y}_n$ in $\mathbb{R}^q$, a *Student- t* density with one degree of freedom is chosen instead of a Gaussian density, so that

$$q_{ij} := \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}, \quad i \neq j,$$

$$q_{ii} := 0,$$

which form another probability distribution on $\{1, \dots, n\}^2$. The Student- t density is *heavy-tailed*, which encourages samples $\mathbf{x}_i$ and $\mathbf{x}_j$ that are far apart to be mapped to points $\mathbf{y}_i$ and $\mathbf{y}_j$ that are even further apart — more than a comparable Gaussian density would. This choice of density helps alleviate the so-called *crowding problem* (van der Maaten and Hinton, 2008, Sec. 3.2) that may occur when trying to represent data with just moderate intrinsic dimensionality in a lower dimensional space, which affects the *stochastic neighbour embedding* (SNE) (Hinton and Roweis, 2002), a precursor of t -SNE, which employs a Gaussian density instead of Student- t — see Figure 1.9 for a comparison.

In t -SNE, the discrepancy between the similarities in $\mathbb{R}^d$ and $\mathbb{R}^q$, i.e., the discrete probability distributions $P := \{p_{ij}\}_{(i,j) \in \{1, \dots, n\}^2}$ and $Q := \{q_{ij}\}_{(i,j) \in \{1, \dots, n\}^2}$, is measured using the *Kullback–Leibler divergence* of P from Q ,

$$D_{\text{KL}}(P \parallel Q) := \sum_{i \neq j} \sum p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

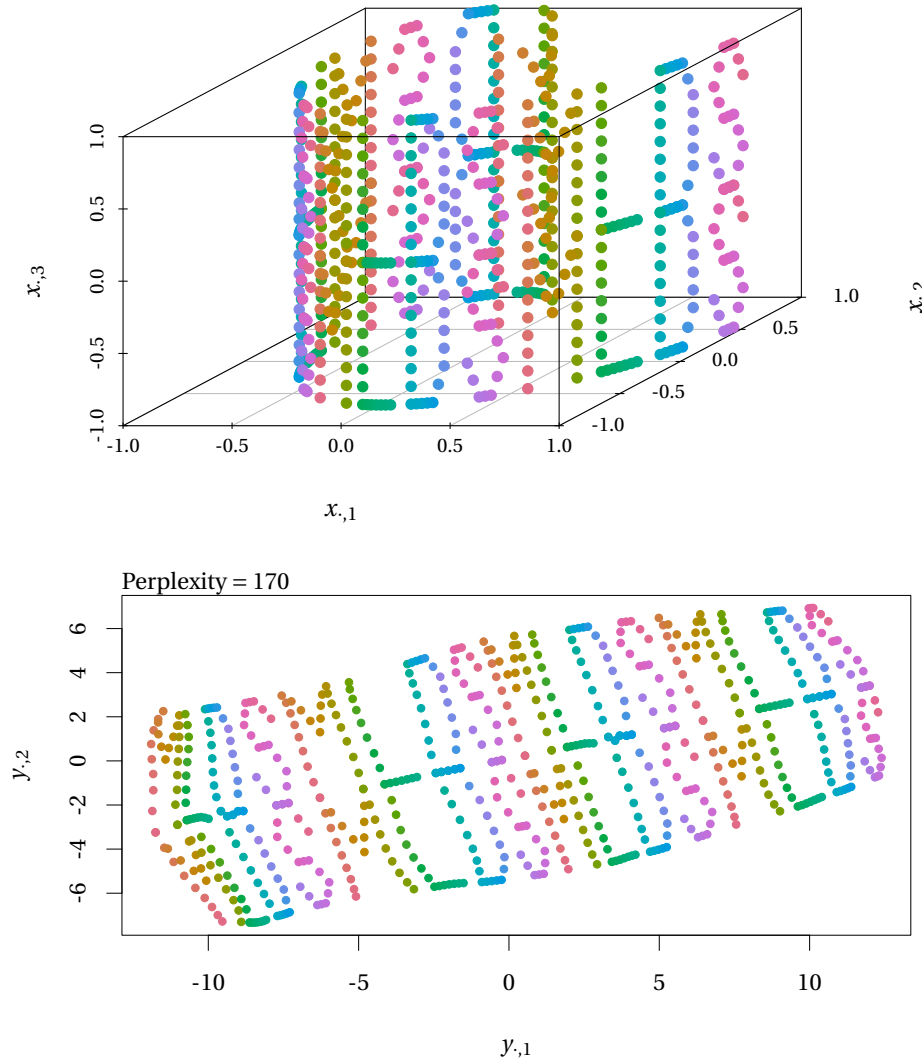


Figure 1.10: A data set that is confined to a non-linear 2-manifold in $\mathbb{R}^3$ (top) and its t -SNE in $\mathbb{R}^2$ (bottom).

which is an asymmetric measure of how much Q differs from the “true” distribution P . The initial values of the embedding $\mathbf{y}_1, \dots, \mathbf{y}_n$ are chosen at random. They are then iteratively updated to minimise $D_{\text{KL}}(P \parallel Q)$ using gradient descent.

The hyperparameters $\sigma_1, \dots, \sigma_n$ are chosen as follows. For any $i = 1, \dots, n$, the *perplexity* of the probability distribution $P_i := \{p_{j|i}\}_{j=1, \dots, n}$, is defined as

$$\text{Perp}(P_i) := 2^{H(P_i)},$$

where $H(P_i)$ is the (discrete) *Shannon entropy* $H(P_i) := -\sum_{j=1}^n p_{j|i} \log_2 p_{j|i}$ and $\log_2$ is the base-2 logarithm. *van der Maaten and Hinton (2008, p. 2582)* suggest that $\text{Perp}(P_i)$, which is a function of σ_i , can be interpreted as a smooth mea-

sure of the effective number of neighbours “seen” by P_i . The hyperparameter selection strategy is then to fix a (global) *target perplexity* and for each $i = 1, \dots, n$ find $\sigma_i > 0$ such that $\text{Perp}(P_i)$ equals the target perplexity using a root-finding algorithm. This results in adaptation where denser regions of the data set receive a smaller value of σ_i compared to sparser regions. An example of a two-dimensional t -SNE of a data set concentrated on a 2-manifold in $\mathbb{R}^3$ is given in Figure 1.10.

Remark 1.42. t -SNE cannot process *new samples* added to the data set without re-training the entire embedding since the mapping $\mathbf{x}_i \mapsto \mathbf{y}_i$ is not explicit. Moreover, since the objective function $D_{\text{KL}}(P \parallel Q)$ is non-convex with respect to $\mathbf{y}_1, \dots, \mathbf{y}_n$ and the initial values are random, we cannot expect the results to be stable when re-training the embedding with added samples. Evidently, this limits the usefulness of t -SNE for feature construction, although [van der Maaten \(2009\)](#) has suggested training a *supervised model* to learn the embedding as a possible workaround.

2 Density estimation

2.1 Overview of density estimation

We assume that $\mathbf{X}_1, \dots, \mathbf{X}_n$ are d -dimensional random vectors that all follow a *probability density function* (pdf) g_X . Denoting by $X_{i,1}, \dots, X_{i,d}$ the components of $\mathbf{X}_i$ for any $i = 1, \dots, n$, we recall that

$$\mathbf{P}[X_{i,1} \leq x_1, \dots, X_{i,d} \leq x_d] = \int_{-\infty}^{x_1} \cdots \int_{-\infty}^{x_d} g_X(u_1, \dots, u_d) du_1 \cdots du_d$$

for any $x_1, \dots, x_d \in \mathbb{R}$.

Given a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, which are seen as realisations of $\mathbf{X}_1, \dots, \mathbf{X}_n$, our task is to try to *estimate* the underlying pdf g_X . There are two main approaches to density estimation.

In *parametric density estimation*, we postulate that there is a family $\mathcal{G}$ of pdfs $g(\cdot; \boldsymbol{\theta})$ parameterised by $\boldsymbol{\theta} \in \Theta$, where Θ is a subset of $\mathbb{R}^p$ for some $p \in \mathbb{N}$, such that the true pdf g_X is a member of this family, namely,

$$g_X = g(\cdot; \boldsymbol{\theta}_0)$$

for some (unique) $\boldsymbol{\theta}_0 \in \Theta$. Then our task is to estimate $\boldsymbol{\theta}_0$, the difficulty of which depends on the breadth and characteristics of the family $\mathcal{G}$.

Example 2.1. The family $\mathcal{G}$ could, for example, consist of all d -dimensional *multivariate normal distributions*, which can be parameterised by their *mean* $\boldsymbol{\mu} \in \mathbb{R}^d$ and *covariance matrix* $\Sigma \in \mathbb{R}^{d \times d}$, i.e., $\boldsymbol{\theta} = (\boldsymbol{\mu}, \Sigma)$. Recalling that a covariance matrix is always positive semi-definite,

$$\Theta = \mathbb{R}^d \times \mathbb{S}_{\geq 0}^d,$$

where $\mathbb{S}_{\geq 0}^d$ is the space of symmetric positive semi-definite $d \times d$ matrices. Since the covariance matrix Σ as a symmetric matrix has at most $\frac{d(d-1)}{2}$ free parameters, we can embed Θ in the space $\mathbb{R}^{d + \frac{d(d-1)}{2}} = \mathbb{R}^{\frac{d(d+1)}{2}}$.

In *non-parametric density estimation*, we do not postulate a parametric family of pdfs, but in principle instead seek a pdf from the universe of all pdfs on $\mathbb{R}^d$. However, the non-parametric density estimators we use in practice, *histograms* and *kernel density estimators*, do not cover entire universe of all pdfs, but are instead able to *approximate* almost any pdf in the asymptotic limit where the number of samples n tends to infinity. We will return to non-parametric density estimation shortly.

Remark 2.2. The strength of parametric density estimation vis-à-vis its non-parametric counterpart is (typically) better *efficiency* (in terms of lower error variance). But this holds provided $\mathcal{G}$ is *correctly specified* (the true pdf belongs to $\mathcal{G}$), which can be difficult to ensure in practice. As a rule, parametric estimation is better suited to situations where the *number of samples is small*, and efficiency is important, while non-parametric methods become more competitive as the number of samples increases.

2.2 Parametric density estimation

2.2.1 Maximum likelihood estimation

Maximum likelihood is the standard method for parametric density estimation. Returning to the setting of Section 2.1 and assuming that the random vectors $\mathbf{X}_1, \dots, \mathbf{X}_n$ are *mutually independent*, their joint pdf can be written as

$$g_{\mathbf{X}_1, \dots, \mathbf{X}_n}(\mathbf{x}_1, \dots, \mathbf{x}_n) = \prod_{i=1}^n g_{\mathbf{X}}(\mathbf{x}_i) = \prod_{i=1}^n g(\mathbf{x}_i; \boldsymbol{\theta}_0). \quad (2.3)$$

Fixing the observed values $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and *releasing* the parameter value $\boldsymbol{\theta}_0 \in \Theta$, we obtain the *likelihood function*

$$L(\boldsymbol{\theta}) := L(\boldsymbol{\theta}; \mathbf{x}_1, \dots, \mathbf{x}_n) := \prod_{i=1}^n g(\mathbf{x}_i; \boldsymbol{\theta}), \quad \boldsymbol{\theta} \in \Theta. \quad (2.4)$$

The *maximum likelihood estimate* of $\boldsymbol{\theta}$, denoted by $\hat{\boldsymbol{\theta}}$ is then defined as the maximiser of the likelihood function,

$$\hat{\boldsymbol{\theta}} := \arg \max_{\boldsymbol{\theta} \in \Theta} L(\boldsymbol{\theta}).$$

The likelihood function is defined as a product (2.4), which is often computationally cumbersome to deal with. Hence, it is convenient to take a logarithmic transform of it, giving rise to the *log-likelihood function*

$$\ell(\boldsymbol{\theta}) := \log L(\boldsymbol{\theta}) = \sum_{i=1}^n \log g(\mathbf{x}_i; \boldsymbol{\theta}), \quad \boldsymbol{\theta} \in \Theta,$$

which is additive. Since $x \mapsto \log x$ is strictly increasing, $L(\boldsymbol{\theta})$ and $\ell(\boldsymbol{\theta})$ are equivalent as far as maximisation is concerned. Thereby, the maximum likelihood estimate can equivalently be obtained by maximising the log-likelihood,

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta} \in \Theta} \ell(\boldsymbol{\theta}).$$

Remark 2.5. The maximum likelihood *estimate* $\hat{\theta} = \hat{\theta}(\mathbf{x}_1, \dots, \mathbf{x}_n)$ is a *non-random* quantity that depends on the observations $\mathbf{x}_1, \dots, \mathbf{x}_n$. In some theoretical developments, it is useful to distinguish it from the corresponding *estimator* where $\mathbf{x}_1, \dots, \mathbf{x}_n$ have been replaced by the underlying random vectors $\mathbf{X}_1, \dots, \mathbf{X}_n$, i.e.,

$$\hat{\theta}(\mathbf{X}_1, \dots, \mathbf{X}_n),$$

which is a *random variable*.

Remark 2.6. The log-likelihood function can be always adjusted by adding (or subtracting) a term that depends on the data $\mathbf{x}_1, \dots, \mathbf{x}_n$ but not on the parameter vector $\boldsymbol{\theta}$, without changing the maximiser.

Remark 2.7. In general, the maximum likelihood estimate *need not be unique* (there may be multiple maximisers) but in most practically relevant cases it is.

For some distributions, the maximum likelihood estimate can be solved *analytically*, like in the following example. But more often than not, one needs to resort to *numerical optimisation*.

Example 2.8 (continuation of Example 2.1). Returning to the multivariate normal case, $g(\cdot; \boldsymbol{\theta}) = \phi(\cdot; \boldsymbol{\mu}, \Sigma)$ is the pdf of the *multivariate normal distribution* $N(\boldsymbol{\mu}, \Sigma)$, i.e.,

$$\phi(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right), \quad \mathbf{x} \in \mathbb{R}^d.$$

The edge case where Σ is not invertible is troublesome and must be ruled out since the above pdf is then ill-defined. Hence, we assume that Σ is *positive definite* (not merely positive semi-definite). Taking logarithms and summing over all observations, we get

$$\ell(\boldsymbol{\mu}, \Sigma) = -\frac{n}{2} \log(\det(\Sigma)) - \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x}_i - \boldsymbol{\mu}), \quad (2.9)$$

where we have also removed the (immaterial) term $-\frac{nd}{2} \log(2\pi)$ following Remark 2.6.

The log likelihood (2.9) can be shown to have a *unique maximiser* $(\hat{\boldsymbol{\mu}}, \hat{\Sigma})$ (Anderson and Olkin, 1985). To find it, we need the following vector and matrix calculus rules, for $\mathbf{x} \in \mathbb{R}^d$ and invertible $A \in \mathbb{R}^{d \times d}$,

$$\nabla_{\mathbf{x}} (\mathbf{x}^T A \mathbf{x}) = 2A\mathbf{x} \in \mathbb{R}^d, \quad (2.10)$$

$$\nabla_A (\mathbf{x}^T A^{-1} \mathbf{x}) = -A^{-1} \mathbf{x} \mathbf{x}^T A^{-1} \in \mathbb{R}^{d \times d}, \quad (2.11)$$

$$\nabla_A \log(\det(A)) = (A^{-1})^T \in \mathbb{R}^{d \times d}. \quad (2.12)$$

Applying (2.10), we first get

$$\nabla_{\boldsymbol{\mu}} \ell(\boldsymbol{\mu}, \Sigma) = \Sigma^{-1} \left(\sum_{i=1}^n \mathbf{x}_i - n\boldsymbol{\mu} \right),$$

and solving for $\nabla_{\boldsymbol{\mu}} \ell(\hat{\boldsymbol{\mu}}, \hat{\Sigma}) = \mathbf{0}$, we obtain

$$\hat{\boldsymbol{\mu}} = \bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i,$$

the *sample mean*. Applying (2.11) and (2.12), we get further

$$\nabla_{\Sigma} \ell(\boldsymbol{\mu}, \Sigma) = -\frac{n}{2} \Sigma^{-1} + \frac{1}{2} \sum_{i=1}^n \Sigma^{-1} (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \Sigma^{-1}.$$

Then, $\nabla_{\Sigma} \ell(\hat{\boldsymbol{\mu}}, \hat{\Sigma}) = 0$ is equivalent to

$$-\frac{n}{2} \hat{\Sigma}^{-1} + \frac{1}{2} \sum_{i=1}^n \hat{\Sigma}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \hat{\Sigma}^{-1} = 0,$$

where we substituted $\hat{\boldsymbol{\mu}} = \bar{\mathbf{x}}$. Pre- and post-multiplying the equation by $\hat{\Sigma}$, it simplifies to

$$-\frac{n}{2} \hat{\Sigma} + \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = 0,$$

from which we solve

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T,$$

the *sample covariance matrix*.

2.2.2 Asymptotic properties

Asymptotic theory has been derived to validate maximum likelihood estimation; see, e.g., van der Vaart (1998, Chapter 5). Under rather general conditions, maximum likelihood estimators are *consistent* in the sense that the maximum likelihood estimator $\hat{\boldsymbol{\theta}}$ converges to the true parameter value $\boldsymbol{\theta}_0$ as the number of samples n tends to infinity. Similarly, the estimator can rather generally be shown to be *asymptotically normal*, which means that the rescaled estimation error

$$\sqrt{n}(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}_0)$$

converges towards a multivariate normal distribution as $n \rightarrow \infty$. Asymptotic normality is useful in *inference* (construction of *confidence intervals* and *hypothesis tests*).

Remark 2.13. We assumed at the outset that the random vectors $X_1, \dots, X_n$ generating the data $\mathbf{x}_1, \dots, \mathbf{x}_n$ are *mutually independent*. Independence is crucial for the joint pdf (2.3) to hold, but it is *not essential* for the validity of the maximum likelihood estimator in general. It is possible to show that the maximum likelihood estimator *remains consistent and asymptotically normal* (Crowder, 1976) under quite general forms of dependent observations, even when the dependence is neglected in the derivation of the estimator. For example, in the case of the multivariate distribution (Example 2.8), the sample mean and covariance matrix are consistent estimators of the population mean and covariance matrix, respectively, under dependent observations satisfying a weak property known as *ergodicity*.¹⁶

2.3 Estimation of mixtures

2.3.1 Mixture model

A common difficulty with applying parametric density estimation to large data sets is that standard parametric families of pdfs struggle with accommodating the complexities of the data. Data sets where samples appear in multiple *clusters* pose a particular problem as they would necessitate a *multimodal* distribution — a density with *multiple peaks*.

Fortunately, multimodal distributions are easy to build from standard families of distributions using *mixtures*. Let $q \geq 2$ and let $g_1(\cdot; \boldsymbol{\theta}_1), \dots, g_q(\cdot; \boldsymbol{\theta}_q)$ be pdfs on $\mathbb{R}^d$ parameterised by $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_q$, respectively. Then their mixture, another pdf $h(\cdot)$ on $\mathbb{R}^d$, is defined via

$$h(\mathbf{x}) := h(\mathbf{x}; \boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_q, \pi_1, \dots, \pi_q) := \sum_{j=1}^q \pi_j g_j(\mathbf{x}; \boldsymbol{\theta}_j), \quad \mathbf{x} \in \mathbb{R}^d, \quad (2.14)$$

where $\pi_1, \dots, \pi_q > 0$ are the *mixing weights*, which must satisfy $\sum_{j=1}^q \pi_j = 1$. Sometimes the component pdfs $g_1(\cdot; \boldsymbol{\theta}_1), \dots, g_q(\cdot; \boldsymbol{\theta}_q)$ are chosen from the *same family*, but with different parameter values, i.e.,

$$g_j(\cdot; \boldsymbol{\theta}_j) = g(\cdot; \boldsymbol{\theta}_j), \quad j = 1, \dots, q,$$

but this is not required in general.

As a rule, the mixture model is more convenient to analyse using a generative representation with a *latent variable*. To this end, we introduce a d -dimensional

¹⁶https://en.wikipedia.org/wiki/Ergodic_theory

random vector $\mathbf{X}$ and a *categorical random variable* Z with values in $\{1, \dots, q\}$ such that

$$\mathbf{X} \mid \{Z = j\} \sim g_j(\cdot; \boldsymbol{\theta}_j), \quad j = 1, \dots, q, \quad (2.15)$$

and

$$\mathbf{P}[Z = j] = \pi_j, \quad j = 1, \dots, q. \quad (2.16)$$

Then $\mathbf{X}$ follows the pdf $h(\cdot)$, given by (2.14), and Z serves as an indicator of which component of the mixture the observed value stems from. Observing $\mathbf{X}$ only, Z remains latent, however.

Example 2.17. Arguably the most common mixture model is specified by

$$g_i(\cdot; \boldsymbol{\theta}_i) := \phi(\cdot; \boldsymbol{\mu}_i, \Sigma_i), \quad i = 1, \dots, q,$$

with some $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_q \in \mathbb{R}^d$ and symmetric positive definite $\Sigma_1, \dots, \Sigma_q \in \mathbb{R}^{d \times d}$, where $\phi(\cdot; \boldsymbol{\mu}, \Sigma)$ is the pdf of the *multivariate normal distribution* with mean vector $\boldsymbol{\mu}$ and covariance matrix Σ . This is called the *Gaussian mixture model* and exemplified in Figure 2.1.

2.3.2 EM algorithm

To estimate the mixture model, we could in principle proceed with *maximum likelihood* using the pdf (2.14). Unfortunately, the pdf as a *sum* does not play nicely with *multiplication* and *logarithms*. Hence, this approach is in most cases a non-starter.

Fortunately, mixture models can be estimated more efficiently using the *expectation–maximisation* (EM) algorithm, popularised by Dempster et al. (1977). (See also Hastie et al. (2009, Section 8.5).) The EM algorithm is based on the representation using the latent variable Z ; equations (2.15) and (2.16). For convenience, we collect all the parameters $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_q$ and $\pi_1, \dots, \pi_q$ into a single vector $\boldsymbol{\theta}$. Suppose that the data $\mathbf{x}_1, \dots, \mathbf{x}_n$ are generated by mutually independent random variables $\mathbf{X}_1, \dots, \mathbf{X}_n \sim h(\cdot)$ and that $Z_1, \dots, Z_n$ are the corresponding categorical latent variables with realised values $z_1, \dots, z_n$. We will then introduce their joint likelihood function

$$L(\boldsymbol{\theta}; \mathbf{x}_1, \dots, \mathbf{x}_n, z_1, \dots, z_n) = \prod_{i=1}^n \prod_{j=1}^q (g_j(\mathbf{x}_i; \boldsymbol{\theta}_j) \pi_j)^{\mathbf{1}_{\{z_i=j\}}},$$

where

$$\mathbf{1}_{\{z_i=j\}} = \begin{cases} 1, & z_i = j, \\ 0, & z_i \neq j. \end{cases}$$

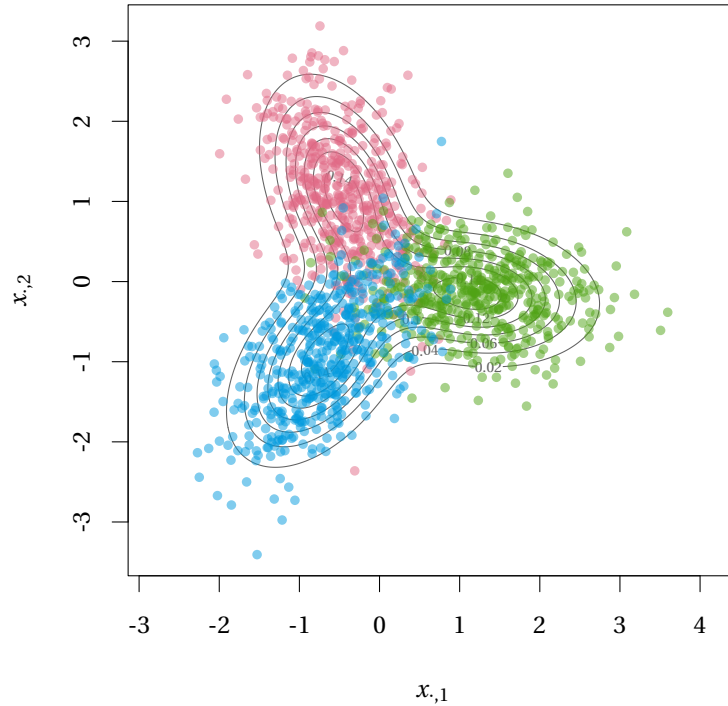


Figure 2.1: 1500 independent samples drawn from an equally-weighted Gaussian mixture model built from three bivariate normal distributions, $N_2\left(\begin{bmatrix} -0.47 \\ 1.11 \end{bmatrix}, \begin{bmatrix} 0.26 & -0.14 \\ -0.14 & 0.54 \end{bmatrix}\right)$ (red), $N_2\left(\begin{bmatrix} 1.19 \\ -0.15 \end{bmatrix}, \begin{bmatrix} 0.59 & -0.05 \\ -0.05 & 0.21 \end{bmatrix}\right)$ (green) and $N_2\left(\begin{bmatrix} -0.72 \\ -0.96 \end{bmatrix}, \begin{bmatrix} 0.35 & 0.19 \\ 0.19 & 0.45 \end{bmatrix}\right)$ (blue), respectively, with the contour lines of the mixture pdf superimposed.

Taking logarithms, we obtain the corresponding log-likelihood function

$$\ell(\boldsymbol{\theta}; \mathbf{x}_1, \dots, \mathbf{x}_n, z_1, \dots, z_n) = \sum_{i=1}^n \sum_{j=1}^q \mathbf{1}_{\{z_i=j\}} (\log g_j(\mathbf{x}_i; \boldsymbol{\theta}_j) + \log \pi_j).$$

The EM algorithm proceeds now *iteratively*. At each iteration, we keep the current parameter values stored in $\boldsymbol{\theta}^{(t)}$. (If this is the first iteration, these are some suitable initial values.) The iteration itself consists of two steps:

- (i) *Expectation*: We compute the expectation of the log-likelihood function with respect to $z_1, \dots, z_n$ by integrating over their distribution given $\boldsymbol{\theta}^{(t)}$ and $\mathbf{x}_1, \dots, \mathbf{x}_n$, namely,

$$\begin{aligned} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) &:= \mathbf{E}_{\boldsymbol{\theta}^{(t)}, \mathbf{x}_1, \dots, \mathbf{x}_n} [\ell(\boldsymbol{\theta}; \mathbf{x}_1, \dots, \mathbf{x}_n, Z_1, \dots, Z_n)] \\ &= \sum_{i=1}^n \sum_{j=1}^q \gamma_{i,j}^{(t)} (\log g_j(\mathbf{x}_i; \boldsymbol{\theta}_j) + \log \pi_j), \end{aligned} \quad (2.18)$$

where the probability $\gamma_{i,j}^{(t)} := \mathbf{P}_{\boldsymbol{\theta}^{(t)}, \mathbf{x}_1, \dots, \mathbf{x}_n} [Z_i = j]$ is known as the *responsibility* of the component j for the i -th sample $\mathbf{x}_i$.

- (ii) *Maximisation*: We obtain the new parameter values, which are stored in $\boldsymbol{\theta}^{(t+1)}$ by maximising $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$, namely,

$$\boldsymbol{\theta}^{(t+1)} := \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}).$$

The iteration is continued until convergence is attained.

Remark 2.19. The EM algorithm is *not* guaranteed to converge to a global maximum, but merely to a local one. Hence, if there are doubts about the optimality of the end result, experimentation with *different initial values* is warranted.

Example 2.20 (Continuation of Example 2.17). We will now study application of the EM algorithm in the case of the *Gaussian mixture model* of Example 2.17.

- (i) In the expectation step, the responsibility $\gamma_{i,j}^{(t)}$ can be computed using *Bayes's theorem* as

$$\gamma_{i,j}^{(t)} = \frac{\pi_j^{(t)} \phi(\mathbf{x}_i; \boldsymbol{\mu}_j^{(t)}, \Sigma_j^{(t)})}{\sum_{j'=1}^q \pi_{j'}^{(t)} \phi(\mathbf{x}_i; \boldsymbol{\mu}_{j'}^{(t)}, \Sigma_{j'}^{(t)})}, \quad i = 1, \dots, n, \quad j = 1, \dots, q.$$

Substituting the multivariate normal density into (2.18), we get moreover

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \sum_{i=1}^n \sum_{j=1}^q \gamma_{i,j}^{(t)} \left(-\frac{d}{2} \log(2\pi) - \frac{1}{2} \log(\det(\Sigma_j)) - \frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \Sigma_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) + \log \pi_j \right).$$

- (ii) Thanks to the *additive structure* of $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$, to maximise it with respect to $\pi_1, \dots, \pi_q$ in the maximisation step, it suffices to maximise

$$\sum_{i=1}^n \sum_{j=1}^q \gamma_{i,j}^{(t)} \log \pi_j$$

under the constraints $\pi_1, \dots, \pi_q > 0$ and $\sum_{j=1}^q \pi_j = 1$. This is analogous to maximum likelihood estimation of the *multinomial distribution* and the solution (which can be obtained using the Lagrange multiplier method) is

$$\pi_j^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_{i,j}^{(t)}, \quad j = 1, \dots, q.$$

Similarly, the maximisation of $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$ with respect to $(\boldsymbol{\mu}_j, \Sigma_j)$ for any $j = 1, \dots, q$ can be accomplished by maximising

$$\sum_{i=1}^n \gamma_{i,j}^{(t)} \left(-\frac{1}{2} \log(\det(\Sigma_j)) - \frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \Sigma_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) \right).$$

The solution can be obtained by modifying the derivation of the maximum likelihood estimate in the (standard) multivariate normal setting (Example 2.8), yielding

$$\begin{aligned} \boldsymbol{\mu}_j^{(t+1)} &= \frac{1}{\sum_{i=1}^n \gamma_{i,j}^{(t)}} \sum_{i=1}^n \gamma_{i,j}^{(t)} \mathbf{x}_i, \\ \Sigma_j^{(t+1)} &= \frac{1}{\sum_{i=1}^n \gamma_{i,j}^{(t)}} \sum_{i=1}^n \gamma_{i,j}^{(t)} (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)}) (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)})^T. \end{aligned}$$

2.4 Histogram

2.4.1 Construction

Histograms are used ubiquitously to visualise distributions of samples. But they are also the simplest form of a *non-parametric density estimator*.

While histograms could in principle be constructed for *any dimensionality* of data, their visualisation becomes difficult in higher dimensions, save for two. Hence, we will develop here a histogram for one-dimensional data $x_1, \dots, x_n \in \mathbb{R}$ only. To construct the histogram, the real line is divided into disjoint intervals, *bins*. The bins need not be of equal size but they are commonly chosen to be so, an assumption we will also make here. Thus, for a reference point $x_0 \in \mathbb{R}$, *bin width* $h > 0$ and $k \in \mathbb{Z}$, we define the k -th bin as

$$B_k := [x_0 + (k-1)h, x_0 + kh) \subset \mathbb{R}. \quad (2.21)$$

The *number of observations* in B_k is then

$$\begin{aligned} N_k &:= \#\{i = 1, \dots, n : x_i \in B_k\} \\ &= \sum_{i=1}^n \mathbf{1}_{\{x_i \in B_k\}}, \end{aligned} \quad (2.22)$$

where

$$\mathbf{1}_{\{x \in B_k\}} := \begin{cases} 1, & x \in B_k, \\ 0, & x \notin B_k, \end{cases}$$

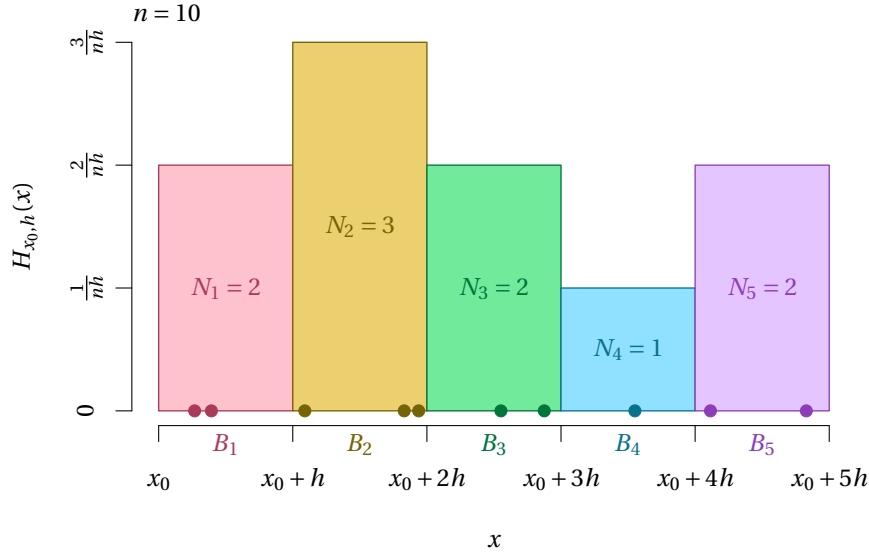


Figure 2.2: Construction of a histogram.

for any $x \in \mathbb{R}$. On the whole, and as illustrated in Figure 2.2, the histogram is then defined as

$$H_{x_0, h}(x) := H_{x_0, h}(x; x_1, \dots, x_n) := \frac{1}{nh} \sum_{k \in \mathbb{Z}} N_k \mathbf{1}_{\{x \in B_k\}}, \quad x \in \mathbb{R}.$$

Remark 2.23. A quantity closely related with the bin width h is the *bin count*, i.e., the smallest number of bins that cover the entire data set $x_1, \dots, x_n$. To define the bin count, we introduce

$$x_{\min} := \min\{x_1, \dots, x_n\},$$

$$x_{\max} := \max\{x_1, \dots, x_n\},$$

and set $x_0 := x_{\min}$. The bin count is then given by

$$\begin{aligned} K &:= \min\{k \in \mathbb{N} : x_0 + kh > x_{\max}\} \\ &= \min\left\{k \in \mathbb{N} : k > \frac{x_{\max} - x_{\min}}{h}\right\}. \end{aligned}$$

If we modify the last bin B_K to be the *closed* interval

$$[x_{\min} + K(h-1), x_{\min} + Kh],$$

then we can write the bin count succinctly as

$$K = \left\lceil \frac{x_{\max} - x_{\min}}{h} \right\rceil,$$

where $\lceil x \rceil := \min\{z \in \mathbb{Z} : z \geq x\}$ is the *ceiling* function. Given bin count $K \in \mathbb{N}$, we recover the bin width h from

$$h = \frac{x_{\max} - x_{\min}}{K}.$$

2.4.2 Histogram as density estimator

To understand why the histogram is a *density estimator*, we study its properties when the data $x_1, \dots, x_n$ have been generated by mutually independent and identically distributed random variables $X_1, \dots, X_n$ with pdf f , i.e.,

$$H_{x_0, h}(x) := H_{x_0, h}(x; X_1, \dots, X_n), \quad x \in \mathbb{R}.$$

This study also paves the way for the practically relevant question *how to choose the bin width* (or *bin count*).

We start by working out the *expectation* of the histogram:

$$\begin{aligned} \mathbf{E}[H_{x_0, h}(x)] &= \mathbf{E}\left[\frac{1}{nh} \sum_{k \in \mathbb{Z}} N_k \mathbf{1}_{\{x \in B_k\}}\right] && \text{(by definition)} \\ &= \frac{1}{nh} \sum_{k \in \mathbb{Z}} \mathbf{E}[N_k] \mathbf{1}_{\{x \in B_k\}} && \text{(only } N_k \text{ is random)} \\ &= \frac{1}{nh} \sum_{k \in \mathbb{Z}} \mathbf{E}\left[\sum_{i=1}^n \mathbf{1}_{\{X_i \in B_k\}}\right] \mathbf{1}_{\{x \in B_k\}} && \text{(by (2.22))} \\ &= \frac{1}{nh} \sum_{k \in \mathbb{Z}} \sum_{i=1}^n \mathbf{P}[X_i \in B_k] \mathbf{1}_{\{x \in B_k\}} && \text{(since } \mathbf{E}[\mathbf{1}_A] = \mathbf{P}[A]) \\ &= \frac{1}{nh} \sum_{k \in \mathbb{Z}} \sum_{i=1}^n \mathbf{P}[x_0 + (k-1)h \leq X_i < x_0 + kh] \mathbf{1}_{\{x \in B_k\}} && \text{(by (2.21))} \\ &= \sum_{k \in \mathbb{Z}} \frac{1}{h} \int_{x_0 + (k-1)h}^{x_0 + kh} f(u) du \mathbf{1}_{\{x \in B_k\}}. && (X_i \sim f(\cdot)) \end{aligned}$$

Comparing it with the value of the pdf, i.e., $f(x)$, we find the bias of the histogram,

$$\text{Bias}[H_{x_0, h}(x)] := \sum_{k \in \mathbb{Z}} \frac{1}{h} \int_{x_0 + (k-1)h}^{x_0 + kh} f(u) du \mathbf{1}_{\{x \in B_k\}} - f(x).$$

If the pdf f is a *continuous function* and if we let the bin width shrink to zero, $h \rightarrow 0$, we get¹⁷

$$\sum_{k \in \mathbb{Z}} \frac{1}{h} \int_{x_0 + (k-1)h}^{x_0 + kh} f(u) du \mathbf{1}_{\{x \in B_k\}} \rightarrow f(x),$$

¹⁷This can be justified using the *mean value theorem*. It implies that

$$\frac{1}{h} \int_{x_0 + (k-1)h}^{x_0 + kh} f(u) du = f(x_k^*)$$

for some $x_k^* \in B_k$. Hence,

$$\sum_{k \in \mathbb{Z}} f(x_k^*) \mathbf{1}_{\{x \in B_k\}} \rightarrow f(x), \quad h \rightarrow 0,$$

by the continuity of f .

i.e., $\text{Bias}[H_{x_0,h}(x)] \rightarrow 0$. A similar (but somewhat lengthier) derivation shows that the *variance* of the histogram is given by

$$\text{Var}[H_{x_0,h}(x)] = \frac{\sum_{k \in \mathbb{Z}} \frac{1}{h} \int_{x_0+(k-1)h}^{x_0+kh} f(u) du \left(1 - \int_{x_0+(k-1)h}^{x_0+kh} f(u) du\right) \mathbf{1}_{\{x \in B_k\}}}{nh}.$$

While the numerator tends to $f(x)$ as $h \rightarrow 0$, the denominator tends to zero, exploding the variance. This shows that there is *bias–variance tradeoff* — smaller bin size results in *lower bias* but also *higher variance*.

Theoretically, if the number of samples, n , tends to infinity, then we can let the bin side h go to zero with *asymptotically vanishing variance* provided

$$nh \rightarrow \infty,$$

that is h can go to zero but *not too fast* (relative to n). In practice, the bin width (or bin count) should thus be *tied to the number of samples*.

2.4.3 Choosing bin width or bin count

Since the variance of the histogram depends on the pdf f itself, there is *no* universal, optimal way of selecting the bin width h (or bin count K) at the outset. However, various *quasi-optimal* rules have been proposed under different assumptions, of which the following three are among the best known:

★ *Sturges' formula* (Sturges, 1926),

$$K := \left\lceil \frac{\log n}{\log 2} \right\rceil + 1,$$

is derived by (implicitly) assuming an approximate normal distribution for the data (Hyndman, 1995). It is the earliest rule published and the *default choice*, e.g., in R and NumPy, but it often results in *overly wide bins*, especially if the number of samples is large or if the data come from a distribution with heavier tails than the normal distribution. Hence, the following two often emerge as *better choices*:

★ *Scott's rule* (Scott, 1979) uses

$$h := \frac{3.49 \hat{\sigma}}{n^{\frac{1}{3}}},$$

where $\hat{\sigma}$ is the sample standard deviation of $x_1, \dots, x_n$. It is optimised, in terms of mean square error, for normally distributed data.

★ The *Freedman–Diaconis rule* (Freedman and Diaconis, 1981) specifies

$$h := \frac{2 \text{IQR}}{n^{\frac{1}{3}}},$$

replacing the standard deviation in Scott’s formula with the *interquartile range* IQR of the data $x_1, \dots, x_n$. The interquartile range makes the formula more robust to the extreme observations compared to Scott’s formula.

Remark 2.24. For visualisation purposes the bin width (or bin count) choice is ultimately subjective. Does one prefer a *smoother* or *noisier* representation of the data? There are good reasons to prefer a slightly noisy histogram.¹⁸ If a smoother representation is sought, it may be better to switch to a *kernel density estimator* (next topic) anyway.

2.5 Kernel density estimation

2.5.1 Construction

Kernel density estimation, pioneered by Parzen (1962) and Rosenblatt (1956), is the main method for non-parametric density estimation. Its key strength over a histogram is that the kernel density estimate can be ensured to be *continuous*, even *differentiable*, unlike the histogram which is inherently discontinuous. The kernel density estimate is also more straightforward to construct and analyse in *higher dimensions*.

To construct a kernel density estimate for data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, we need to first choose a *kernel function* $k : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ such that

$$\int_{\mathbb{R}^d} k(\mathbf{x}) d\mathbf{x} = 1 \tag{2.25}$$

and *bandwidth* $h > 0$. (We will review standard examples of kernels below in Example 2.27.) To construct the estimate, we create a “copy” of the kernel k for each sample such that k is *centred* at the sample. Moreover, we *scale* the argument of the kernel by the reciprocal of the bandwidth h . Finally, we *sum* the “copies” and *scale* them such that the end result is a pdf. More precisely, we define

$$\hat{f}_{k,h}(\mathbf{x}) := \hat{f}_{k,h}(\mathbf{x}; \mathbf{x}_1, \dots, \mathbf{x}_n) := \frac{1}{nh^d} \sum_{i=1}^n k(h^{-1}(\mathbf{x} - \mathbf{x}_i)), \quad \mathbf{x} \in \mathbb{R}^d.$$

Remark 2.26. (i) The bandwidth h is analogous to the *bin width* in a histogram.

¹⁸See: https://statmodeling.stat.columbia.edu/2009/10/23/variations_on_t/

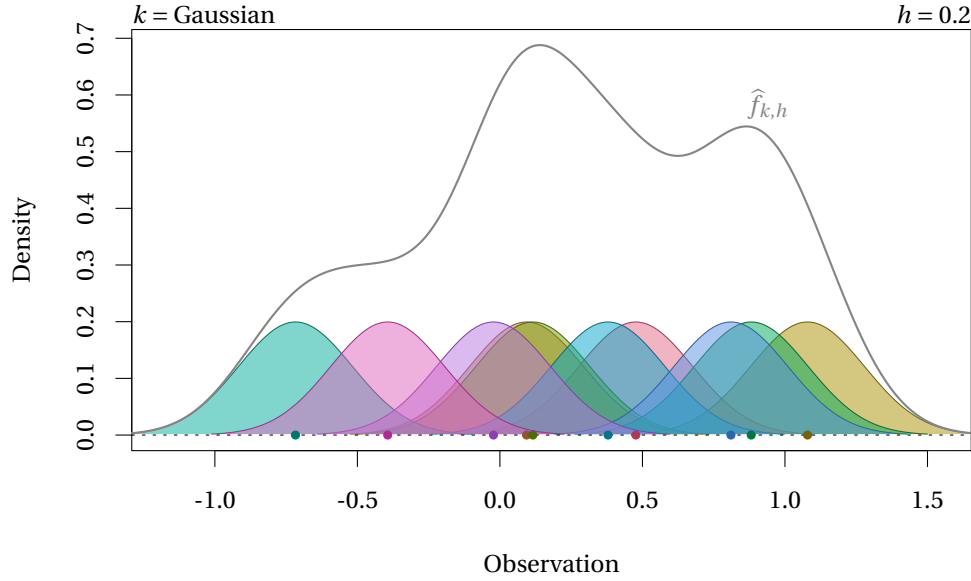


Figure 2.3: An example of a kernel density estimate and its constituents.

- (ii) The condition (2.25) on the kernel k is equivalent to assuming that it is a *pdf*. But it is key to note that choosing a particular kernel k is *not* an assumption that the data $\mathbf{x}_1, \dots, \mathbf{x}_n$ have been drawn from the pdf k . In fact, asymptotically, as the number of samples tends to infinity, the precise form of k becomes mostly *irrelevant*.
- (iii) The kernel density estimate *inherits* its *regularity properties* from the kernel. If k is continuous (resp. differentiable), then also $\hat{f}_{k,h}$ is continuous (resp. differentiable).

Example 2.27. In the case $d = 1$, the kernel k is typically chosen to be an *even* function, $k(x) = k(-x)$ for any $x \in \mathbb{R}$. Standard choices include:

★ *Gaussian:* $k(x) := \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, x \in \mathbb{R};$

★ *Triangular:* $k(x) := \begin{cases} 1 - |x|, & |x| \leq 1, \\ 0, & |x| > 1; \end{cases}$

★ *Rectangular:*¹⁹ $k(x) := \begin{cases} \frac{1}{2}, & |x| \leq 1, \\ 0, & |x| > 1; \end{cases}$

¹⁹The kernel density estimate with a rectangular kernel is not equivalent to a histogram.

$$\star \text{ Epanechnikov: } k(x) := \begin{cases} \frac{3(1-\frac{1}{5}x^2)}{4\sqrt{5}}, & |x| \leq \sqrt{5}, \\ 0, & |x| > \sqrt{5}; \end{cases}$$

$$\star \text{ Biweight: } k(x) := \begin{cases} \frac{15(1-x^2)^2}{16}, & |x| \leq 1, \\ 0, & |x| > 1. \end{cases}$$

For an illustration using the Gaussian kernel, see Figure 2.3. In higher dimensions it is common, albeit not necessary, to use *rotation-invariant* (*isotropic*) kernels constructed via

$$k(\mathbf{x}) \propto \tilde{k}(\|\mathbf{x}\|), \quad \mathbf{x} \in \mathbb{R}^d,$$

from a one-dimensional kernel $\tilde{k} : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$.

2.5.2 Properties and asymptotic behaviour

Like with histograms, it is helpful to study the *bias* and *variance* of kernel density estimates, and how they behave *asymptotically* as the number of samples increases. For simplicity, we consider the case $d = 1$ only.

To this end, we work with the kernel density *estimator* (a random variable)

$$\hat{f}_{k,h}(x) := \hat{f}_{k,h}(x; X_1, \dots, X_n) = \frac{1}{n} \sum_{i=1}^n \underbrace{\frac{1}{h} k\left(\frac{x - X_i}{h}\right)}_{=: Z_i}, \quad x \in \mathbb{R},$$

with mutually independent random variables $X_1, \dots, X_n$ that follow a differentiable pdf f with bounded derivative f' . Moreover, we assume that

$$\int_{\mathbb{R}} |x| k(x) dx < \infty, \quad \int_{\mathbb{R}} k(x)^2 dx < \infty, \quad \int_{\mathbb{R}} |x| k(x)^2 dx < \infty,$$

which are satisfied by all standard kernels.

Note that the random variables $Z_1, \dots, Z_n$ are also independent and identically distributed. Hence, we get

$$\begin{aligned} \mathbf{E}[\hat{f}_{k,h}(x)] &= \frac{1}{n} \sum_{i=1}^n \mathbf{E}[Z_i] = \mathbf{E}[Z_1], \\ \mathbf{Var}[\hat{f}_{k,h}(x)] &= \frac{1}{n^2} \sum_{i=1}^n \mathbf{Var}[Z_i] = \frac{\mathbf{Var}[Z_1]}{n}. \end{aligned}$$

The expectation $\mathbf{E}[\hat{f}_{k,h}(x)] = \mathbf{E}[Z_1]$ can be computed via

$$\mathbf{E}[\hat{f}_{k,h}(x)] = \int_{\mathbb{R}} \frac{1}{h} k\left(\frac{x-u}{h}\right) f(u) du = \int_{\mathbb{R}} k(y) f(x-hy) dy,$$

substituting $y := \frac{x-u}{h}$. Now,

$$\int_{\mathbb{R}} k(y) f(x-hy) dy = \int_{\mathbb{R}} k(y) f(x) dy + \int_{\mathbb{R}} k(y) (f(x-hy) - f(x)) dy, \quad (2.28)$$

where

$$\int_{\mathbb{R}} k(y) f(x) dy = f(x) \underbrace{\int_{\mathbb{R}} k(y) dy}_{=1} = f(x).$$

By the *mean value theorem*, $f(x-hy) - f(x) = hyf'(\xi_y)$ for some ξ_y between $x-hy$ and x . Hence,

$$\left| \int_{\mathbb{R}} k(y) (f(x-hy) - f(x)) dy \right| \leq h \int_{\mathbb{R}} k(y) |y| |f'(\xi_y)| dy \leq h \underbrace{\sup_{u \in \mathbb{R}} |f'(u)|}_{< \infty} \underbrace{\int_{\mathbb{R}} |y| k(y) dy}_{< \infty}, \quad (2.29)$$

whereby

$$\mathbf{E}[\hat{f}_{k,h}(x)] = \mathbf{E}[Z_1] = f(x) + O(h),$$

where $O(h)$ denotes a term bounded in absolute value by a constant times h . In particular,

$$\lim_{h \rightarrow 0} \mathbf{E}[\hat{f}_{k,h}(x)] = f(x),$$

that is, $\hat{f}_{k,h}(x)$ becomes asymptotically *unbiased* when $h \rightarrow 0$.

To compute the *variance*, it is useful to first recall that $\mathbf{Var}[Z_1] = \mathbf{E}[Z_1^2] - \mathbf{E}[Z_1]^2$, where

$$\mathbf{E}[Z_1^2] = \frac{1}{h} \int_{\mathbb{R}} \frac{1}{h} k\left(\frac{x-u}{h}\right)^2 f(u) du = \frac{1}{h} \int_{\mathbb{R}} k(y)^2 f(x+yh) dy.$$

Proceeding as in (2.28) and (2.29), we get

$$\int_{\mathbb{R}} k(y)^2 f(x+yh) dy = f(x) \int_{\mathbb{R}} k(y)^2 dy + \int_{\mathbb{R}} k(y)^2 (f(x+yh) - f(x)) dy,$$

where

$$\left| \int_{\mathbb{R}} k(y)^2 (f(x+yh) - f(x)) dy \right| \leq h \underbrace{\sup_{u \in \mathbb{R}} |f'(u)|}_{< \infty} \underbrace{\int_{\mathbb{R}} |y| k(y)^2 dy}_{< \infty} = O(h).$$

Putting everything together, we get

$$\begin{aligned} \mathbf{Var}[\hat{f}_{k,h}(x)] &= \frac{1}{n} (\mathbf{E}[Z_1^2] - \mathbf{E}[Z_1]^2) \\ &= \frac{1}{nh} \left(f(x) \int_{\mathbb{R}} k(y)^2 dy + O(h) \right) - \frac{1}{n} (f(x) + O(h))^2 \\ &= \frac{f(x) \int_{\mathbb{R}} k(y)^2 dy}{nh} + O\left(\frac{1}{n}\right) + O\left(\frac{h}{n}\right). \end{aligned} \quad (2.30)$$

The leading term $\frac{f(x) \int_{\mathbb{R}} k(y)^2 dy}{nh}$ reveals a *bias–variance tradeoff* similar to what we saw when studying histograms (Section 2.4.2). If we let $h \rightarrow 0$, the estimator’s bias tends to zero but its variance *explodes*. But again, asymptotically vanishing variance, $\text{Var}[\hat{f}_{k,h}(x)] \rightarrow 0$, is attainable when both $h \rightarrow 0$ and $n \rightarrow \infty$ as long as

$$nh \rightarrow \infty,$$

in line with the result for histograms.

2.5.3 Bandwidth selection

All approximate expressions for the variance (such as (2.30)) and *mean square error* of the kernel density estimator involve the unknown density f itself. Hence, universal rules for optimal bandwidth selection are *unobtainable*. However, various *quasi-optimal* bandwidth selection rules (under different assumptions) have been proposed.

The most basic form of bandwidth rule (default in R and SciPy), attributed to [Silverman \(1986\)](#), and some variants to [Scott \(1992\)](#), is derived assuming a *normal distribution* and uses

$$h := \frac{C \hat{s}}{n^{\frac{1}{5}}},$$

where C is a constant, specifically,

$$C := \begin{cases} 0.9 & \text{(Silverman's rule)} \\ 1.06 & \text{(Scott's rule)} \end{cases}$$

and $\hat{s}$ is an estimate of the *scale*, the sample standard deviation $\hat{\sigma}$, the scaled interquartile range $\frac{1}{1.34}\text{IQR}$ or the minimum of the two. This rule may produce *oversmoothed* estimates when the data are *multimodal* or *heavy-tailed*, however. More sophisticated bandwidth selection rules exist, including leave-one-out *cross validation*. For a comprehensive survey, see [Sheather \(2004\)](#).

3 Clustering

3.1 K -means clustering

3.1.1 Formulation

The objective of *clustering* is to develop algorithms that are able to automatically detect a *grouping* of samples in a data set based on a *dissimilarity* measure (e.g., Euclidean distance). In small, low-dimensional (one- or two-dimensional) data sets, such a grouping is not difficult to determine visually and subjectively, but the task becomes harder in *high dimensions* and when the *number of samples is large*, which is why proper algorithms are called for.

K-means clustering (Lloyd, 1982)²⁰ is one of the simplest clustering algorithms. The algorithm can be applied to a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, which it will split into $K \geq 2$ groups, called *clusters*, where K is a user-defined hyperparameter. (We will discuss ways to *evaluate* the results of clustering algorithms in Section 3.4. They can be used as guidance in the choice of K , in case an exploratory analysis or context does not provide any obvious candidate.) K -means clustering uses the Euclidean distance as its dissimilarity measure.

In K -means clustering, we split the set $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ of samples into K non-empty, disjoint subsets $\mathcal{C}_1, \dots, \mathcal{C}_K$, which are the clusters. For each cluster, we define its *mean* as the sample mean of its members,

$$\boldsymbol{\mu}_j := \frac{1}{\#\mathcal{C}_j} \sum_{\mathbf{x}_i \in \mathcal{C}_j} \mathbf{x}_i = \frac{1}{\sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j\}}} \sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j\}} \mathbf{x}_i, \quad j = 1, \dots, K.$$

The *compactness* of each cluster is measured via the sum of squared distances of the cluster members from the cluster mean,

$$\sum_{\mathbf{x}_i \in \mathcal{C}_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 = \sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2, \quad j = 1, \dots, K.$$

This quantity can be thought of as a measure of *variation within the cluster*, which we would like to keep as small as possible for all clusters. Thus, the objective of the algorithm is to minimise the *sum of compactness measures*,

$$\mathcal{E}_K := \sum_{j=1}^K \sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2, \quad (3.1)$$

over clusters $\mathcal{C}_1, \dots, \mathcal{C}_K$. The quantity $\mathcal{E}_K$ is also known as the *distortion measure* of the clustering $\mathcal{C}_1, \dots, \mathcal{C}_K$ — this relates to *quantisation*, see Remark 3.2 below.

²⁰Stuart P. Lloyd (of Bell Labs) proposed the algorithm in 1957, but it was not published in a journal until 1982.

3.1.2 Algorithm

A basic version of the K -means clustering algorithm (Lloyd, 1982) to minimise $\mathcal{E}_K$ can be described as follows. The algorithm is iterative, and at each iteration $t = 0, 1, \dots$, updated clusters $\mathcal{C}_1^{(t)}, \dots, \mathcal{C}_K^{(t)}$ and their means $\boldsymbol{\mu}_1^{(t)}, \dots, \boldsymbol{\mu}_K^{(t)}$ are produced in two steps:

- (i) **Assignment:** Given the current cluster means $\boldsymbol{\mu}_1^{(t)}, \dots, \boldsymbol{\mu}_K^{(t)}$, new cluster memberships are determined such that each $\mathbf{x}_i$ is assigned to a cluster corresponding to its *closest cluster mean*, see Figure 3.1. Formally,

$$\mathbf{x}_i \in \mathcal{C}_j^{(t+1)} \quad \text{provided} \quad \underbrace{j = \arg \min_{j'=1, \dots, K} \|\mathbf{x}_i - \boldsymbol{\mu}_{j'}^{(t)}\|}_{\boldsymbol{\mu}_j^{(t)} \text{ is the mean closest to } \mathbf{x}_i}$$

for any $i = 1, \dots, n$.^{21,22} Thus, we obtain the new clusters $\mathcal{C}_1^{(t+1)}, \dots, \mathcal{C}_K^{(t+1)}$.

- (ii) **Update:** We compute the means $\boldsymbol{\mu}_1^{(t+1)}, \dots, \boldsymbol{\mu}_K^{(t+1)}$ under the new clusters:

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{\sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j^{(t+1)}\}}} \sum_{i=1}^n \mathbf{1}_{\{\mathbf{x}_i \in \mathcal{C}_j^{(t+1)}\}} \mathbf{x}_i, \quad j = 1, \dots, K.$$

Iteration is continued until convergence is attained. Modern implementations of K -means clustering use *more refined versions* of the algorithm (e.g., `kmeans()` in R uses an algorithm by Hartigan and Wong (1979)). There are various *initialisation strategies*, including:

- ★ Draw randomly (without replacement) K samples from $\mathbf{x}_1, \dots, \mathbf{x}_n$ to define them as means $\boldsymbol{\mu}_1^{(0)}, \dots, \boldsymbol{\mu}_K^{(0)}$,
- ★ Assign samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ randomly to clusters $\mathcal{C}_1^{(0)}, \dots, \mathcal{C}_K^{(0)}$,
- ★ Choose K samples non-randomly from $\mathbf{x}_1, \dots, \mathbf{x}_n$ according to their distance to the overall sample mean $\bar{\mathbf{x}}$ (Hartigan and Wong, 1979).

Remark 3.2. K -means clustering is a special case of (and originally developed for) *vector quantisation* (Hastie et al., 2009, Sect. 14.3.9), which is a problem in *signal processing*. In vector quantisation, given a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, we seek a set

²¹There are various methods (e.g., randomisation) to break *ties* (two or more means with equal distance to a sample), should they occur.

²²The assignment equivalent to the *Voronoi tessellation* associated with $\boldsymbol{\mu}_1^{(t)}, \dots, \boldsymbol{\mu}_K^{(t)}$. We will return to this in a computational example.

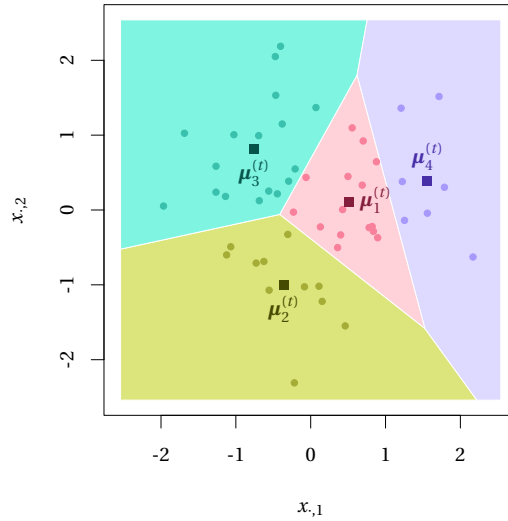


Figure 3.1: Illustration of K -means cluster assignment with $K = 4$.

of vectors $\{\mathbf{y}_1, \dots, \mathbf{y}_K\} \subset \mathbb{R}^d$, which is called a *codebook*, such that each sample $\mathbf{x}_i$ can be approximated by some $\mathbf{y}_j$ with minimal loss of information or distortion. Vector quantisation is used in *lossy data compression*, in particular. If K is much smaller than n , then the quantised version of the data set requires *much less storage space* — we will only have to store the codebook and an n -dimensional integer vector containing the cluster assignments.

It is not difficult to see that, on the surface, the K -means algorithm, with its *two steps*, bears some resemblance to the *EM algorithm* for the *Gaussian mixture model* (Example 2.17). But the resemblance is not merely superficial since there is in fact a formal *connection between the two*. If we estimate a mixture of the multivariate normal pdfs $\phi(\cdot; \boldsymbol{\mu}_1, \sigma I_d), \dots, \phi(\cdot; \boldsymbol{\mu}_K, \sigma I_d)$, where $\sigma > 0$ is fixed and I_d is the $d \times d$ identity matrix, using the EM algorithm, then *in the limit* $\sigma \rightarrow 0$ we *recover* K -means clustering. The Gaussian mixture model, estimated using the EM algorithm, is sometimes seen as a form of (and used for) clustering with *soft* assignments ($\mathbf{x}_i$ belongs to cluster j with probability in $(0, 1)$), in contrast to K -means algorithm's *hard* assignments ($\mathbf{x}_i$ either belongs to cluster j or doesn't).

3.2 K -medoids clustering

3.2.1 Medoid

Since K -means clustering is based on *Euclidean distances* and *means*, it has some inherent limitations:

- ★ It seeks to detect clusters that have a *spherical* shape, but it may fail to correctly detect clusters with *other shapes* (e.g., elliptical). Note that *rescaling* of the data at a preprocessing stage may create or break spherical clusters.
- ★ Means are not robust to *outliers* and *anomalies*, which may have an out-sized contribution to a sample mean.
- ★ The Euclidean distance is *not* always the appropriate distance measure.

K-medoids clustering (Kaufman and Rousseeuw, 1990, Chap. 2) modifies *K*-means by replacing means with *medoids*, which are more robust²³ measures of the centre of a multivariate data set, while also permitting the use of more general distance measures in place of the Euclidean distance. The *medoid* of a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ under dissimilarities $d_{i,j}$, $i, j = 1, \dots, n$, is defined as

$$\text{Medoid}(\mathbf{x}_1, \dots, \mathbf{x}_n) := \mathbf{x}_{i^*}, \quad \text{where } i^* := \arg \min_{i=1, \dots, n} \sum_{j \neq i} d_{i,j}.$$

That is, the medoid is the sample that *minimises the sum of distances* to other samples. Figure 3.2 compares the mean and medoid, illustrating how they are very close in an approximately symmetric data set but differ significantly in a skewed data set.

Remark 3.3. In the case where $d = 1$ and n is an odd number, $\text{Medoid}(x_1, \dots, x_n)$ coincides with the *sample median* of the data set $x_1, \dots, x_n \in \mathbb{R}$ if we use

$$d_{i,j} := |x_i - x_j|, \quad i, j = 1, \dots, n.$$

(When n is even, the sample median may not be any of the samples $x_1, \dots, x_n$, in which case it would not be equal to the medoid.)

3.2.2 Formulation and algorithm

Given dissimilarities $d_{i,j}$, $i, j = 1, \dots, n$, corresponding to samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, *K*-medoids clustering can be found by iteration of the following two steps:

- (i) *Update:* Given current clusters $\mathcal{C}_1^{(t)}, \dots, \mathcal{C}_K^{(t)} \subset \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, determine their medoids:

$$\mathbf{m}_j^{(t)} := \text{Medoid}(\mathcal{C}_j^{(t)}) \in \mathbb{R}^d, \quad j = 1, \dots, K.$$

²³in the sense of being less influenced by outliers and anomalies

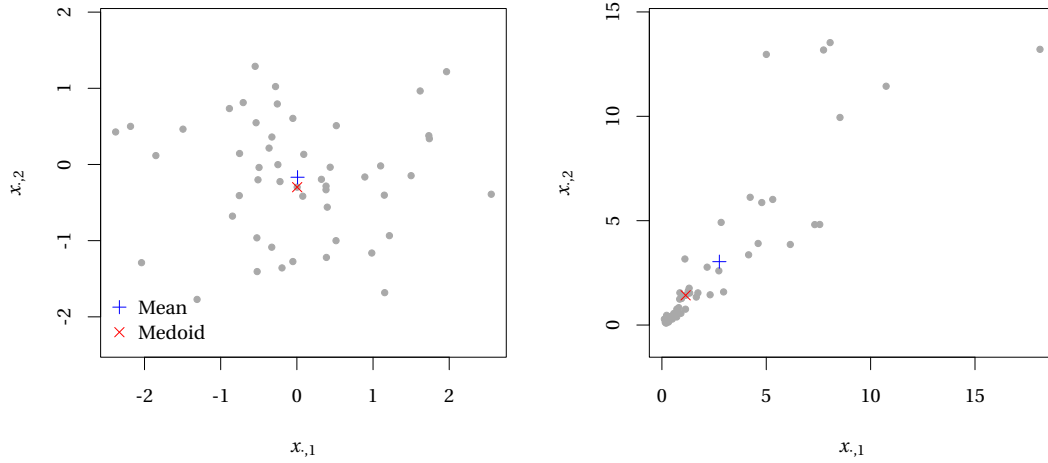


Figure 3.2: Comparison of mean and medoid (using $d_{i,j} := \|x_i - x_j\|$) with two two-dimensional data sets.

- (ii) **Assignment:** Using the medoids $\mathbf{m}_1^{(t)}, \dots, \mathbf{m}_K^{(t)}$, form new clusters by associating each sample with the closest medoid. Suppose that $\mathbf{m}_j^{(t)} = \mathbf{x}_{i_j^*}$ for some $i_j^* = 1, \dots, n$ for any $j = 1, \dots, K$. Then,

$$\mathbf{x}_i \in \mathcal{C}_j^{(t+1)} \quad \text{provided} \quad \underbrace{j = \arg \min_{j'=1, \dots, K} d_{i, i_{j'}^*}}_{\mathbf{m}_j^{(t)} \text{ is the medoid closest to } \mathbf{x}_i} \quad (3.4)$$

for any $i = 1, \dots, n$.

Again, iteration is continued until convergence is attained.

Remark 3.5. While the samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ appear in the algorithm, they serve just as *identifiers*. The only quantitative information we need about the samples are their dissimilarities $d_{i,j}$, $i, j = 1, \dots, n$. This is unlike K -means clustering, where we need the actual samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ to be able to compute the cluster means.

The *overall objective* of K -medoids clustering is to minimise

$$\sum_{j=1}^K \underbrace{\sum_{i: \mathbf{x}_i \in \mathcal{C}_j} d_{i, i_j^*}}_{\text{sum of dissimilarities with medoid in } \mathcal{C}_j}, \quad (3.6)$$

with respect to distinct $i_1^*, \dots, i_K^* \in \{1, \dots, n\}$ and disjoint $\mathcal{C}_1, \dots, \mathcal{C}_K \subset \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ such that $\bigcup_{j=1}^K \mathcal{C}_j = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and $\mathbf{x}_{i_j^*} \in \mathcal{C}_j$ for any $j = 1, \dots, K$, analogous to (3.1). The iteration is a *heuristic search strategy* towards minimising (3.6). The original *partitioning around medoids* (PAM) algorithm proposed by Kaufman and

Rousseeuw (1990, Chap. 2) also minimises (3.6) but differs from the above algorithm somewhat by using swap-based reassignment instead of the *Voronoi*-style reassignment (3.4).

3.3 Hierarchical clustering

3.3.1 Basics

Hierarchical clustering is a clustering technique that uses a gradual process to create a hierarchy of clusters. Unlike K -means or K -medoids clustering, hierarchical clustering does *not* require that the number of clusters be *predetermined*. Instead, the number of clusters can be decided after the full hierarchy of clusters has been generated.

Hierarchical clustering is either *agglomerative* (*bottom-up*) or *divisive* (*top-down*). In agglomerative clustering, each sample is initially in their *own* (singleton) *cluster*. Clusters are then recursively *joined* to form larger clusters until, in the end, all samples belong to a *single cluster*. Divisive clustering adopts the *opposite* approach, whereby all samples are initially in a single cluster, which is recursively *divided* into smaller clusters until, eventually, each sample resides in their own cluster. The agglomerative approach tends to be more commonly used, and we will concentrate on it here.

The results of hierarchical clustering, i.e., the hierarchy of clusters, are conventionally presented in a tree-like diagram called a *dendrogram*,²⁴ see Figure 3.3 for an example. Cutting the tree depicted in the dendrogram at some height produces a particular clustering of the data set.

Hierarchical clustering is based on the (pairwise) *dissimilarities* $d_{i,j}$, $i, j = 1, \dots, n$, between the samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ to be clustered — the actual samples are not required. In addition to distances between individual samples, we also need to be able to measure the dissimilarity $d_{A,B}$ between any pair of *sets of samples* (i.e., clusters) $A, B \subset \{1, \dots, n\}$ using just the dissimilarities between the constituents of A and B . There are *many possible ways* to define $d_{A,B}$ — common choices include:

★ *Maximum* or *complete linkage*:

$$d_{A,B} := \max_{i \in A, j \in B} d_{i,j} \quad (3.7)$$

²⁴The word “dendrogram” is derived from the ancient greek word “déndron” for “tree”.

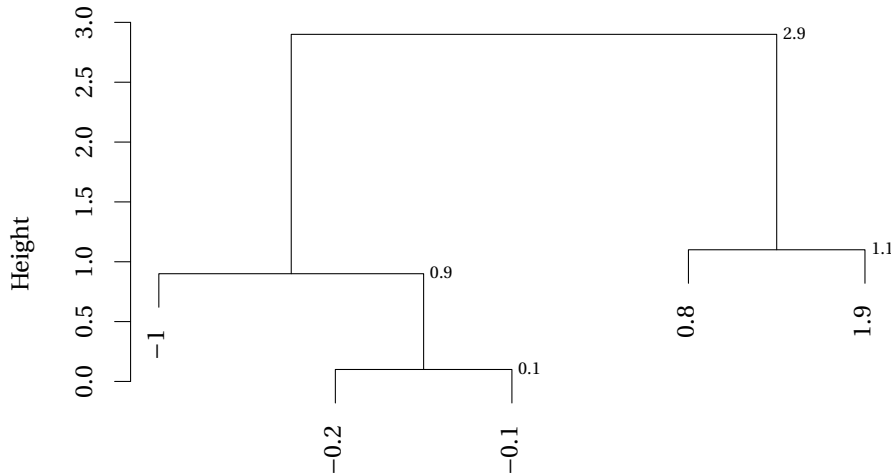


Figure 3.3: Dendrogram representing agglomerative complete-linkage clustering of samples -1.0 , -0.2 , -0.1 , 0.8 , and 1.9 . (The small numbers denote the heights at which the clusters are joined, see Section 3.3.2.)

★ *Minimum or single linkage:*

$$d_{A,B} := \min_{i \in A, j \in B} d_{i,j} \quad (3.8)$$

★ *Average linkage or unweighted pair group method with arithmetic mean (UPGMA) (Sokal and Michener, 1958):*

$$d_{A,B} := \frac{1}{\#A \cdot \#B} \sum_{i \in A, j \in B} d_{i,j}$$

The order in which clusters are joined in agglomerative clustering is determined using $d_{A,B}$, $A, B \subset \{1, \dots, n\}$. In particular, the pair (A, B) of clusters with the *lowest dissimilarity* $d_{A,B}$ will be joined next. Note that different specifications of $d_{A,B}$ may lead to different clusters and hierarchies.

Remark 3.9. Unlike K -means and K -medoids clustering, hierarchical clustering is purely *heuristic* — it does *not* try to minimise any overall objective function.

3.3.2 Case study: complete-linkage clustering

To gain more concrete understanding of how hierarchical clustering works, we work out agglomerative clustering of samples -1.0 , -0.2 , -0.1 , 0.8 , and 1.9 in $\mathbb{R}$ using the standard distance $d_{i,j} = |x_i - x_j|$ for pairwise dissimilarities and the *complete-linkage* measure (3.7) for dissimilarities between clusters. This also explains how the dendrogram in Figure 3.3 is drawn.

As mentioned above, in agglomerative clustering, initially each sample resides in their own cluster. New levels in the hierarchy of clusters are obtained by recursively joining clusters based on their dissimilarities. The dissimilarity matrix for the initial clusters is:

$\{-0.2\}$	0.8			
$\{-0.1\}$	0.9	0.1		
$\{0.8\}$	1.8	1.0	0.9	
$\{1.9\}$	2.9	2.1	2.0	1.1
	$\{-1.0\}$	$\{-0.2\}$	$\{-0.1\}$	$\{0.8\}$

We note that the singletons $\{-0.2\}$ and $\{-0.1\}$ have the lowest dissimilarity. Hence they will be joined first to form cluster $\{-0.2, -0.1\}$ at a *height* which equals their dissimilarity, i.e.,

$$\text{height} = d_{\{-0.2\}, \{-0.1\}} = 0.1.$$

We can then calculate an updated dissimilarity matrix using (3.7):

$\{-0.2, -0.1\}$	0.9			
$\{0.8\}$	1.8	1.0		
$\{1.9\}$	2.9	2.1	1.1	
	$\{-1.0\}$	$\{-0.2, -0.1\}$	$\{0.8\}$	

Now, clusters $\{-1.0\}$ and $\{-0.2, -0.1\}$ have the lowest dissimilarity, so they will be joined next at

$$\text{height} = d_{\{-1.0\}, \{-0.2, -0.1\}} = 0.9.$$

Subsequently, the new dissimilarity matrix is:

$\{0.8\}$	1.8	
$\{1.9\}$	2.9	1.1
	$\{-1.0, -0.2, -0.1\}$	$\{0.8\}$

There, the clusters $\{0.8\}$ and $\{1.9\}$ have the lowest dissimilarity, so it is their turn to be joined at

$$\text{height} = d_{\{0.8\}, \{1.9\}} = 1.1.$$

Finally, the remaining two clusters $\{-1.0, -0.2, -0.1\}$ and $\{0.8, 1.9\}$ are joined at

$$\begin{aligned} \text{height} &= d_{\{-1.0, -0.2, -0.1\}, \{0.8, 1.9\}} = \max\{d_{\{-1.0, -0.2, -0.1\}, \{0.8\}}, d_{\{-1.0, -0.2, -0.1\}, \{1.9\}}\} \\ &= \max\{1.8, 2.9\} = 2.9. \end{aligned}$$

Hence, we have obtained a hierarchy of clusters and heights consistent with the dendrogram in Figure 3.3.

3.3.3 Properties and limitations

Different forms of hierarchical clustering have their strength and weaknesses, as discussed by Hastie et al. (2009, pp. 523–525). Single-linkage clustering, using the minimum dissimilarity (3.8) between the constituents of the respective clusters, sets a very low bar for joining clusters. Hence, the resulting clusters may have *poor compactness*, i.e., variation within a cluster may be large. By contrast, complete-linkage clustering typically produces clusters with small variation within cluster, but the clusters may be *close to each other* — more so than what is preferable. Average-linkage clustering represents a *compromise* between the two extremes, seeking clusters with moderate compactness and moderately large distances between clusters.

Single-linkage and complete-linkage clustering can, however, be seen as more *robust* than average-linkage clustering. Applying any *strictly increasing transformation* $f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ to the dissimilarities, i.e., using $\tilde{d}_{i,j} := f(d_{i,j})$, $i, j = 1, \dots, n$, will not change the results of single-linkage and complete-linkage clustering, but it may change those of average-linkage clustering drastically.

Finally, it is worth pointing out that dendrograms representing the results of hierarchical clustering should be *interpreted with care*, as noted by Hastie et al. (2009, pp. 521–522). A dendrogram should primarily be seen as a graphical representation of the results of clustering rather than a graphical summary of the intrinsic structure of the data set. In particular, a dendrogram should not be interpreted as a “proof” or “evidence” of cluster structure in data — a dendrogram can be produced even when no such structure exists! Moreover, changes to the choices of dissimilarity measures for individual samples or clusters may give rise to *significant variation* in the results of hierarchical clustering, in particular when the samples do not exhibit a clear cluster structure.

3.4 Evaluation measures

3.4.1 Rationale and types

It is useful to be able to *assess quantitatively*, i.e., *evaluate*, the results of clustering. This is essential in the choice of the *number of clusters*, but it also helps us draw *comparisons* between different clustering algorithms.

Evaluation measures for clustering can be divided into *extrinsic* and *intrinsic*. An extrinsic measure is used for a *comparison* between *two clusterings* of the same data set, one of which could, in particular, represent the *ground truth* (if

the actual labels are available). Intrinsic measures, by contrast, are computed for a single clustering *without* relying on any *additional information*.

3.4.2 Rand index

Suppose that $\mathcal{C}_1, \dots, \mathcal{C}_p$, for some $p \geq 2$, are a *clustering* (i.e., a partition) of a data set $\mathbf{x}_1, \dots, \mathbf{x}_n$ and $\mathcal{R}_1, \dots, \mathcal{R}_q$, for some $q \geq 2$ (not necessarily equal to p), are another clustering, *reference clustering*, of the same data set. For the sake of simpler terminology, we adopt here the interpretation of the reference clustering as the ground truth.

We will assign each *unordered* pair $\{\mathbf{x}, \mathbf{x}'\}$ of *distinct* samples from $\mathbf{x}_1, \dots, \mathbf{x}_n$ into one of the following four disjoint classes:

★ *True positives* (TP):

$$\text{TP} := \{(\mathbf{x}, \mathbf{x}') : \mathbf{x}, \mathbf{x}' \in \mathcal{C}_i \text{ for some } i, \text{ and } \mathbf{x}, \mathbf{x}' \in \mathcal{R}_k \text{ for some } k\}$$

(i.e., $\mathbf{x}$ and $\mathbf{x}'$ are *correctly* assigned to the *same cluster*)

★ *True negatives* (TN):

$$\begin{aligned} \text{TN} := \{(\mathbf{x}, \mathbf{x}') : \mathbf{x} \in \mathcal{C}_i, \mathbf{x}' \in \mathcal{C}_j \text{ for some } i \neq j, \\ \text{and } \mathbf{x} \in \mathcal{R}_k, \mathbf{x}' \in \mathcal{R}_l \text{ for some } k \neq l\} \end{aligned}$$

(i.e., $\mathbf{x}$ and $\mathbf{x}'$ are *correctly* assigned to *separate clusters*)

★ *False positives* (FP):

$$\text{FP} := \{(\mathbf{x}, \mathbf{x}') : \mathbf{x}, \mathbf{x}' \in \mathcal{C}_i \text{ for some } i, \text{ and } \mathbf{x} \in \mathcal{R}_k, \mathbf{x}' \in \mathcal{R}_l \text{ for some } k \neq l\}$$

(i.e., $\mathbf{x}$ and $\mathbf{x}'$ are *incorrectly* assigned to the *same cluster*)

★ *False negatives* (FN):

$$\text{FN} := \{(\mathbf{x}, \mathbf{x}') : \mathbf{x} \in \mathcal{C}_i, \mathbf{x}' \in \mathcal{C}_j \text{ for some } i \neq j, \text{ and } \mathbf{x}, \mathbf{x}' \in \mathcal{R}_k \text{ for some } k\}$$

(i.e., $\mathbf{x}$ and $\mathbf{x}'$ are *incorrectly* assigned to *separate clusters*)

The *Rand index* R (Rand, 1971) is then defined via

$$R := \frac{\#TP + \#TN}{\#TP + \#FP + \#FN + \#TN} = \frac{\#TP + \#TN}{\binom{n}{2}} \in [0, 1]. \quad (3.10)$$

(Above, $\binom{n}{m} := \frac{n!}{m!(n-m)!}$ is the binomial coefficient.) The index measures *similarity* between $\mathcal{C}_1, \dots, \mathcal{C}_p$ and $\mathcal{R}_1, \dots, \mathcal{R}_q$, whereby a higher value of R indicates

higher similarity. In the extreme case $R = 1$ the clusterings are *equivalent* (except that the ordering of the clusters may differ).

A well known *shortcoming* of the Rand index is that it may assume relatively *high values* by chance, when a trivial²⁵ *random* method for clustering is used. Consider, e.g., a scenario where the ground truth is that the samples belong to *two clusters* in *equal proportions*. Then purely random assignment into two clusters based on *flipping a coin* will attain $R \approx 0.5$ on average. (This phenomenon is analogous to the one that affects *accuracy* as a measure of performance in classification.)

For this reason, it is standard to adjust the Rand index for chance. To this end, we define a *contingency table* of cluster assignments

$$n_{i,j} := \#(\mathcal{C}_i \cap \mathcal{R}_j), \quad i = 1, \dots, p, \quad j = 1, \dots, q, \quad (3.11)$$

and the associated column and row sums (*marginals*)

$$n_{i,+} := \sum_{j=1}^q n_{i,j}, \quad i = 1, \dots, p, \quad n_{+,j} := \sum_{i=1}^p n_{i,j}, \quad j = 1, \dots, q. \quad (3.12)$$

Following Gates and Ahn (2017, p. 6), we have

$$\begin{aligned} \#TP &= \sum_{i=1}^p \sum_{j=1}^q \binom{n_{i,j}}{2}, & \#FP &= \sum_{i=1}^p \binom{n_{i,+}}{2} - \#TP, \\ \#FN &= \sum_{j=1}^q \binom{n_{+,j}}{2} - \#TP, & \#TN &= \binom{n}{2} - \#TP - \#FP - \#FN. \end{aligned} \quad (3.13)$$

The adjustment is typically done assuming a *permutation model* where the entries $n_{i,j}$, $i = 1, \dots, p$, $j = 1, \dots, q$, become random variables that follow a *generalised hypergeometric distribution* such that the marginals $n_{i,+}$, $i = 1, \dots, p$, and $n_{+,j}$, $j = 1, \dots, q$, remain fixed (non-random). It can be shown (Hubert and Arabie, 1985) that, under this model,

$$\mathbf{E}_{n_{i,j} \sim \text{Perm}} \left[\binom{n_{i,j}}{2} \right] = \frac{1}{\binom{n}{2}} \binom{n_{i,+}}{2} \binom{n_{+,j}}{2}. \quad (3.14)$$

The *adjusted Rand index* (Hubert and Arabie, 1985) is then defined as

$$R_{\text{adj}} := \frac{R - \mathbf{E}_{n_{i,j} \sim \text{Perm}}[R]}{1 - \mathbf{E}_{n_{i,j} \sim \text{Perm}}[R]}.$$

Using (3.10), (3.13), (3.14) and some algebra, the adjusted index can explicitly be expressed (Gates and Ahn, 2017, p. 7) as

$$R_{\text{adj}} = \frac{\binom{n}{2} \sum_{i=1}^p \sum_{j=1}^q \binom{n_{i,j}}{2} - \sum_{i=1}^p \binom{n_{i,+}}{2} \sum_{j=1}^q \binom{n_{+,j}}{2}}{\frac{1}{2} \binom{n}{2} \left(\sum_{i=1}^p \binom{n_{i,+}}{2} + \sum_{j=1}^q \binom{n_{+,j}}{2} \right) - \sum_{i=1}^p \binom{n_{i,+}}{2} \sum_{j=1}^q \binom{n_{+,j}}{2}}.$$

²⁵in the sense that it does not use any information about the samples

Remark 3.15. Unlike the original Rand index, the adjusted index can become *negative*. This signals that the evaluated clustering is worse than what is expected under the random permutation model.

Remark 3.16. The permutation model is idealised and its properties can be criticised (Gates and Ahn, 2017, pp. 2–3). For example, the assumption that the *cluster sizes are fixed* is not quite realistic for any of the clustering methods we have studied. Hence, adjustments to the Rand index have also been derived under alternative random models (Gates and Ahn, 2017, Sect. 4).

3.4.3 Mutual information

Like above, we endeavour to evaluate the similarity between clusterings $\mathcal{C}_1, \dots, \mathcal{C}_p$ and $\mathcal{R}_1, \dots, \mathcal{R}_q$. The contingency table defined in (3.11) gives rise to a discrete (joint) *probability distribution* on $\{1, \dots, p\} \times \{1, \dots, q\}$ defined via

$$P_{\mathcal{C}, \mathcal{R}}(i, j) := \frac{n_{i,j}}{n}, \quad (i, j) \in \{1, \dots, p\} \times \{1, \dots, q\},$$

with marginals

$$P_{\mathcal{C}}(i) := \frac{n_{i,+}}{n}, \quad i = 1, \dots, p, \quad P_{\mathcal{R}}(j) := \frac{n_{+,j}}{n}, \quad j = 1, \dots, q.$$

Recall that in Section 1.8 (on independent component analysis), we used *mutual information*, defined using *differential entropy*, as a measure of *independence*. We can use their discrete counterparts to measure similarity between the two clusterings. Define first the (discrete) entropy of the joint distribution $P_{\mathcal{C}, \mathcal{R}}$ via

$$H(\mathcal{C}, \mathcal{R}) := - \sum_{i=1}^p \sum_{j=1}^q P_{\mathcal{C}, \mathcal{R}}(i, j) \log P_{\mathcal{C}, \mathcal{R}}(i, j) = - \sum_{i=1}^p \sum_{j=1}^q \frac{n_{i,j}}{n} \log \frac{n_{i,j}}{n}$$

and the entropies of the marginals via

$$\begin{aligned} H(\mathcal{C}) &:= - \sum_{i=1}^p P_{\mathcal{C}}(i) \log P_{\mathcal{C}}(i) = - \sum_{i=1}^p \frac{n_{i,+}}{n} \log \frac{n_{i,+}}{n}, \\ H(\mathcal{R}) &:= - \sum_{j=1}^q P_{\mathcal{R}}(j) \log P_{\mathcal{R}}(j) = - \sum_{j=1}^q \frac{n_{+,j}}{n} \log \frac{n_{+,j}}{n}. \end{aligned}$$

The mutual information between $\mathcal{C}$ and $\mathcal{R}$ is then defined as

$$\begin{aligned} I(\mathcal{C}, \mathcal{R}) &:= H(\mathcal{C}) + H(\mathcal{R}) - H(\mathcal{C}, \mathcal{R}) \\ &= \sum_{i=1}^p \sum_{j=1}^q P_{\mathcal{C}, \mathcal{R}}(i, j) \log \frac{P_{\mathcal{C}, \mathcal{R}}(i, j)}{P_{\mathcal{C}}(i) P_{\mathcal{R}}(j)} \\ &= \sum_{i=1}^p \sum_{j=1}^q \frac{n_{i,j}}{n} \log \frac{n_{i,j} n}{n_{i,+} n_{+,j}}. \end{aligned}$$

In line with the continuous case, $I(\mathcal{C}, \mathcal{R}) \geq 0$, and $I(\mathcal{C}, \mathcal{R}) = 0$ is attained when the cluster assignments in $\mathcal{C}$ and $\mathcal{R}$, respectively, are *mutually independent* — this is when the clusterings have the *lowest similarity*. Accordingly, $I(\mathcal{C}, \mathcal{R})$ is an increasing measure of similarity between the clusterings — the higher the value is the more similar $\mathcal{C}$ and $\mathcal{R}$ are.

It is not difficult to show that

$$I(\mathcal{C}, \mathcal{R}) \leq \min\{H(\mathcal{C}), H(\mathcal{R})\},$$

which motivates *normalisation* of the measure for the sake of better *comparability* and *interpretability*. Unfortunately, no consensus has emerged over which quantity $I(\mathcal{C}, \mathcal{R})$ should be divided by (Vinh et al., 2010). Each of the following six (listed in non-decreasing order),

$$\begin{array}{lll} \min\{H(\mathcal{C}), H(\mathcal{R})\}, & \sqrt{H(\mathcal{C})H(\mathcal{R})}, & \frac{H(\mathcal{C}) + H(\mathcal{R})}{2}, \\ \max\{H(\mathcal{C}), H(\mathcal{R})\}, & \max\{\log p, \log q\}, & \log n, \end{array}$$

is used, and in each case the resulting measure is called *normalised mutual information* (Gates and Ahn, 2017, p. 10).

Remark 3.17. Like the Rand index, mutual information is *not immune* to the problem that it may attain relatively high values even under trivial, randomly assigned clustering. Thus, *adjustments* have been introduced for mutual information and its normalised variants under the permutation model (and other models), although the expressions are more complicated than those for the Rand index (Gates and Ahn, 2017, Sect. 5).

3.4.4 Gap statistic

The *gap statistic* (Tibshirani et al., 2001) is an intrinsic measure that can be used to estimate the *number of clusters* under a clustering algorithm whenever the number is a hyperparameter that the user must set. Given a clustering $\mathcal{C} := \{\mathcal{C}_1, \dots, \mathcal{C}_p\}$ of a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, we define the *within-dispersion measure*

$$W_p := W_p(\mathbf{x}_1, \dots, \mathbf{x}_n; \mathcal{C}) := \sum_{k=1}^p \frac{1}{2 \cdot \#\mathcal{C}_k} \sum_{i,j \in \mathcal{C}_k} d_{i,j}, \quad (3.18)$$

where the dissimilarity $d_{i,j}$ between $\mathbf{x}_i$ and $\mathbf{x}_j$ is typically measured using their squared Euclidean distance,

$$d_{i,j} := \|\mathbf{x}_i - \mathbf{x}_j\|^2.$$

The gap statistic is then conceptually defined as the difference

$$\text{Gap}(p) := \mathbf{E}_{(\mathbf{X}_1, \dots, \mathbf{X}_n) \sim \text{Null}} [\log W_p(\mathbf{X}_1, \dots, \mathbf{X}_n; \tilde{\mathcal{C}})] - \log W_p(\mathbf{x}_1, \dots, \mathbf{x}_n; \mathcal{C}),$$

where the term $\mathbf{E}_{(\mathbf{X}_1, \dots, \mathbf{X}_n) \sim \text{Null}} [\log W_p(\mathbf{X}_1, \dots, \mathbf{X}_n)]$ is the expectation of $\log W_p$ applied to iid random vectors $\mathbf{X}_1, \dots, \mathbf{X}_n$ that follow a reference distribution, the *null model, without cluster structure* and to their clustering $\tilde{\mathcal{C}}$ produced using the *same algorithm* that was used to cluster the original data. Tibshirani et al. (2001, Sect. 3) recommend that a d -dimensional *uniform distribution* be chosen as the null model. The range of the uniform distribution can be chosen to be a *rectangle* determined by the *component-wise maxima and minima* of $\mathbf{x}_1, \dots, \mathbf{x}_n$ (or their principal components, if a rotationally invariant choice is preferred). Our objective is then to find a *small* value of p such that $\text{Gap}(p)$ is *large* (variation within clusters is low for a clustering where the number of clusters is small).

In practice, the gap statistic would be applied to a data set $\mathbf{x}_1, \dots, \mathbf{x}_n$ and some clustering algorithm as follows:

- (i) Select a *range* $p = 1, \dots, p_{\max}$ for the number of clusters.
- (ii) For any $p = 1, \dots, p_{\max}$, generate a clustering $\mathcal{C}^{(p)} := \{\mathcal{C}_1^{(p)}, \dots, \mathcal{C}_p^{(p)}\}$ of the data $\mathbf{x}_1, \dots, \mathbf{x}_n$ using the algorithm. Then compute the within-dispersion measure $W_p(\mathbf{x}_1, \dots, \mathbf{x}_n; \mathcal{C}^{(p)})$.
- (iii) Select $B \in \mathbb{N}$, the number of *Monte Carlo* replications. For any $b = 1, \dots, B$, *sample* $\mathbf{x}_1^{(b)}, \dots, \mathbf{x}_n^{(b)}$ *independently* from the null model. Then, for any $p = 1, \dots, p_{\max}$, generate clustering $\tilde{\mathcal{C}}^{(b,p)} := \{\tilde{\mathcal{C}}_1^{(b,p)}, \dots, \tilde{\mathcal{C}}_p^{(b,p)}\}$ of $\mathbf{x}_1^{(b)}, \dots, \mathbf{x}_n^{(b)}$ using the algorithm. Finally, compute $W_p(\mathbf{x}_1^{(b)}, \dots, \mathbf{x}_n^{(b)}; \tilde{\mathcal{C}}^{(b,p)})$.
- (iv) For any $p = 1, \dots, p_{\max}$, compute the *estimated* gap statistic

$$\widehat{\text{Gap}}(p) := \frac{1}{B} \sum_{b=1}^B \log W_p(\mathbf{x}_1^{(b)}, \dots, \mathbf{x}_n^{(b)}; \tilde{\mathcal{C}}^{(b,p)}) - \log W_p(\mathbf{x}_1, \dots, \mathbf{x}_n; \mathcal{C}^{(p)}).$$

- (v) For any $p = 1, \dots, p_{\max}$, let sd_p denote the sample standard deviation of

$$\log W_p(\mathbf{x}_1^{(b)}, \dots, \mathbf{x}_n^{(b)}; \tilde{\mathcal{C}}^{(b,p)}), \quad b = 1, \dots, B.$$

Then compute $s_p := \text{sd}_p \sqrt{1 + \frac{1}{B}}$, the *standard error* of $\widehat{\text{Gap}}(p)$.

- (vi) *Estimate* the number of clusters $\hat{p}$ via

$$\hat{p} := \min \left\{ p \geq 1 : \widehat{\text{Gap}}(p) \geq \widehat{\text{Gap}}(p+1) - s_{p+1} \right\}. \quad (3.19)$$

Remark 3.20. The estimate (3.19) is an application of the *one-standard-error rule* commonly used, e.g., in *cross validation*.

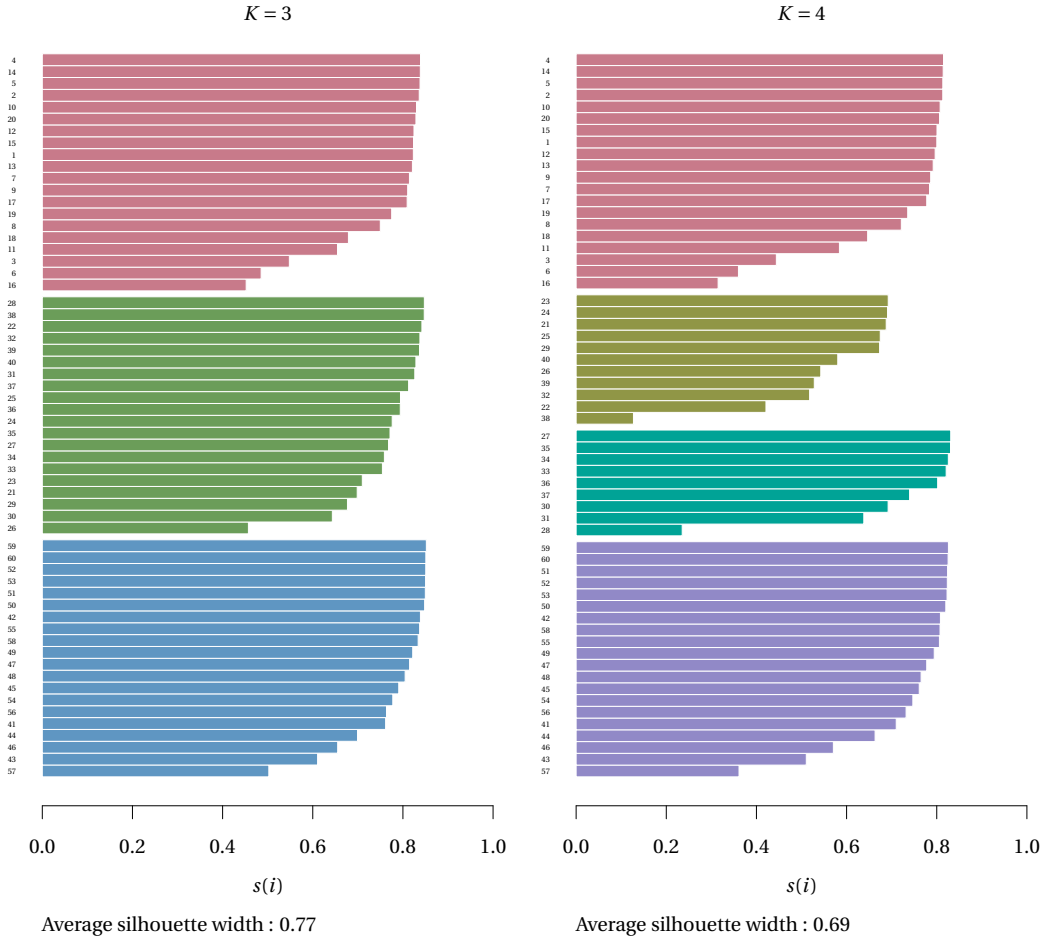


Figure 3.4: Silhouette widths for the results of K -medoids clustering, with $K = 3$ and $K = 4$, respectively, applied to a simulated data set of 60 samples with 3 “true” clusters.

3.4.5 Silhouette

Silhouette (Rousseeuw, 1987) is another intrinsic measure to evaluate a clustering. Its name stems from the standard way of presenting the values of the measure using a *bar plot*.

To construct the silhouette, let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be a data set with dissimilarities $d_{i,j}$, $i, j = 1, \dots, n$, and let $\mathcal{C}_1, \dots, \mathcal{C}_p$ be a clustering of the data set. (The method does not require the actual samples; their dissimilarities suffice.) For any $i = 1, \dots, n$, let $j(i) = 1, \dots, p$ be the unique index such that $\mathbf{x}_i \in \mathcal{C}_{j(i)}$ (i.e., sample i is assigned to the j -th cluster). Then we define

$$a(i) := \frac{1}{\#\mathcal{C}_{j(i)} - 1} \sum_{i' \in \mathcal{C}_{j(i)} \setminus \{i\}} d_{i,i'}, \quad i = 1, \dots, n,$$

which is the *average dissimilarity* between sample i and *other samples in its cluster*. Moreover, we define

$$b(i) := \min_{j' \neq j(i)} \frac{1}{\#\mathcal{C}_{j'}} \sum_{i' \in \mathcal{C}_{j'}} d_{i,i'}, \quad i = 1, \dots, n,$$

which is the *average dissimilarity* between sample i and the *samples in the cluster that is closest to it* (excluding its own cluster $\mathcal{C}_{j(i)}$). Using these quantities, the *silhouette width* for any sample $i = 1, \dots, n$ is defined as

$$s(i) := \begin{cases} \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, & \#\mathcal{C}_{j(i)} > 1, \\ 0, & \#\mathcal{C}_{j(i)} = 1. \end{cases}$$

By construction, $s(i) \in [-1, 1]$, and a value close to 1 indicates that the sample is (on average) *close to its fellow cluster members* and *far away from other clusters*, which is the preferred scenario. Values *near zero* indicate samples that lie *between two clusters*, and values close to -1 are for samples that are *closer to a neighbouring cluster* than *samples in its own cluster*, which hints at misspecification of the cluster.

The values $s(i)$, $i = 1, \dots, n$, are typically grouped according to the clusters $\mathcal{C}_1, \dots, \mathcal{C}_p$ and presented *in decreasing order* using a bar plot; see Figure 3.4. The values can be summarised through the *average silhouette width*

$$\bar{s} := \frac{1}{n} \sum_{i=1}^n s(i)$$

(which can also be computed separately for each cluster). We can then select the number of clusters, p , such that it gives the *highest* average silhouette width $\bar{s}$ within some range $p = 1, \dots, p_{\max}$ (Rousseeuw, 1987, p. 59).

4 Anomaly and outlier detection

4.1 Anomalies and outliers

Informally, an *outlier* is a sample or data point that is *separated* from the bulk of other samples (under some dissimilarity measure). The term *anomaly* is often used interchangeably with outlier, but there have been attempts to distinguish the two. For example, outliers could be thought as being *naturally occurring* while anomalies would represent *abnormal behaviour* — impossible or unlikely within the extent of normal behaviour. Note, however, that “normal” is a rather flexible concept. Hence, it is not clear if there is actually a distinction between the two terms that is unambiguous, practically applicable and has validity across different contexts. The closely related term *novelty* is also often used.

At any rate, we can identify two somewhat more well-defined problems in this domain:

- (i) *Identify outliers* in a data set that may contain both normal and abnormal samples.
- (ii) Given an existing, reference data set and a new (“unseen”) sample (or a set of new samples), *determine* whether the new sample(s) manifest *abnormal behaviour* vis-à-vis the existing data set.

For simplicity, we shall refer to (i) and (ii) as *outlier detection* and *anomaly detection*, respectively, while remaining mindful of the caveats about these terms, as discussed above. The latter is also often called *novelty detection*.²⁶

Before studying methods to detect outliers and anomalies, let us briefly review potential *reasons* behind their occurrence:

- ★ *Data errors*: faulty measurement devices, data corruption and mistakes in the preprocessing of data can all lead to outliers or anomalies in a data set. Hence, it is worthwhile to try to investigate the reasons behind outliers and anomalies, especially egregious ones, since it may be possible to *remove* or *mitigate* these sources of error.
- ★ *Natural occurrence*: extreme observations (that are far from the bulk) are inherent to, e.g., *heavy-tailed* distributions (such as Student t , Pareto and

²⁶the term is used, e.g., in scikit-learn; see: https://scikit-learn.org/stable/modules/outlier_detection.html

stable distributions). Such distributions are relevant to various real-world phenomena: wealth and income, popularity of content on social media, citations to scientific articles etc.

★ *Sampling errors*: samples that are included in a data set but should not be may appear as outliers or anomalies — e.g., a data set of images of the digit 3 that inadvertently includes a few images of 5 (which could arise from flaws in labelling). Like in data errors, the underlying causes may be rectifiable, hence they are worth investigating.

Remark 4.1. Outliers can be a nuisance for machine learning and statistical methods, resulting in poor fit or instability of parameters. Hence, it may sometimes be tempting to *remove* them or apply a method that is *robust* to outliers²⁷ (in the sense that it gives them little or no weight). However, this should only be done after reflecting on the *practical consequences* of outliers. In some contexts, outliers are the *most consequential* samples in the data set (e.g., extreme wind speeds and temperatures, large financial losses, spikes in electricity consumption or abnormal input values that cause a device to malfunction).

In what follows, we will study two, rather different, unsupervised methods which can both be applied to outlier detection (for *existing samples*) and anomaly detection (for *new samples*) alike.

4.2 Local outlier factor

Local outlier factor (LOF) (Breunig et al., 2000) is a method to detect outliers based on the *density* of samples.²⁸

LOF requires the dissimilarities $d_{i,j}$, $i, j = 1, \dots, n$, of the samples in a data set $\mathbf{x}_1, \dots, \mathbf{x}_n$, but not the actual samples. For any $i = 1, \dots, n$, let $\mathcal{N}_i^{(K)}$ denote the set of the *K-nearest neighbours* of sample $\mathbf{x}_i$. Moreover, write $d_i^{(K)}$ for the dissimilarity between $\mathbf{x}_i$ and its K -th nearest neighbour (or neighbours if there are ties). We then define the *reachability distance* of sample $\mathbf{x}_i$ from sample $\mathbf{x}_j$ via

$$\text{rd}_{j \rightarrow i}^{(K)} := \max\{d_j^{(K)}, d_{i,j}\}, \quad i \neq j.$$

²⁷e.g., regression trained with *Huber loss* (Hastie et al., 2009, p. 349) instead of ordinary least squares

²⁸Here, “density” refers literally to how densely the samples are packed in any region, instead of a probability density, or an estimate thereof. Of course, the two concepts are somewhat related.

This reachability distance assigns equal dissimilarity to all of the K -nearest neighbours of $\mathbf{x}_j$, but beyond those neighbours it coincides with the usual dissimilarity. The purpose of this choice is smoothen the fluctuations of the dissimilarities between $\mathbf{x}_j$ and its neighbours (Breunig et al., 2000, p. 95). (A simplified version of LOF can be constructed just by using $d_{i,j}$ in place of $\text{rd}_{j \rightarrow i}^{(K)}$.)

Remark 4.2. Note that, mathematically, $\text{rd}_{j \rightarrow i}^{(K)}$ is *not* a *metric* since $\text{rd}_{j \rightarrow i}^{(K)}$ and $\text{rd}_{i \rightarrow j}^{(K)}$ need not be equal (i.e., no symmetry).

Subsequently, we define the *local reachability density* of $\mathbf{x}_i$ as

$$\text{lrd}_i^{(K)} := \left(\frac{1}{\#\mathcal{N}_i^{(K)}} \sum_{j: \mathbf{x}_j \in \mathcal{N}_i^{(K)}} \text{rd}_{j \rightarrow i}^{(K)} \right)^{-1}, \quad i = 1, \dots, n.$$

It is the inverse of the average reachability of $\mathbf{x}_i$ from its K -nearest neighbours. Intuitively, the closer the K -nearest neighbours of $\mathbf{x}_i$ are to it, the higher $\text{lrd}_i^{(K)}$ is. (In an extreme case where the K -nearest neighbours of $\mathbf{x}_i$ are equal to it, $\text{lrd}_i^{(K)} = \infty$.) The LOF of each sample can then be defined via

$$\text{LOF}_i^{(K)} := \frac{1}{\#\mathcal{N}_i^{(K)}} \sum_{j: \mathbf{x}_j \in \mathcal{N}_i^{(K)}} \frac{\text{lrd}_j^{(K)}}{\text{lrd}_i^{(K)}}, \quad i = 1, \dots, n.$$

LOF thus compares the local densities of the K -nearest neighbours of $\mathbf{x}_i$ to its own local density. If the local density of $\mathbf{x}_i$ is similar to its neighbours' local densities ($\text{lrd}_i^{(K)} \approx \text{lrd}_j^{(K)}$ for any neighbour with index j), then $\text{LOF}_i^{(K)} \approx 1$ and $\mathbf{x}_i$ is *not an outlier*. (This is, in particular, the case for samples *deep inside a cluster*; see Breunig et al. (2000, Lemma 1).) However, if the local density of $\mathbf{x}_i$ is much lower than those of its neighbours, then $\mathbf{x}_i$ is *an outlier* and $\text{LOF}_i^{(K)} \gg 1$. (If $\text{LOF}_i^{(K)} \ll 1$, then $\mathbf{x}_i$ is said to be an *inlier*.)

The upshot is thus that samples with LOF *significantly greater than one* can be deemed as outliers (or anomalies if they are new samples introduced to a data set). Unfortunately, there is *no universal threshold* for how much higher than one LOF should be for a sample to be considered an outlier, since this can be rather *data-specific*. However, in subsequent refinements of LOF, *normalisation* has been proposed for better interpretability; see Kriegel et al. (2011).

Remark 4.3. In LOF, K is a *hyperparameter* that the user must choose. Breunig et al. (2000, Sect. 6) demonstrate that $\text{LOF}_i^{(K)}$ is *not monotonic* in K , which complicates the choice. Breunig et al. (2000, Sect. 6.2) propose some heuristics, most prominently to determine a relevant range for K , within which the *maximum* of $\text{LOF}_i^{(K)}$ can be adopted.

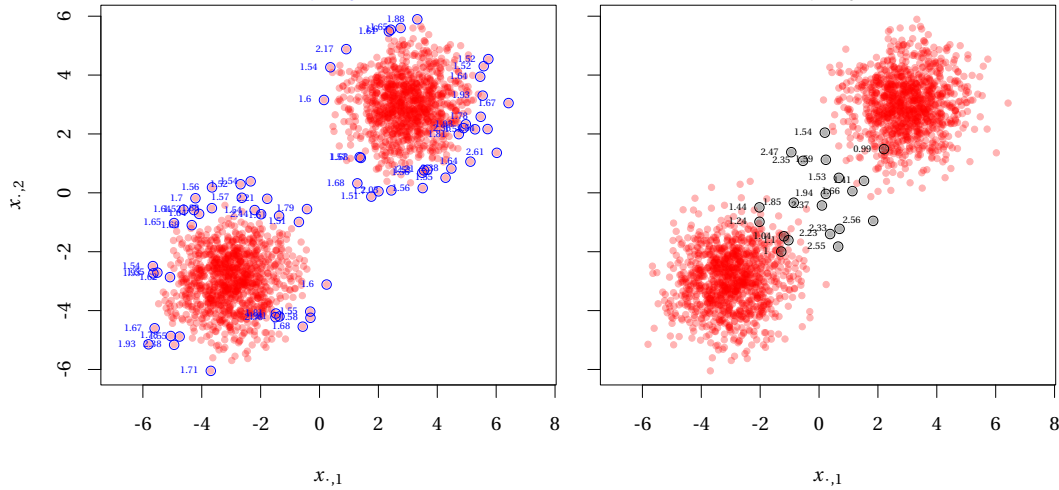


Figure 4.1: *Left*: LOF applied to outlier detection in a simulated bivariate data set consisting of 1 000 samples from $N([-3 \ -3]^T, I_2)$ and another 1 000 samples from $N([3 \ 3]^T, I_2)$ with $K = 10$. Samples with $\text{LOF}_i^{(K)} > 1.5$ are highlighted in blue. *Right*: LOF applied to anomaly detection, where 20 new samples from $N(\mathbf{0}, I_2)$ were added to the data set. For each sample, $\text{LOF}_i^{(K)}$ with $K = 10$ was computed after adding the sample to the existing data set of 2 000 samples. The new samples are highlighted in grey/black.

An application of LOF to a simulated data set for both outlier and anomaly detection is demonstrated in Figure 4.1. Therein, LOF is able to identify most of the extreme values (outliers) in the two clusters and most of the anomalous new samples added to the data set.

4.3 One-class support vector machine

We recall that the *support vector machine* (SVM) (Boser et al., 1992) is standard technique used primarily in *supervised learning* to build classifiers and beyond (Hastie et al., 2009, Chap. 12).

The basic setup for a binary SVM classifier (Hastie et al., 2009, Sect. 12.2) is that we have a data set of features $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and matching binary labels $y_1, \dots, y_n \in \{-1, 1\}$. In an ideal scenario, the two subsets of samples, *classes*, with $y_i = -1$ and $y_i = 1$, respectively, are such that we can find a *separating hyperplane*, which is $d - 1$ -dimensional, such that one of the classes is “above” the hyperplane and the other class is “below” it. The hyperplane then serves as the classification boundary between the two classes. Mathematically, representing the hyperplane as $\{\mathbf{x} \in \mathbb{R}^d : \mathbf{w}^T \mathbf{x} + b = 0\}$ with parameters $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$, we

solve

$$\operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}} \|\mathbf{w}\| \quad \text{such that} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

The condition $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ ensures that the sample $\mathbf{x}_i$ is “above” (resp. “below”) the hyperplane if $y_i = 1$ (resp. $y_i = -1$). The band that separates the two classes, known as the *margin*, has width $\frac{2}{\|\mathbf{w}\|}$ (Hastie et al., 2009, p. 418). By minimising $\|\mathbf{w}\|$, we maximise the width of the margin.

However, *perfect separation* by a hyperplane is often impossible in practice (the classes are not *linearly separable*; e.g., if they partially overlap), in which case we relax the separation condition by allowing some (but ideally a small number) of the samples remain on the wrong side of the hyperplane (i.e., become *misclassified*). This gives rise to the *soft-margin* SVM classifier (Schölkopf et al., 2000; Chen et al., 2005), which solves

$$\begin{aligned} \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}, \rho \geq 0, \xi_1, \dots, \xi_n \geq 0} \quad & \frac{1}{2} \|\mathbf{w}\|^2 - \nu \rho + \frac{1}{2} \sum_{i=1}^n \xi_i \\ \text{such that} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq \rho - \xi_i, \quad i = 1, \dots, n. \end{aligned}$$

The hyperparameter $\nu \in [0, 1]$ controls to what degree the samples are allowed to be misclassified — the lower the value of ν is, the less misclassification is allowed. The degree of misclassification is reflected in the threshold ρ (the width of the margin is $\frac{2\rho}{\|\mathbf{w}\|}$) and in the *slack variables* $\xi_1, \dots, \xi_n$. (There are alternative parameterisations of the soft-margin SVM classifier; see, e.g., Hastie et al. (2009, p. 420).)

The SVM classifier can further be generalised to accommodate non-linearities by applying a *feature map* $\mathbf{f} : \mathbb{R}^p \rightarrow \mathbb{R}^q$ to the samples $\mathbf{x}_1, \dots, \mathbf{x}_n$. Combining this with the soft-margin version of SVM, we get

$$\begin{aligned} \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^q, b \in \mathbb{R}, \rho \geq 0, \xi_1, \dots, \xi_n \geq 0} \quad & \frac{1}{2} \|\mathbf{w}\|^2 - \nu \rho + \frac{1}{2} \sum_{i=1}^n \xi_i \\ \text{such that} \quad & y_i(\mathbf{w}^T \mathbf{f}(\mathbf{x}_i) + b) \geq \rho - \xi_i, \quad i = 1, \dots, n. \end{aligned} \tag{4.4}$$

Like in *kernel PCA*, we can apply the *kernel trick* to SVM (Cortes and Vapnik, 1995), specifying the feature map $\mathbf{f}$ via a kernel $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$\mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{x}') = k(\mathbf{x}, \mathbf{x}'), \quad \mathbf{x}, \mathbf{x}' \in \mathbb{R}^d.$$

The standard kernels, seen in Section 1.4, are all used for this purpose too. The optimisation problem (4.4) can then be solved using the kernel k without having to compute the feature map $\mathbf{f}$; for details, we refer to Hastie et al. (2009, Sect. 12.3).

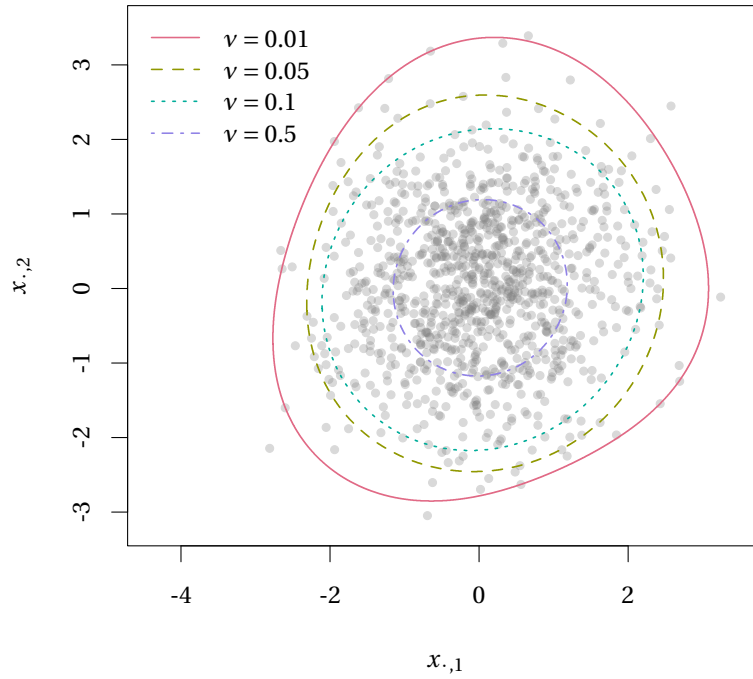


Figure 4.2: Classification boundaries of one-class SVMs applied to a simulated bivariate data set consisting of 1000 samples from $N(\mathbf{0}, I_2)$ with $\nu = 0.01, 0.05, 0.1, 0.5$. The SVMs use the radial basis function kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-0.1\|\mathbf{x} - \mathbf{x}'\|^2)$.

Returning to unsupervised learning and outlier and anomaly detection, we can apply the SVM to estimate an *approximate boundary* for a data set $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ (Schölkopf et al., 2001). The boundary can then be used to detect outliers and anomalies. This approach is called the *one-class SVM*. The one-class SVM is obtained by removing the label y_i and the intercept term b , such that

$$\begin{aligned} \argmin_{\mathbf{w} \in \mathbb{R}^q, \rho \geq 0, \xi_1, \dots, \xi_n \geq 0} & \frac{1}{2} \|\mathbf{w}\|^2 + \frac{1}{\nu n} \sum_{i=1}^n \xi_i - \rho \\ \text{such that} & \quad \mathbf{w}^T \mathbf{f}(\mathbf{x}_i) \geq \rho - \xi_i, \quad i = 1, \dots, n, \end{aligned}$$

which is solvable using the same techniques as kernel SVM. *Classification* of any point $\mathbf{x} \in \mathbb{R}^d$ is then performed via

$$g(\mathbf{x}) := \text{sign}(\mathbf{w}^T \mathbf{f}(\mathbf{x}) - \rho) = \begin{cases} 1, & \mathbf{x} \text{ is an inlier,} \\ 0, & \mathbf{x} \text{ is on the classification boundary,} \\ -1, & \mathbf{x} \text{ is an outlier.} \end{cases} \quad (4.5)$$

Note that $\mathbf{w}^T \mathbf{f}(\mathbf{x})$ can be computed using the kernel k , like in kernel PCA (see

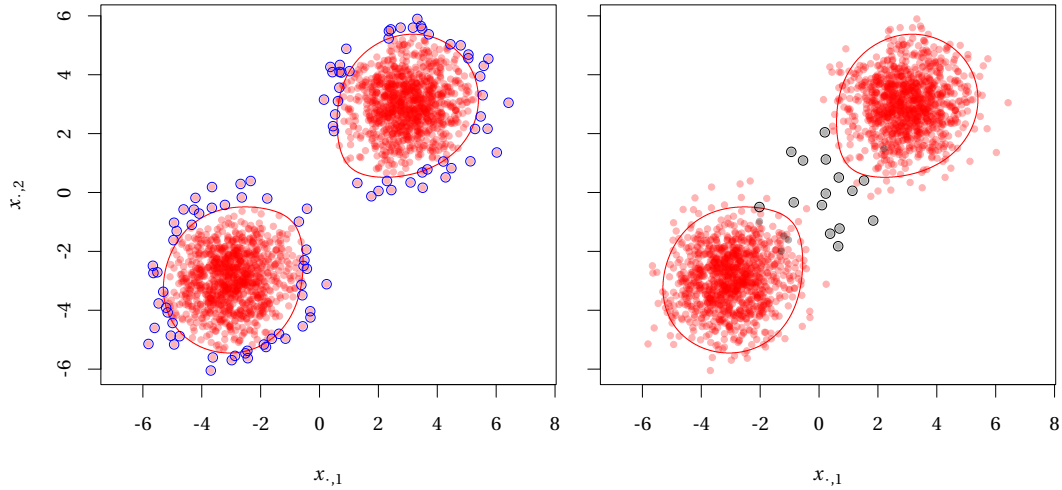


Figure 4.3: The one-class SVM applied to the data sets of Figure 4.1 with $\nu = 0.05$ and the radial basis function kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2)$. *Left:* Outlier detection applied to the original 2000 samples. Outliers outside the classification boundary are highlighted in blue. *Right:* Anomaly detection for the new 20 samples, using the one-class SVM trained on the original samples. Anomalous new samples are highlighted by black circles.

Section 1.4). The hyperparameter $\nu > 0$ controls the *sensitivity* of detecting outliers — the smaller the value of ν is, the less outliers are detected; see Figure 4.2.

After the one-class SVM has been trained, outlier detection is performed simply by applying (4.5). Anomaly detection is performed similarly by applying (4.5), but *not* training the SVM on the new samples. Application of the one-class SVM to the same simulated data sets that were analysed using LOF in Section 4.2 is demonstrated in Figure 4.3. The results are reassuringly similarly to those using LOF.

Bibliography

- T. W. Anderson and I. Olkin (1985): Maximum-likelihood estimation of the parameters of a multivariate normal distribution. *Linear Algebra and its Applications* **70**, 147–171.
- R. E. Bellman (1957): *Dynamic Programming*. Princeton University Press, Princeton.
- C. M. Bishop (2006): *Pattern Recognition and Machine Learning*. Springer, New York.
- B. E. Boser, I. M. Guyon and V. N. Vapnik (1992): A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory (COLT '92)*, pp. 144–152.
- H. Bourlard and Y. Kamp (1988): Auto-association by multilayer perceptrons and singular value decomposition. *Biological Cybernetics* **59**, 291–294.
- M. M. Breunig, H.-P. Kriegel, R. T. Ng and J. Sander (2000): LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, vol. 29, pp. 93–104.
- L. Chen and A. Buja (2009): Local multidimensional scaling for nonlinear dimension reduction, graph drawing, and proximity analysis. *Journal of the American Statistical Association* **104**(485), 209–219.
- P.-H. Chen, C.-J. Lin and B. Schölkopf (2005): A tutorial on ν -support vector machines. *Applied Stochastic Models in Business and Industry* **21**, 111–136.
- P. Comon (1994): Independent component analysis, A new concept? *Signal Processing* **36**(3), 287–314.
- C. Cortes and V. Vapnik (1995): Support-vector networks. *Machine Learning* **20**, 273–297.
- M. J. Crowder (1976): Maximum likelihood estimation for dependent observations. *Journal of the Royal Statistical Society. Series B (Methodological)* **38**(1), 45–53.

- A. P. Dempster, N. M. Laird and D. B. Rubin (1977): Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B (Methodological)* **39**(1), 1–22.
- D. Donoho and V. Stodden (2003): When does non-negative matrix factorization give a correct decomposition into parts? In S. Thrun, L. Saul and B. Schölkopf (eds): *Advances in Neural Information Processing Systems, vol. 16*, 8 pp.
- D. Freedman and P. Diaconis (1981): On the histogram as a density estimator: L_2 theory. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* **57**, 453–476.
- A. J. Gates and Y.-Y. Ahn (2017): The impact of random models on clustering similarity. *Journal of Machine Learning Research* **18**(87), 1–28.
- J. A. Hartigan and M. A. Wong (1979): Algorithm AS 136: A K -means clustering algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)* **28**(1), 100–108.
- T. Hastie, R. Tibshirani and J. Friedman (2009): *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edition. Springer, New York.
- G. E. Hinton and S. T. Roweis (2002): Stochastic neighbor embedding. In S. Becker, S. Thrun and K. Obermayer (eds): *Advances in Neural Information Processing Systems, vol. 15*, 8 pp.
- H. Hotelling (1933): Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology* **24**(6), 417–441.
- H. Hotelling (1936): Relations between two sets of variates. *Biometrika* **28**(3/4), 321–377.
- L. Hubert and P. Arabie (1985): Comparing partitions. *Journal of Classification* **2**, 193–218.
- R. J. Hyndman (1995): The problem with Sturges’ rule for constructing histograms. Preprint, available from <https://robjhyndman.com/papers/sturges.pdf>
- A. Hyvärinen and E. Oja (2000): Independent component analysis: algorithms and applications. *Neural Networks* **13**(4/5), 411–430.

- I. M. Johnstone and D. Paul (2018): PCA in high dimensions: An orientation. *Proceedings of the IEEE* **106**(8), 1277–1292.
- I. T. Jolliffe, N. T. Trendafilov and M. Uddin (2003): A modified principal component technique based on the LASSO. *Journal of Computational and Graphical Statistics* **12**(3), 531–547.
- L. Kaufman, P. J. Rousseeuw (1990): *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, New York.
- M. A. Kramer (1991): Nonlinear principal component analysis using autoassociative neural networks. *AIChE Journal* **37**(2), 233–243.
- H.-P. Kriegel, P. Kroger, E. Schubert and A. Zimek (2011): Interpreting and unifying outlier scores. In *Proceedings of the 2011 SIAM International Conference on Data Mining (SDM)*, pp. 13–24.
- J. B. Kruskal (1964) Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika* **29**, 1–27.
- S. Lang (1995): *Differential and Riemannian Manifolds*. Springer, New York.
- D. Lee and H. Seung (1999): Learning the parts of objects by non-negative matrix factorization. *Nature* **401**, 788–791.
- D. Lee and H. S. Seung (2000): Algorithms for non-negative matrix factorization. In T. Leen, T. Dietterich and V. Tresp (eds): *Advances in Neural Information Processing Systems, vol. 13*, 7 pp.
- S. Lloyd (1982): Least squares quantization in PCM. *IEEE Transactions on Information Theory* **28**(2), 129–137.
- E. Parzen (1962): On estimation of a probability density function and mode. *The Annals of Mathematical Statistics* **33**(3), 1065–1076.
- K. Pearson (1901): On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* **2**(11), 559–572.
- W. M. Rand (1971): Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association* **66**(336), 846–850.

- M. Rosenblatt (1956): Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics* **27**(3), 832–837.
- P. J. Rousseeuw (1987): Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* **20**, 53–65.
- S. T. Roweis and L. K. Saul (2000): Nonlinear dimensionality reduction by locally linear embedding. *Science* **290**(5500), 2323–2326.
- J. W. Sammon (1969): A nonlinear mapping for data structure analysis. *IEEE Transactions on Computers* **C-18**(5), 401–409.
- B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola and R. C. Williamson (2001): Estimating the support of a high-dimensional distribution. *Neural Computation* **13**(7), 1443–1471.
- B. Schölkopf, A. Smola and K. R. Müller (1997): Kernel principal component analysis. In W. Gerstner, A. Germond, M. Hasler and J. D. Nicoud (eds): *Artificial Neural Networks — ICANN'97, Lecture Notes in Computer Science, vol. 1327*, pp. 583–588.
- B. Schölkopf, A. Smola and K.-R. Müller (1998): Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation* **10**(5), 1299–1319.
- B. Schölkopf, A. J. Smola, R. C. Williamson and P. L. Bartlett (2000): New support vector algorithms. *Neural Computation* **12**(5), 1207–1245.
- D. W. Scott (1979): On optimal and data-based histograms. *Biometrika* **66**(3), 605–610.
- D. W. Scott (1992): *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley, New York.
- S. J. Sheather (2004): Density estimation. *Statistical Science* **19**(4), 588–597.
- B. W. Silverman (1986): *Density Estimation for Statistics and Data Analysis*. Chapman & Hall, London.
- R. R. Sokal and C. D. Michener (1958): A statistical method for evaluating systematic relationships. *University of Kansas Science Bulletin* **38**, 1409–1438.

- H. A. Sturges (1926): The choice of a class interval. *Journal of the American Statistical Association* **21**(153), 65–66.
- Joshua B. Tenenbaum, Vin De Silva and John C. Langford (2000): A global geometric framework for nonlinear dimensionality reduction. *Science* **290**(5500), 2319–2323.
- R. Tibshirani, G. Walther and T. Hastie (2001): Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)* **63**(2), 411–423.
- W. S. Torgerson (1958): *Theory & Methods of Scaling*. Wiley, New York.
- L. J. P. van der Maaten and G. E. Hinton (2008): Visualizing data using t-SNE. *Journal of Machine Learning Research* **9**(86), 2579–2605.
- L. J. P. van der Maaten (2009): Learning a parametric embedding by preserving local structure. In *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 384–391.
- A. van der Vaart (1998): *Asymptotic Statistics*. Cambridge University Press, Cambridge.
- N. X. Vinh, J. Epps and J. Bailey (2010): Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research* **11**(95), 2837–2854.
- Wikipedia (2023): *Canonical correlation*. Accessed: 20 September 2023.
- H. Zou, T. Hastie and R. Tibshirani (2006): Sparse principal component analysis. *Journal of Computational and Graphical Statistics* **15**(2), 265–286.